Script to validate points in ReSurvey database using RS data (S2 + SatEmb + Landsat)

Fitting Random Forest models with ntree = 1500, excluding manipulated observations

Alicia Valdés

29 enero 2026

This R script is used to validate the points in the ReSurvey database using either RS indicators from S2 or Landsat (indices + phenology + canopy height), or bands from the Google Satellite Embedding dataset: https://developers.google.com/earth-engine/datasets/catalog/GOOGLE_SATELLITE_EMBEDDING_V1_ANNUAL#description Here, we filter out manipulated observations to avoid using them in analyses.

Load libraries

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.1     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.2.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(here)
```

```
## here() starts at C:/Users/jimenezalfaro/OneDrive - Universidad de Oviedo/IMIB/Analyses/MOTIVATE/MOTI
```

```r
library(parallel)
library(doParallel)
```

```
## Cargando paquete requerido: foreach
##
## Adjuntando el paquete: 'foreach'
##
## The following objects are masked from 'package:purrr':
##
##     accumulate, when
##
## Cargando paquete requerido: iterators
```

```r
library(moreparty)
```

```
## Cargando paquete requerido: party
## Cargando paquete requerido: grid
## Cargando paquete requerido: mvtnorm
## Cargando paquete requerido: modeltools
## Cargando paquete requerido: stats4
## Cargando paquete requerido: strucchange
## Cargando paquete requerido: zoo
##
## Adjuntando el paquete: 'zoo'
##
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
##
## Cargando paquete requerido: sandwich
##
## Adjuntando el paquete: 'strucchange'
##
## The following object is masked from 'package:stringr':
##
##     boundary
##
##
## Adjuntando el paquete: 'party'
##
## The following object is masked from 'package:dplyr':
##
##     where
```

```r
library(caret)
```

```
## Cargando paquete requerido: lattice
##
## Adjuntando el paquete: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Adjuntando el paquete: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```r
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Adjuntando el paquete: 'pROC'
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```r
library(rlang)
```

```
##
## Adjuntando el paquete: 'rlang'
##
## The following objects are masked from 'package:purrr':
##
##     flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
##     flatten_raw, invoke, splice
```

```r
library(stringr)
library(beepr)
library(permimp)
library(purrr)
library(gghalves)
library(patchwork)
```

Load functions

```r
knitr::purl("functions_20260122.Rmd", output = "functions_20260122.R")
```

```
## [1] "functions_20260122.R"
```

```r
source("functions_20260122.R")
```

Load previously created objects

```r
# Define the folder path
folder_path <- here("objects", "RF_1500_20260128")

# List all .RData or .rda files in the folder
```

```r
rdata_files <- list.files(folder_path, full.names = TRUE)

# Load each file
lapply(rdata_files, load, envir = .GlobalEnv)
```

```
## [[1]]
## [1] "filtered_data_both_Landsat_with_probs"
##
## [[2]]
## [1] "filtered_data_both_Landsat_with_probs_2x"
##
## [[3]]
## [1] "filtered_data_both_S2_with_probs"
##
## [[4]]
## [1] "filtered_data_both_S2_with_probs_2x"
##
## [[5]]
## [1] "filtered_data_months_Landsat_with_probs"
##
## [[6]]
## [1] "filtered_data_months_Landsat_with_probs_2x"
##
## [[7]]
## [1] "filtered_data_months_S2_with_probs"
##
## [[8]]
## [1] "filtered_data_months_S2_with_probs_2x"
##
## [[9]]
## [1] "filtered_data_SatEmb_CH_with_probs"
##
## [[10]]
## [1] "filtered_data_SatEmb_CH_with_probs_2x"
##
## [[11]]
## [1] "filtered_data_SatEmb_with_probs"
##
## [[12]]
## [1] "filtered_data_SatEmb_with_probs_2x"
##
## [[13]]
## [1] "filtered_data_slope_Landsat_with_probs"
##
## [[14]]
## [1] "filtered_data_slope_Landsat_with_probs_2x"
##
## [[15]]
## [1] "filtered_data_slope_S2_with_probs"
##
## [[16]]
## [1] "filtered_data_slope_S2_with_probs_2x"
##
```

```
## [[17]]
## [1] "rf_both_Landsat"
##
## [[18]]
## [1] "rf_both_Landsat_prediction"
##
## [[19]]
## [1] "rf_both_Landsat_refin_20_10_2x"
##
## [[20]]
## [1] "rf_both_Landsat_refin_20_10_2x_prediction"
##
## [[21]]
## [1] "rf_both_Landsat_refin_20_10"
##
## [[22]]
## [1] "rf_both_Landsat_refin_20_10_prediction"
##
## [[23]]
## [1] "rf_both_S2"
##
## [[24]]
## [1] "rf_both_S2_prediction"
##
## [[25]]
## [1] "rf_both_S2_refin_20_10_2x"
##
## [[26]]
## [1] "rf_both_S2_refin_20_10_2x_prediction"
##
## [[27]]
## [1] "rf_both_S2_refin_20_10"
##
## [[28]]
## [1] "rf_both_S2_refin_20_10_prediction"
##
## [[29]]
## [1] "rf_months_Landsat"
##
## [[30]]
## [1] "rf_months_Landsat_prediction"
##
## [[31]]
## [1] "rf_months_Landsat_refin_20_10_2x"
##
## [[32]]
## [1] "rf_months_Landsat_refin_20_10_2x_prediction"
##
## [[33]]
## [1] "rf_months_Landsat_refin_20_10"
##
## [[34]]
## [1] "rf_months_Landsat_refin_20_10_prediction"
##
```

```
## [[35]]
## [1] "rf_months_S2"
##
## [[36]]
## [1] "rf_months_S2_prediction"
##
## [[37]]
## [1] "rf_months_S2_refin_20_10_2x"
##
## [[38]]
## [1] "rf_months_S2_refin_20_10_2x_prediction"
##
## [[39]]
## [1] "rf_months_S2_refin_20_10"
##
## [[40]]
## [1] "rf_months_S2_refin_20_10_prediction"
##
## [[41]]
## [1] "rf_SatEmb_CH"
##
## [[42]]
## [1] "rf_SatEmb_CH_prediction"
##
## [[43]]
## [1] "rf_SatEmb_CH_refin_20_10_2x"
##
## [[44]]
## [1] "rf_SatEmb_CH_refin_20_10_2x_prediction"
##
## [[45]]
## [1] "rf_SatEmb_CH_refin_20_10"
##
## [[46]]
## [1] "rf_SatEmb_CH_refin_20_10_prediction"
##
## [[47]]
## [1] "rf_SatEmb"
##
## [[48]]
## [1] "rf_SatEmb_prediction"
##
## [[49]]
## [1] "rf_SatEmb_refin_20_10_2x"
##
## [[50]]
## [1] "rf_SatEmb_refin_20_10_2x_prediction"
##
## [[51]]
## [1] "rf_SatEmb_refin_20_10"
##
## [[52]]
## [1] "rf_SatEmb_refin_20_10_prediction"
##
```

```
## [[53]]
## [1] "rf_slope_Landsat"
##
## [[54]]
## [1] "rf_slope_Landsat_prediction"
##
## [[55]]
## [1] "rf_slope_Landsat_refin_20_10_2x"
##
## [[56]]
## [1] "rf_slope_Landsat_refin_20_10_2x_prediction"
##
## [[57]]
## [1] "rf_slope_Landsat_refin_20_10"
##
## [[58]]
## [1] "rf_slope_Landsat_refin_20_10_prediction"
##
## [[59]]
## [1] "rf_slope_S2"
##
## [[60]]
## [1] "rf_slope_S2_prediction"
##
## [[61]]
## [1] "rf_slope_S2_refin_20_10_2x"
##
## [[62]]
## [1] "rf_slope_S2_refin_20_10_2x_prediction"
##
## [[63]]
## [1] "rf_slope_S2_refin_20_10"
##
## [[64]]
## [1] "rf_slope_S2_refin_20_10_prediction"
```

Read data

```r
data_validation_S2 <- read_tsv(here(
  "data", "clean","final_RS_data_bands_S2_all_20251021.csv"))
```

```
## Rows: 36552 Columns: 137
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr   (7): EUNISa_1, EUNISa_1_descr, EUNISa_2, EUNISa_2_descr, biogeo, unit,...
## dbl (130): PlotObservationID, EVI_auc_mar_oct, EVI_auc_slope, EVI_auc_thresh...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
data_validation_SatEmb <- read_tsv(here(
  "data", "clean","final_RS_data_SatEmb_20251010.csv")) %>%
  rename(EUNISa_1_descr = EUNIS_1_d)
```

```
## Rows: 36552 Columns: 115
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr  (31): Country, Dataset, Dtofrcr, EUNIS_1_d, EUNIS_2_d, EUNIS_3_d, EUNIS...
## dbl  (69): PlotObservationID, A00, A01, A02, A03, A04, A05, A06, A07, A08, A...
## lgl  (14): EUNIS_4_d, EUNISa_4, EUNISb_1, EUNISb_4, EUNISb_4_, EUNISc, EUNIS...
## date  (1): date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
data_validation_Landsat <- read_tsv(here(
  "data", "clean","final_RS_data_bands_Landsat_all_20251021.csv"))
```

```
## Rows: 175039 Columns: 137
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr   (7): EUNISa_1, EUNISa_1_descr, EUNISa_2, EUNISa_2_descr, biogeo, unit,...
## dbl (130): PlotObservationID, EVI_auc_mar_oct, EVI_auc_slope, EVI_auc_thresh...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

No parsing issues!

```r
nrow(data_validation_S2)
```

```
## [1] 36552
```

```r
nrow(data_validation_Landsat)
```

```
## [1] 175039
```

```r
nrow(data_validation_SatEmb)
```

```
## [1] 36552
```

Filter out manipulated observations

Read list of PlotObservationIDs to keep

```r
PlotObservationID_list <- read_csv(
  here("data", "clean","PlotObservationIDs_filtered_QRST_20260122.csv"))
```

```
## Rows: 165385 Columns: 1
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (1): PlotObservationID
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Filter out data to keep only those PlotObservationIDs

```
data_validation_S2 <- data_validation_S2 %>%
  filter(PlotObservationID %in% PlotObservationID_list$PlotObservationID)
data_validation_SatEmb <- data_validation_SatEmb %>%
  filter(PlotObservationID %in% PlotObservationID_list$PlotObservationID)
data_validation_Landsat <- data_validation_Landsat %>%
  filter(PlotObservationID %in% PlotObservationID_list$PlotObservationID)
```

```
nrow(data_validation_S2)
```

```
## [1] 31359
```

```
nrow(data_validation_Landsat)
```

```
## [1] 148130
```

```
nrow(data_validation_SatEmb)
```

```
## [1] 31359
```

Descriptives

Number of observations with S2 data:

```
data_validation_S2 %>%
  filter(if_any(c(2:86, 91:132), ~ !is.na(.))) %>%
  nrow()
```

```
## [1] 31308
```

Number of observations with Landsat data:

```
data_validation_Landsat %>%
  filter(if_any(c(2:86, 91:132), ~ !is.na(.))) %>%
  nrow()
```

```
## [1] 103316
```

Number of observations with SatEmb data:

```r
data_validation_SatEmb %>%
  filter(if_any(c(2:65), ~ !is.na(.))) %>%
  nrow()
```

```
## [1] 31359
```

Define lists of predictor vars

S2 and Landsat

Moments with slope method

Not using threshold method cause it gave worse results earlier.

```r
vars_RF_slope <- c(
  # Min values of all indices
  "NDVI_min", "EVI_min", "NDMI_min", "NDWI_min", "SAVI_min",
  # Max values of NDMI and NDWI
  "NDMI_max", "NDWI_max",
  # AUC of NDVI, EVI and SAVI
  "NDVI_auc_slope", "EVI_auc_slope", "SAVI_auc_slope",
  # Values of NDVI, EVI and SAVI at sos, pos and eos
  "NDVI_sos_slope_value", "NDVI_pos_value", "NDVI_eos_slope_value",
  "EVI_sos_slope_value", "EVI_pos_value", "EVI_eos_slope_value",
  "SAVI_sos_slope_value", "SAVI_pos_value", "SAVI_eos_slope_value",
  # Differences pos-sos in value and doy
  "NDVI_diff_pos_sos_slope_value", "EVI_diff_pos_sos_slope_value",
  "SAVI_diff_pos_sos_slope_value","NDVI_diff_pos_sos_slope_doy",
  "EVI_diff_pos_sos_slope_doy", "SAVI_diff_pos_sos_slope_doy",
  # Differences pos-eos in value and doy
  "NDVI_diff_pos_eos_slope_value", "EVI_diff_pos_eos_slope_value",
  "SAVI_diff_pos_eos_slope_value", "NDVI_diff_pos_eos_slope_doy",
  "EVI_diff_pos_eos_slope_doy", "SAVI_diff_pos_eos_slope_doy",
  # Growing season duration
  "NDVI_slope_gsd", "EVI_slope_gsd", "SAVI_slope_gsd",
  # Canopy height
  "canopy_height")
```

```r
# Excluding EVI because many values outside of range -1, 1
vars_RF_slope_Landsat <- c(
  # Min values of all indices
  "NDVI_min", "NDMI_min", "NDWI_min", "SAVI_min",
  # Max values of NDMI and NDWI
  "NDMI_max", "NDWI_max",
  # AUC of NDVI and SAVI
  "NDVI_auc_slope", "SAVI_auc_slope",
  # Values of NDVI and SAVI at sos, pos and eos
  "NDVI_sos_slope_value", "NDVI_pos_value", "NDVI_eos_slope_value",
  "SAVI_sos_slope_value", "SAVI_pos_value", "SAVI_eos_slope_value",
  # Differences pos-sos in value and doy
  "NDVI_diff_pos_sos_slope_value","SAVI_diff_pos_sos_slope_value",
```

```
  "NDVI_diff_pos_sos_slope_doy", "SAVI_diff_pos_sos_slope_doy",
  # Differences pos-eos in value and doy
  "NDVI_diff_pos_eos_slope_value", "SAVI_diff_pos_eos_slope_value",
  "NDVI_diff_pos_eos_slope_doy", "SAVI_diff_pos_eos_slope_doy",
  # Growing season duration
  "NDVI_slope_gsd", "SAVI_slope_gsd",
  # Canopy height
  "canopy_height")
```

Months method

```
vars_RF_months <- c(
  # Min values of all indices
  "NDVI_min", "EVI_min", "NDMI_min", "NDWI_min", "SAVI_min",
  # Max values of NDMI and NDWI
  "NDMI_max", "NDWI_max",
  # AUC of NDVI, EVI and SAVI between march and oct
  "NDVI_auc_mar_oct", "EVI_auc_mar_oct", "SAVI_auc_mar_oct",
  # Values of NDVI, EVI and SAVI at march, pos and oct
  "NDVI_avg_value_03", "NDVI_pos_value", "NDVI_avg_value_10",
  "EVI_avg_value_03", "EVI_pos_value", "EVI_avg_value_10",
  "SAVI_avg_value_03", "SAVI_pos_value", "SAVI_avg_value_10",
  # Differences pos-march in value and doy
  "NDVI_diff_pos_march_value", "EVI_diff_pos_march_value",
  "SAVI_diff_pos_march_value","NDVI_diff_pos_march_doy",
  "EVI_diff_pos_march_doy", "SAVI_diff_pos_march_doy",
  # Differences pos-oct in value and doy
  "NDVI_diff_pos_oct_value", "EVI_diff_pos_oct_value",
  "SAVI_diff_pos_oct_value", "NDVI_diff_pos_oct_doy",
  "EVI_diff_pos_oct_doy", "SAVI_diff_pos_oct_doy",
  # Canopy height
  "canopy_height")
```

```
# Excluding EVI because many values outside of range -1, 1
vars_RF_months_Landsat <- c(
  # Min values of all indices
  "NDVI_min", "NDMI_min", "NDWI_min", "SAVI_min",
  # Max values of NDMI and NDWI
  "NDMI_max", "NDWI_max",
  # AUC of NDVI, and SAVI between march and oct
  "NDVI_auc_mar_oct", "SAVI_auc_mar_oct",
  # Values of NDVI and SAVI at march, pos and oct
  "NDVI_avg_value_03", "NDVI_pos_value", "NDVI_avg_value_10",
  "SAVI_avg_value_03", "SAVI_pos_value", "SAVI_avg_value_10",
  # Differences pos-march in value and doy
  "NDVI_diff_pos_march_value", "SAVI_diff_pos_march_value",
  "NDVI_diff_pos_march_doy", "SAVI_diff_pos_march_doy",
  # Differences pos-oct in value and doy
  "NDVI_diff_pos_oct_value", "SAVI_diff_pos_oct_value",
  "NDVI_diff_pos_oct_doy", "SAVI_diff_pos_oct_doy",
  # Canopy height
  "canopy_height")
```

Both

Slope + months (Borja's suggestion).

```
vars_RF_both <- c(
  # Min values of all indices
  "NDVI_min", "EVI_min", "NDMI_min", "NDWI_min", "SAVI_min",
  # Max values of NDMI and NDWI
  "NDMI_max", "NDWI_max",
  # AUC of NDVI, EVI and SAVI
  "NDVI_auc_slope", "EVI_auc_slope", "SAVI_auc_slope",
  # AUC of NDVI, EVI and SAVI between march and oct
  "NDVI_auc_mar_oct", "EVI_auc_mar_oct", "SAVI_auc_mar_oct",
  # Values of NDVI, EVI and SAVI at sos, pos and eos
  "NDVI_sos_slope_value", "NDVI_pos_value", "NDVI_eos_slope_value",
  "EVI_sos_slope_value", "EVI_pos_value", "EVI_eos_slope_value",
  "SAVI_sos_slope_value", "SAVI_pos_value", "SAVI_eos_slope_value",
  # Differences pos-sos in value and doy
  "NDVI_diff_pos_sos_slope_value", "EVI_diff_pos_sos_slope_value",
  "SAVI_diff_pos_sos_slope_value","NDVI_diff_pos_sos_slope_doy",
  "EVI_diff_pos_sos_slope_doy", "SAVI_diff_pos_sos_slope_doy",
  # Differences pos-eos in value and doy
  "NDVI_diff_pos_eos_slope_value", "EVI_diff_pos_eos_slope_value",
  "SAVI_diff_pos_eos_slope_value", "NDVI_diff_pos_eos_slope_doy",
  "EVI_diff_pos_eos_slope_doy", "SAVI_diff_pos_eos_slope_doy",
  # Growing season duration
  "NDVI_slope_gsd", "EVI_slope_gsd", "SAVI_slope_gsd",
  # Values of NDVI, EVI and SAVI at march and oct
  "NDVI_avg_value_03", "NDVI_avg_value_10",
  "EVI_avg_value_03", "EVI_avg_value_10",
  "SAVI_avg_value_03", "SAVI_avg_value_10",
  # Differences pos-march in value and doy
  "NDVI_diff_pos_march_value", "EVI_diff_pos_march_value",
  "SAVI_diff_pos_march_value","NDVI_diff_pos_march_doy",
  "EVI_diff_pos_march_doy", "SAVI_diff_pos_march_doy",
  # Differences pos-oct in value and doy
  "NDVI_diff_pos_oct_value", "EVI_diff_pos_oct_value",
  "SAVI_diff_pos_oct_value", "NDVI_diff_pos_oct_doy",
  "EVI_diff_pos_oct_doy", "SAVI_diff_pos_oct_doy",
  # Canopy height
  "canopy_height")
```

```
# Excluding EVI because many values outside of range -1, 1
vars_RF_both_Landsat <- c(
  # Min values of all indices
  "NDVI_min", "NDMI_min", "NDWI_min", "SAVI_min",
  # Max values of NDMI and NDWI
  "NDMI_max", "NDWI_max",
  # AUC of NDVI and SAVI
  "NDVI_auc_slope", "SAVI_auc_slope",
  # AUC of NDVI, and SAVI between march and oct
```

```
  "NDVI_auc_mar_oct", "SAVI_auc_mar_oct",
  # Values of NDVI and SAVI at sos, pos and eos
  "NDVI_sos_slope_value", "NDVI_pos_value", "NDVI_eos_slope_value",
  "SAVI_sos_slope_value", "SAVI_pos_value", "SAVI_eos_slope_value",
  # Differences pos-sos in value and doy
  "NDVI_diff_pos_sos_slope_value","SAVI_diff_pos_sos_slope_value",
  "NDVI_diff_pos_sos_slope_doy", "SAVI_diff_pos_sos_slope_doy",
  # Differences pos-eos in value and doy
  "NDVI_diff_pos_eos_slope_value", "SAVI_diff_pos_eos_slope_value",
  "NDVI_diff_pos_eos_slope_doy", "SAVI_diff_pos_eos_slope_doy",
  # Growing season duration
  "NDVI_slope_gsd", "SAVI_slope_gsd",
  # Values of NDVI and SAVI at march and oct
  "NDVI_avg_value_03", "NDVI_avg_value_10",
  "SAVI_avg_value_03", "SAVI_avg_value_10",
  # Differences pos-march in value and doy
  "NDVI_diff_pos_march_value", "SAVI_diff_pos_march_value",
  "NDVI_diff_pos_march_doy", "SAVI_diff_pos_march_doy",
  # Differences pos-oct in value and doy
  "NDVI_diff_pos_oct_value", "SAVI_diff_pos_oct_value",
  "NDVI_diff_pos_oct_doy", "SAVI_diff_pos_oct_doy",
  # Canopy height
  "canopy_height")
```

SatEmb

```
vars_RF_SatEmb <- colnames(
  data_validation_SatEmb)[startsWith(colnames(data_validation_SatEmb), "A")]
vars_RF_SatEmb_CH <- c(vars_RF_SatEmb, "canopy_height")
```

Filter data to use in RF models

Functions

```
filter_data_S2 <- function(data, variables) {
  data %>%
    mutate(EUNISa_1 = as.factor(EUNISa_1)) %>%
    # Remove rows with missing values in predictors
    dplyr::filter(if_all(all_of(variables), ~ !is.na(.))) %>%
    # Remove all rows with wrong values of indices (not between -1 and 1)
    dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
    dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
    dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
    dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
    dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
    # Select only variables needed
    dplyr::select(PlotObservationID, EUNISa_1, all_of(variables), Lctnmth,
           valid_1) %>%
```

```r
    # Keep only rows with differences > 0 in variables
    filter(if_all(any_of(variables[grepl("diff", variables)]),
                  ~ .x > 0)) %>%
    # Select only GPS points
    dplyr::filter(Lctnmth == "Location with GPS" |
                    Lctnmth == "Location with differential GPS")
}
```

```r
filter_data_Landsat <- function(data, variables) {
  data %>%
    mutate(EUNISa_1 = as.factor(EUNISa_1)) %>%
    # Remove rows with missing values in predictors
    dplyr::filter(if_all(all_of(variables), ~ !is.na(.))) %>%
    # Remove all rows with wrong values of indices (not between -1 and 1)
    dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
    # With Landsat data, many observations removed because of high/low EVI values
    # In Landsat_extraction, values of EVI outside of expected range were set as NA,
    # so better not to use EVI here
    # dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
    dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
    dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
    dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
    # Select only variables needed
    dplyr::select(PlotObservationID, EUNISa_1, all_of(variables), Lctnmth,
           valid_1) %>%
    # Keep only rows with differences > 0 in variables
    filter(if_all(any_of(variables[grepl("diff", variables)]),
                  ~ .x > 0)) %>%
    # Select only GPS points
    dplyr::filter(Lctnmth == "Location with GPS" |
                    Lctnmth == "Location with differential GPS")
}
```

```r
filter_data_SatEmb <- function(data, variables) {
  data %>%
    mutate(EUNISa_1 = as.factor(EUNISa_1)) %>%
    # Remove rows with missing values in predictors
    dplyr::filter(if_all(all_of(variables), ~ !is.na(.))) %>%
    # Select only variables needed
    dplyr::select(PlotObservationID, EUNISa_1, all_of(variables),
           Lctnmth, valid_1) %>%
    # Select only GPS points
    dplyr::filter(Lctnmth == "Location with GPS" |
                    Lctnmth == "Location with differential GPS")
  }
```

Filtered data

Select only GPS points that have passed the rough validation in each case.

For S2 and Landsat, use only slope and months methods, as threshold method has many missing values.

```r
# S2
filtered_data_slope_S2 <- data_validation_S2 %>%
  # Rough validation based on CH and NDWI
  rough_validation_S2()  %>%
  # Data filtering
  filter_data_S2(vars_RF_slope) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value")

filtered_data_months_S2 <- data_validation_S2 %>%
  # Rough validation based on CH and NDWI
  rough_validation_S2() %>%
  # Data filtering
  filter_data_S2(vars_RF_months) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value")

filtered_data_both_S2 <- data_validation_S2 %>%
  # Rough validation based on CH and NDWI
  rough_validation_S2() %>%
  # Data filtering
  filter_data_S2(vars_RF_both) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value")

# Landsat
filtered_data_slope_Landsat <- data_validation_Landsat %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  # Data filtering
  filter_data_Landsat(vars_RF_slope_Landsat) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value")

filtered_data_months_Landsat <- data_validation_Landsat %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  # Data filtering
  filter_data_Landsat(vars_RF_months_Landsat) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value")

filtered_data_both_Landsat <- data_validation_Landsat %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  # Data filtering
  filter_data_Landsat(vars_RF_both_Landsat) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value")

# SatEmb
filtered_data_SatEmb <- data_validation_SatEmb %>%
  # Rough validation based on CH
```

```r
  rough_validation_SatEmb() %>%
  # Join with S2 data to get NDWI_max info
  left_join(
    data_validation_S2 %>% select(PlotObservationID, NDWI_max)
  ) %>%
  # Rough validation based on NDWI (from S2)
  # Remove those with NDWI_max > 0.3, keep if NDWI_max is NA
  filter(NDWI_max <= 0.3 | is.na(NDWI_max)) %>%
  # Remove unneeded var
  select(-NDWI_max) %>%
  # Data filtering
  filter_data_SatEmb(vars_RF_SatEmb) %>%
  # Join with S2 data to get NDVI_pos_value info
  left_join(
    data_validation_S2 %>% select(PlotObservationID, NDVI_pos_value)
  ) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value") %>%
  # Remove unneeded var
  select(-NDVI_pos_value)
```

```
## Joining with `by = join_by(PlotObservationID)`
## Joining with `by = join_by(PlotObservationID)`
```

```r
filtered_data_SatEmb_CH <- data_validation_SatEmb %>%
  # Rough validation based on CH
  rough_validation_SatEmb() %>%
  # Join with S2 data to get NDWI_max info
  left_join(
    data_validation_S2 %>% select(PlotObservationID, NDWI_max)
  ) %>%
  # Rough validation based on NDWI (from S2)
  # Remove those with NDWI_max > 0.3, keep if NDWI_max is NA
  filter(NDWI_max <= 0.3 | is.na(NDWI_max)) %>%
  # Remove unneeded var
  select(-NDWI_max) %>%
  # Data filtering
  filter_data_SatEmb(vars_RF_SatEmb_CH) %>%
  # Join with S2 data to get NDVI_pos_value info
  left_join(
    data_validation_S2 %>% select(PlotObservationID, NDVI_pos_value)
  ) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value") %>%
  # Remove unneeded vars
  select(-NDVI_pos_value)
```

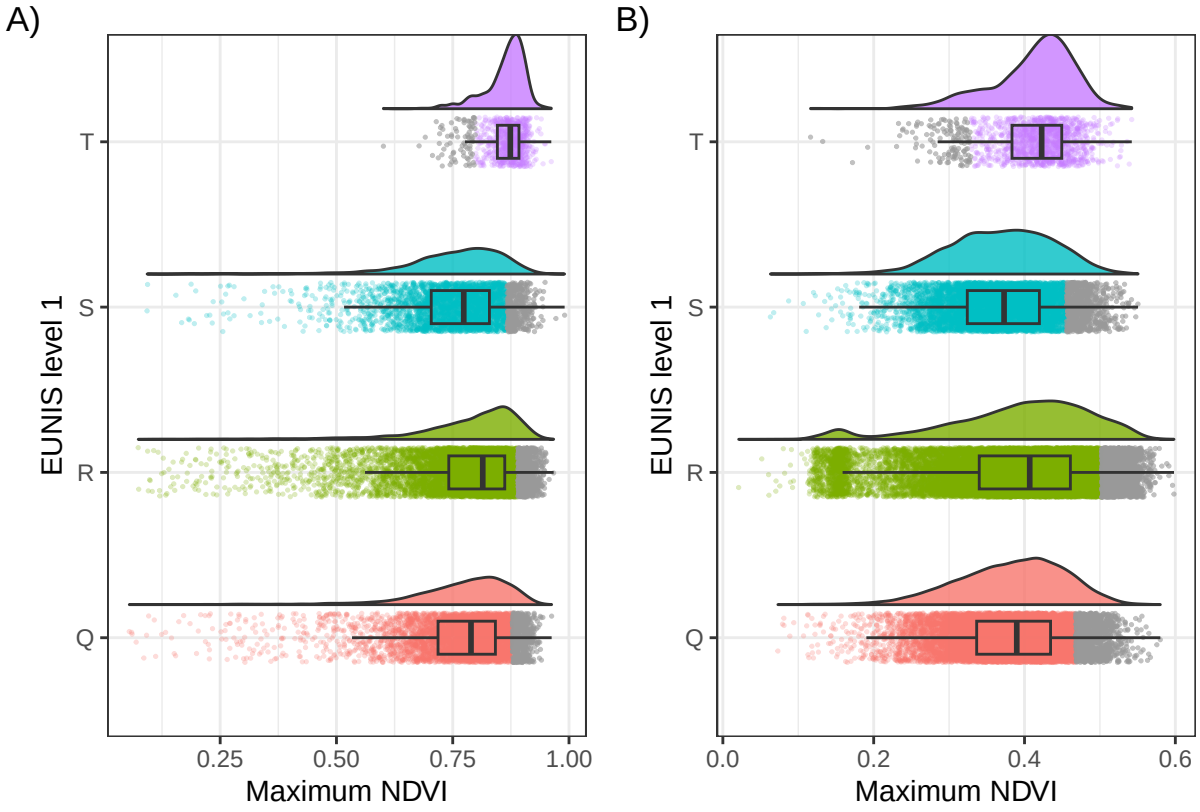
```
## Joining with `by = join_by(PlotObservationID)`
## Joining with `by = join_by(PlotObservationID)`
```

Plots

```
plot_perc_S2 <- data_validation_S2 %>%
  # Rough validation based on CH and NDWI
  rough_validation_S2() %>%
  # Data filtering
  filter_data_S2(vars_RF_months) %>%
  distr_plot("NDVI_pos_value", "Maximum NDVI", grey = T)
```

```
plot_perc_Landsat <- data_validation_Landsat %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  # Data filtering
  filter_data_Landsat(vars_RF_months_Landsat) %>%
  distr_plot("NDVI_pos_value", "Maximum NDVI", grey = T)
```

```
plot_perc_S2 + plot_perc_Landsat +
  plot_annotation(tag_levels = "A", tag_suffix = ")")
```



Fit RF models (level 1)

Without refinement

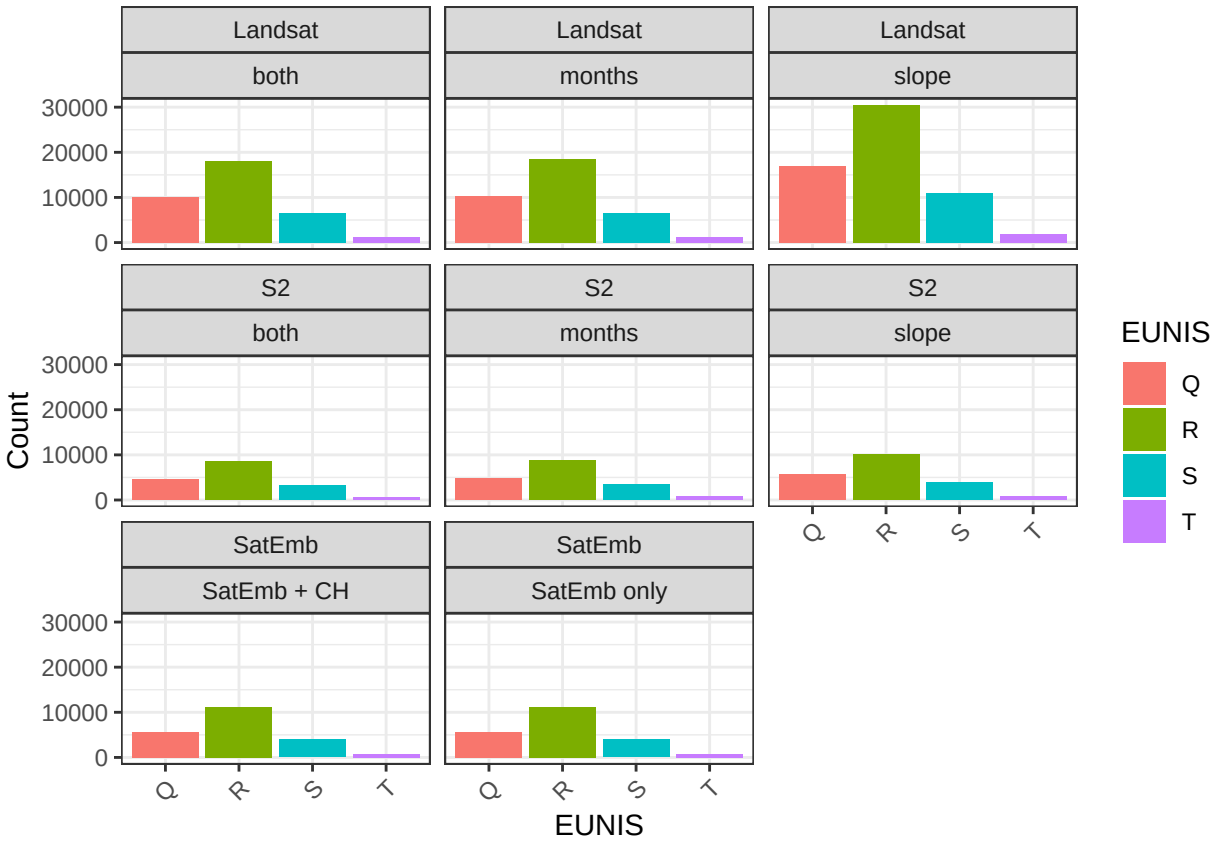
N points per category

```
bind_rows(
  # S2
  filtered_data_slope_S2 %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "S2", method = "slope") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_months_S2 %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "S2", method = "months") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_both_S2 %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "S2", method = "both") %>%
    select(sensor, method, Q, R, S, T),
  # # Landsat
  filtered_data_slope_Landsat %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "slope") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_months_Landsat %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "months") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_both_Landsat %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "both") %>%
    select(sensor, method, Q, R, S, T),
  # SatEmb
  filtered_data_SatEmb %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "SatEmb", method ="SatEmb only") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_SatEmb_CH %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "SatEmb", method ="SatEmb + CH") %>%
    select(sensor, method, Q, R, S, T)
)
```

```
## # A tibble: 8 x 6
##   sensor  method          Q     R     S     T
##   <chr>   <chr>       <int> <int> <int> <int>
## 1 S2      slope        5543 10081  3861   688
## 2 S2      months       4679  8668  3364   694
## 3 S2      both         4617  8565  3313   649
## 4 Landsat slope       16897 30426 11079  1864
## 5 Landsat months      10338 18559  6621  1175
```

```
## 6 Landsat both        10068 18149  6444  1171
## 7 SatEmb  SatEmb only  5733 11141  3979   738
## 8 SatEmb  SatEmb + CH  5733 11141  3979   738
```

```r
bind_rows(
  # S2
  filtered_data_slope_S2 %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "S2", method = "slope") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_months_S2 %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "S2", method = "months") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_both_S2 %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "S2", method = "both") %>%
    select(sensor, method, Q, R, S, T),
  # # Landsat
  filtered_data_slope_Landsat %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "slope") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_months_Landsat %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "months") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_both_Landsat %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "Landsat", method = "both") %>%
    select(sensor, method, Q, R, S, T),
  # SatEmb
  filtered_data_SatEmb %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "SatEmb", method ="SatEmb only") %>%
    select(sensor, method, Q, R, S, T),
  filtered_data_SatEmb_CH %>% count(EUNISa_1) %>%
    pivot_wider(names_from = EUNISa_1, values_from = n) %>%
    mutate(sensor = "SatEmb", method ="SatEmb + CH") %>%
    select(sensor, method, Q, R, S, T)
) %>%
  pivot_longer(cols = 3:6, names_to = "EUNIS", values_to = "Count") %>%
  ggplot(aes(x = EUNIS, y = Count, fill = EUNIS, group = EUNIS)) +
  geom_col() +
  facet_wrap(~ sensor + method) + theme_bw() +
  # ggtitle("Reduction in number of observations with filtering steps") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Split into training and test data sets

Split data:

```
# S2
# Slope
set.seed(123)
split_data_slope_S2 <- split_train_test(filtered_data_slope_S2)

train_data_slope_S2 <- split_data_slope_S2$train
test_data_slope_S2 <- split_data_slope_S2$test

# Check overlap between training and testing datasets
sum(train_data_slope_S2$PlotObservationID %in% test_data_slope_S2$PlotObservationID)
```

```
## [1] 0
```

```
# Months
set.seed(123)
split_data_months_S2 <- split_train_test(filtered_data_months_S2)

train_data_months_S2 <- split_data_months_S2$train
test_data_months_S2 <- split_data_months_S2$test
```

```r
# Check overlap between training and testing datasets
sum(train_data_months_S2$PlotObservationID %in% test_data_months_S2$PlotObservationID)
```

```
## [1] 0
```

```r
# Both
set.seed(123)
split_data_both_S2 <- split_train_test(filtered_data_both_S2)

train_data_both_S2 <- split_data_both_S2$train
test_data_both_S2 <- split_data_both_S2$test

# Check overlap between training and testing datasets
sum(train_data_both_S2$PlotObservationID %in% test_data_both_S2$PlotObservationID)
```

```
## [1] 0
```

```r
# Landsat
# Slope
set.seed(123)
split_data_slope_Landsat <- split_train_test(filtered_data_slope_Landsat)

train_data_slope_Landsat <- split_data_slope_Landsat$train
test_data_slope_Landsat <- split_data_slope_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_slope_Landsat$PlotObservationID %in% test_data_slope_Landsat$PlotObservationID)
```

```
## [1] 0
```

```r
# Months
set.seed(123)
split_data_months_Landsat <- split_train_test(filtered_data_months_Landsat)

train_data_months_Landsat <- split_data_months_Landsat$train
test_data_months_Landsat <- split_data_months_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_months_Landsat$PlotObservationID %in% test_data_months_Landsat$PlotObservationID)
```

```
## [1] 0
```

```r
# Both
set.seed(123)
split_data_both_Landsat <- split_train_test(filtered_data_both_Landsat)

train_data_both_Landsat <- split_data_both_Landsat$train
test_data_both_Landsat <- split_data_both_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_both_Landsat$PlotObservationID %in% test_data_both_Landsat$PlotObservationID)
```

```
## [1] 0
```

```r
# SatEmb
set.seed(123)
split_data_SatEmb <- split_train_test(filtered_data_SatEmb)

train_data_SatEmb <- split_data_SatEmb$train
test_data_SatEmb <- split_data_SatEmb$test

# Check overlap between training and testing datasets
sum(train_data_SatEmb$PlotObservationID %in% test_data_SatEmb$PlotObservationID)
```

```
## [1] 0
```

```r
# SatEmb_CH
set.seed(123)
split_data_SatEmb_CH <- split_train_test(filtered_data_SatEmb_CH)

train_data_SatEmb_CH <- split_data_SatEmb_CH$train
test_data_SatEmb_CH <- split_data_SatEmb_CH$test

# Check overlap between training and testing datasets
sum(train_data_SatEmb_CH$PlotObservationID %in% test_data_SatEmb_CH$PlotObservationID)
```

```
## [1] 0
```

Fit models

```r
models_no_refin <- c("rf_slope_S2", "rf_months_S2", "rf_both_S2",
                     "rf_slope_Landsat", "rf_months_Landsat", "rf_both_Landsat",
                     "rf_SatEmb", "rf_SatEmb_CH")
```

Print the time:

```r
for (model_name in models_no_refin) {
  obj <- get(model_name)
  cat("Time for", model_name, ":\n")
  print(obj$time)
}
```

```
## Time for rf_slope_S2 :
##    user  system elapsed
##   18.22    3.19   74.88
## Time for rf_months_S2 :
##    user  system elapsed
##   10.64    2.34   59.28
## Time for rf_both_S2 :
##    user  system elapsed
##   30.75    7.87  145.37
## Time for rf_slope_Landsat :
```

```
##    user  system elapsed
##   53.36    8.75  425.42
## Time for rf_months_Landsat :
##    user  system elapsed
##   27.22    4.51  161.14
## Time for rf_both_Landsat :
##    user  system elapsed
##   47.75    7.96  271.98
## Time for rf_SatEmb :
##    user  system elapsed
##   42.07    4.57  212.54
## Time for rf_SatEmb_CH :
##    user  system elapsed
##   71.59   12.19  268.61
```

Save:

Predictions

```r
predictions_no_refin <- c("rf_slope_S2_prediction", "rf_months_S2_prediction",
                          "rf_both_S2_prediction",
                          "rf_slope_Landsat_prediction",
                          "rf_months_Landsat_prediction",
                          "rf_both_Landsat_prediction",
                          "rf_SatEmb_prediction", "rf_SatEmb_CH_prediction")
```

Save:

Confusion matrices

```r
for (prediction_name in predictions_no_refin) {
  # Derive test data name by replacing "_prediction" with ""
  # and "rf_" with "test_data_"
  test_data_name <- paste0("test_data_", sub("^rf_", "", sub("_prediction$", "",
                                                            prediction_name)))

  # Get prediction and test data
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Compute confusion matrix
  cat("Confusion Matrix for", sub("_prediction$", "", prediction_name), ":\n")
  print(confusionMatrix(prediction, test_data$EUNISa_1))
  cat("\n")
}
```

```
## Confusion Matrix for rf_slope_S2 :
## Confusion Matrix and Statistics
##
```

```
##           Reference
## Prediction    Q    R    S    T
##          Q  231  176  162    0
##          R 1421 2829  918    1
##          S   29   36   46    2
##          T    0    0    0  201
##
## Overall Statistics
##
##                Accuracy : 0.5464
##                  95% CI : (0.5338, 0.559)
##     No Information Rate : 0.5025
##     P-Value [Acc > NIR] : 4.216e-12
##
##                   Kappa : 0.1603
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity           0.13742   0.9303 0.040853  0.98529
## Specificity           0.92267   0.2228 0.986399  1.00000
## Pos Pred Value        0.40598   0.5473 0.407080  1.00000
## Neg Pred Value        0.73555   0.7599 0.818151  0.99949
## Prevalence            0.27776   0.5025 0.186054  0.03371
## Detection Rate        0.03817   0.4674 0.007601  0.03321
## Detection Prevalence  0.09402   0.8541 0.018672  0.03321
## Balanced Accuracy     0.53005   0.5766 0.513626  0.99265
##
## Confusion Matrix for rf_months_S2 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  277  245  186    0
##          R 1050 2356  709    2
##          S   58   48   94    0
##          T    0    0    0  197
##
## Overall Statistics
##
##                Accuracy : 0.5599
##                  95% CI : (0.5463, 0.5735)
##     No Information Rate : 0.5073
##     P-Value [Acc > NIR] : 1.384e-14
##
##                   Kappa : 0.2077
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
```

```
## Sensitivity           0.20000   0.8894  0.09505  0.98995
## Specificity           0.88767   0.3156  0.97496  1.00000
## Pos Pred Value        0.39124   0.5723  0.47000  1.00000
## Neg Pred Value        0.75454   0.7348  0.82178  0.99960
## Prevalence            0.26522   0.5073  0.18939  0.03811
## Detection Rate        0.05304   0.4512  0.01800  0.03773
## Detection Prevalence  0.13558   0.7884  0.03830  0.03773
## Balanced Accuracy     0.54384   0.6025  0.53500  0.99497
##
## Confusion Matrix for rf_both_S2 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  296  262  236    0
##          R 1014 2225  665    2
##          S   81   78   91    0
##          T    0    0    0  194
##
## Overall Statistics
##
##                Accuracy : 0.5455
##                  95% CI : (0.5318, 0.5592)
##     No Information Rate : 0.4986
##     P-Value [Acc > NIR] : 9.682e-12
##
##                   Kappa : 0.201
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity           0.21280   0.8674  0.09173  0.98980
## Specificity           0.86731   0.3482  0.96171  1.00000
## Pos Pred Value        0.37280   0.5696  0.36400  1.00000
## Neg Pred Value        0.74828   0.7254  0.81590  0.99960
## Prevalence            0.27041   0.4986  0.19285  0.03810
## Detection Rate        0.05754   0.4325  0.01769  0.03771
## Detection Prevalence  0.15435   0.7593  0.04860  0.03771
## Balanced Accuracy     0.54005   0.6078  0.52672  0.99490
##
## Confusion Matrix for rf_slope_Landsat :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  711  599  420   13
##          R 4052 8171 2512  212
##          S  292  239  356    1
##          T   48  101   30  323
##
## Overall Statistics
##
```

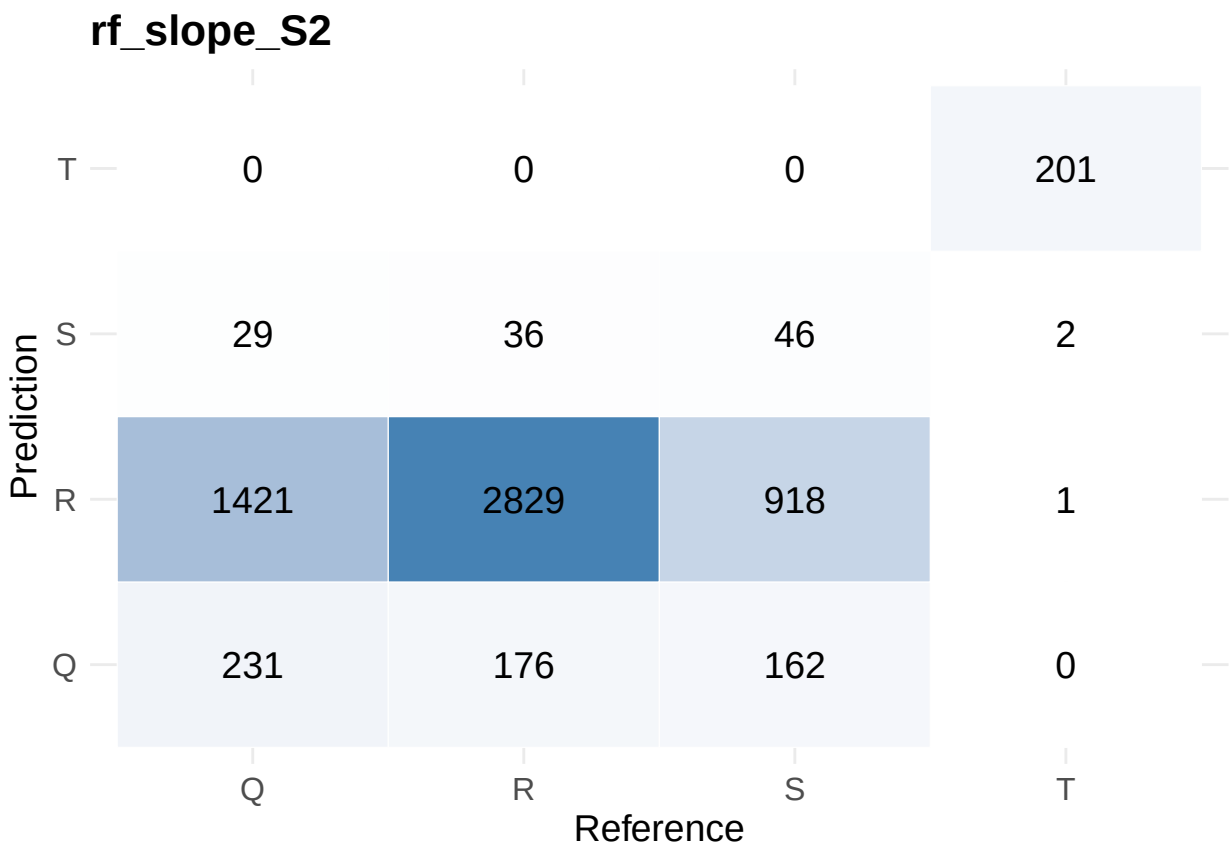
```
##                Accuracy : 0.5288
##                  95% CI : (0.5215, 0.5361)
##     No Information Rate : 0.5039
##     P-Value [Acc > NIR] : 1.02e-11
##
##                   Kappa : 0.1376
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity           0.13933   0.8969  0.10729  0.58834
## Specificity           0.92047   0.2446  0.96396  0.98979
## Pos Pred Value        0.40792   0.5467  0.40090  0.64343
## Neg Pred Value        0.73116   0.7003  0.82771  0.98714
## Prevalence            0.28225   0.5039  0.18352  0.03037
## Detection Rate        0.03933   0.4519  0.01969  0.01787
## Detection Prevalence  0.09640   0.8267  0.04912  0.02777
## Balanced Accuracy     0.52990   0.5708  0.53563  0.78907
##
## Confusion Matrix for rf_months_Landsat :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  667  513  377   19
##          R 2280 4860 1411  121
##          S  131  136  167    0
##          T   22   54   21  229
##
## Overall Statistics
##
##                Accuracy : 0.5381
##                  95% CI : (0.5287, 0.5474)
##     No Information Rate : 0.5054
##     P-Value [Acc > NIR] : 3.51e-12
##
##                   Kappa : 0.1654
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity           0.21516   0.8736  0.08451  0.62060
## Specificity           0.88505   0.2999  0.97044  0.99088
## Pos Pred Value        0.42322   0.5604  0.38479  0.70245
## Neg Pred Value        0.74205   0.6991  0.82892  0.98689
## Prevalence            0.28161   0.5054  0.17951  0.03352
## Detection Rate        0.06059   0.4415  0.01517  0.02080
## Detection Prevalence  0.14317   0.7878  0.03943  0.02961
## Balanced Accuracy     0.55011   0.5868  0.52748  0.80574
##
```

```
## Confusion Matrix for rf_both_Landsat :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  667  476  367   13
##          R 2211 4781 1373   99
##          S  123  120  166    0
##          T   29   74   21  230
##
## Overall Statistics
##
##                Accuracy : 0.5436
##                  95% CI : (0.5342, 0.5531)
##     No Information Rate : 0.5071
##     P-Value [Acc > NIR] : 1.757e-14
##
##                   Kappa : 0.1747
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity           0.22013   0.8771  0.08614  0.67251
## Specificity           0.88912   0.3050  0.97246  0.98809
## Pos Pred Value        0.43795   0.5649  0.40587  0.64972
## Neg Pred Value        0.74390   0.7069  0.82971  0.98923
## Prevalence            0.28186   0.5071  0.17926  0.03181
## Detection Rate        0.06205   0.4447  0.01544  0.02140
## Detection Prevalence  0.14167   0.7873  0.03805  0.03293
## Balanced Accuracy     0.55463   0.5910  0.52930  0.83030
##
## Confusion Matrix for rf_SatEmb :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  526  382  296    0
##          R 1068 2865  717   29
##          S  108  110  165    0
##          T    0   12    1  199
##
## Overall Statistics
##
##                Accuracy : 0.5797
##                  95% CI : (0.5675, 0.5917)
##     No Information Rate : 0.5201
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.2542
##
##  Mcnemar's Test P-Value : NA
##
```

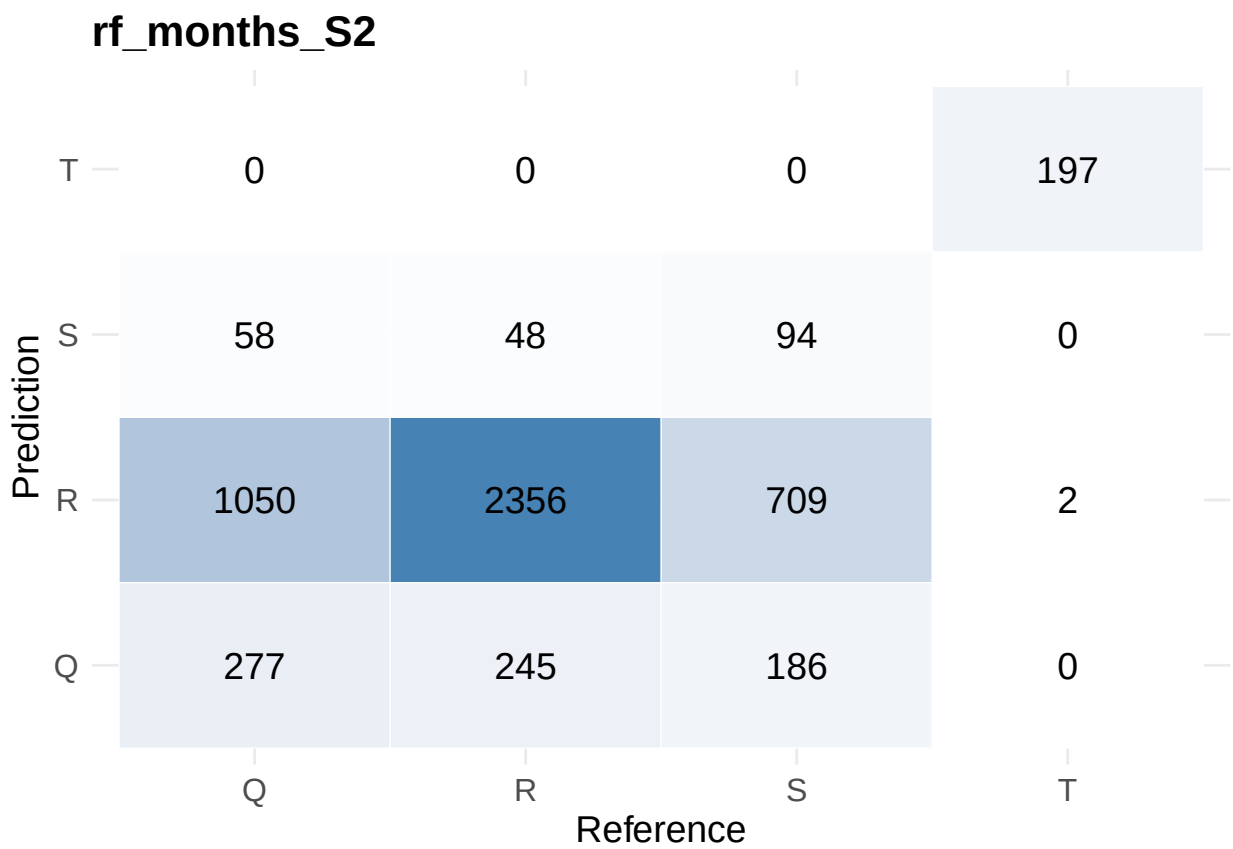
```
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.3090   0.8504  0.13995  0.87281
## Specificity            0.8580   0.4165  0.95886  0.99792
## Pos Pred Value         0.4369   0.6123  0.43081  0.93868
## Neg Pred Value         0.7770   0.7198  0.83363  0.99537
## Prevalence             0.2627   0.5201  0.18200  0.03520
## Detection Rate         0.0812   0.4423  0.02547  0.03072
## Detection Prevalence   0.1859   0.7223  0.05912  0.03273
## Balanced Accuracy      0.5835   0.6335  0.54940  0.93536
##
## Confusion Matrix for rf_SatEmb_CH :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  528  369  329    0
##          R 1076 2893  673    2
##          S   98  105  176    0
##          T    0    2    1  226
##
## Overall Statistics
##
##                Accuracy : 0.5902
##                  95% CI : (0.5781, 0.6022)
##     No Information Rate : 0.5201
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.2753
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity           0.31022   0.8587  0.14928  0.99123
## Specificity           0.85385   0.4368  0.96169  0.99952
## Pos Pred Value        0.43067   0.6230  0.46438  0.98690
## Neg Pred Value        0.77647   0.7405  0.83555  0.99968
## Prevalence            0.26274   0.5201  0.18200  0.03520
## Detection Rate        0.08151   0.4466  0.02717  0.03489
## Detection Prevalence  0.18926   0.7169  0.05851  0.03535
## Balanced Accuracy     0.58204   0.6478  0.55548  0.99537
```

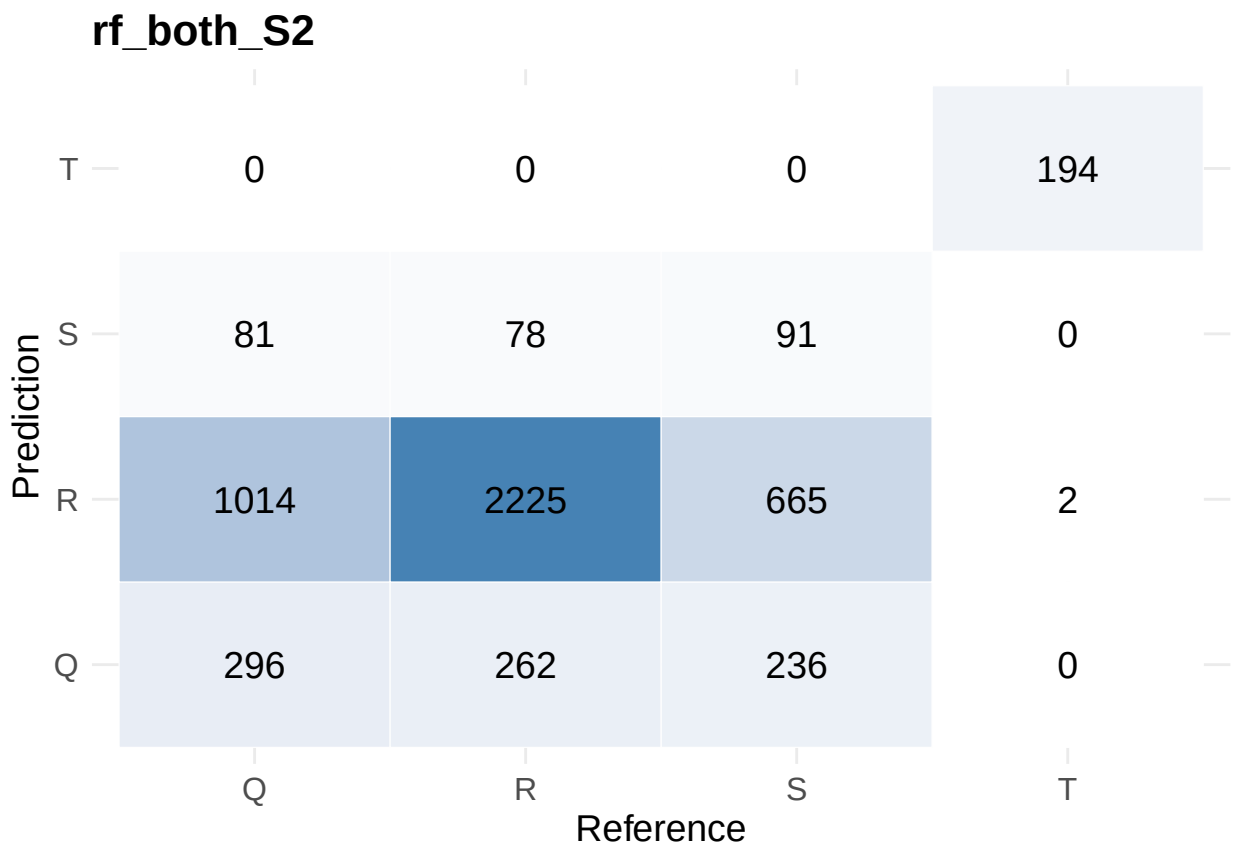
```r
for (prediction_name in predictions_no_refin) {
  # Derive name of test data
  test_data_name <- paste0("test_data_", sub("^rf_", "", sub("_prediction$", "",
                                                            prediction_name)))

  # Obtain objects
```

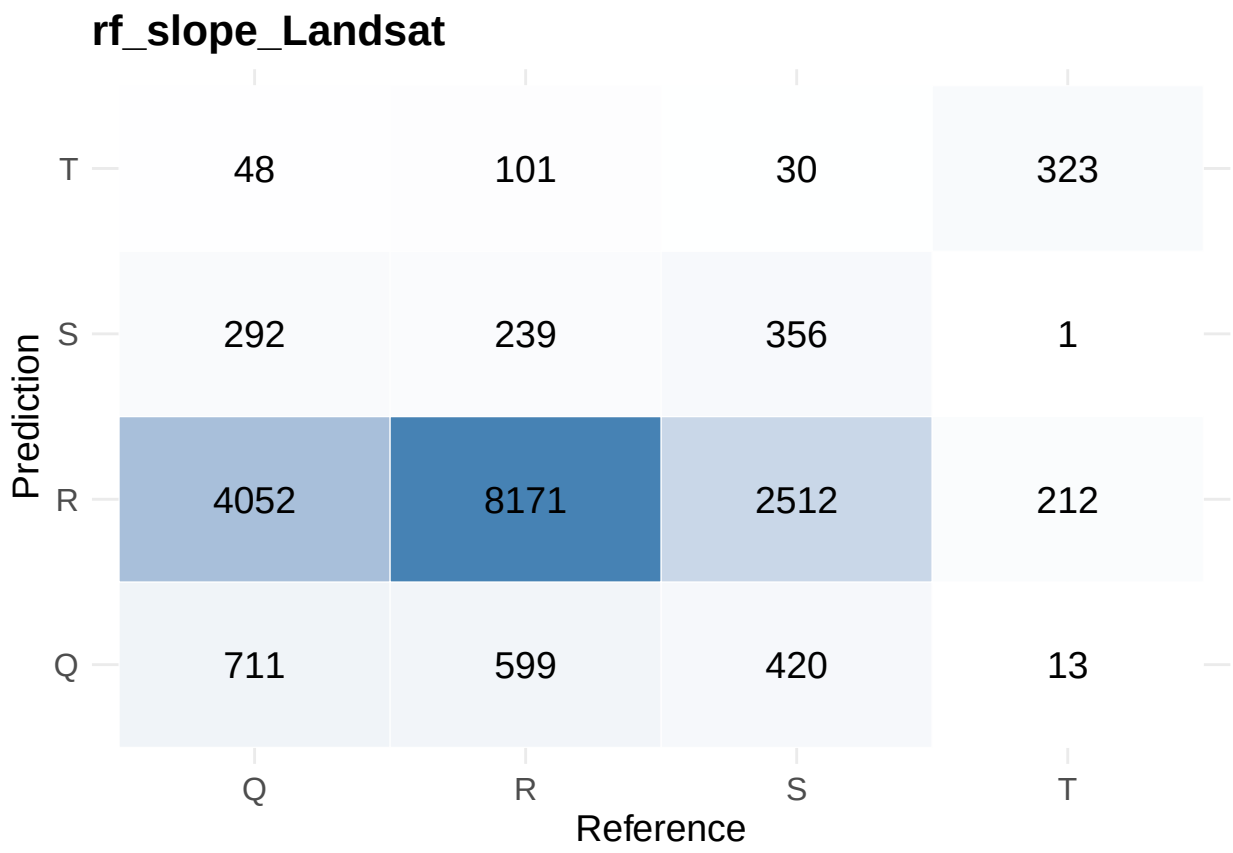
```r
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Personalized title
  title <- paste(#"Confusion Matrix for",
    sub("_prediction$", "",
                                              prediction_name))

  # Plot
  print(plot_confusion_matrix(prediction, test_data$EUNISa_1, title))
}
```

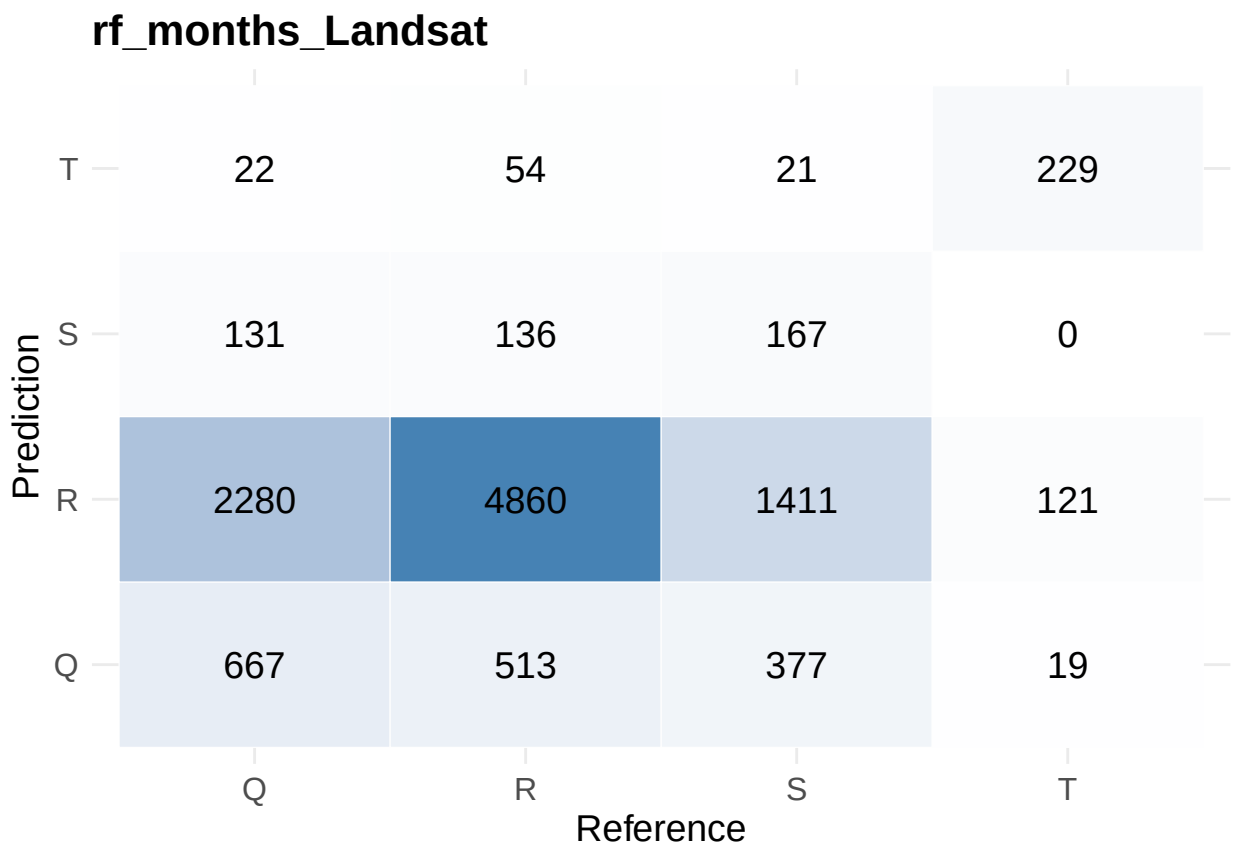

rf_slope_S2
T
S
R
Q
Prediction
0
0
0
201
29
36
46
2
1421
2829
918
1
231
176
162
0
Q
R
S
T
Reference

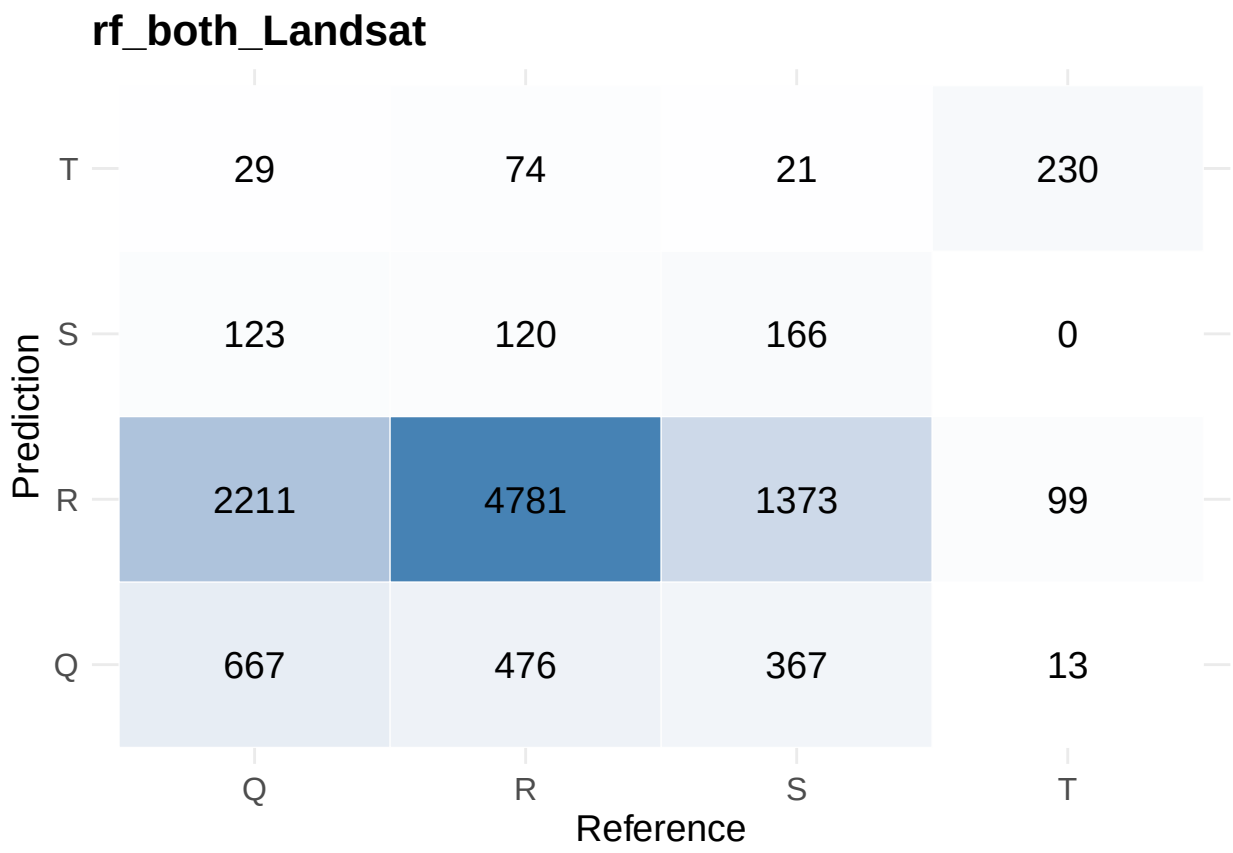
Plots

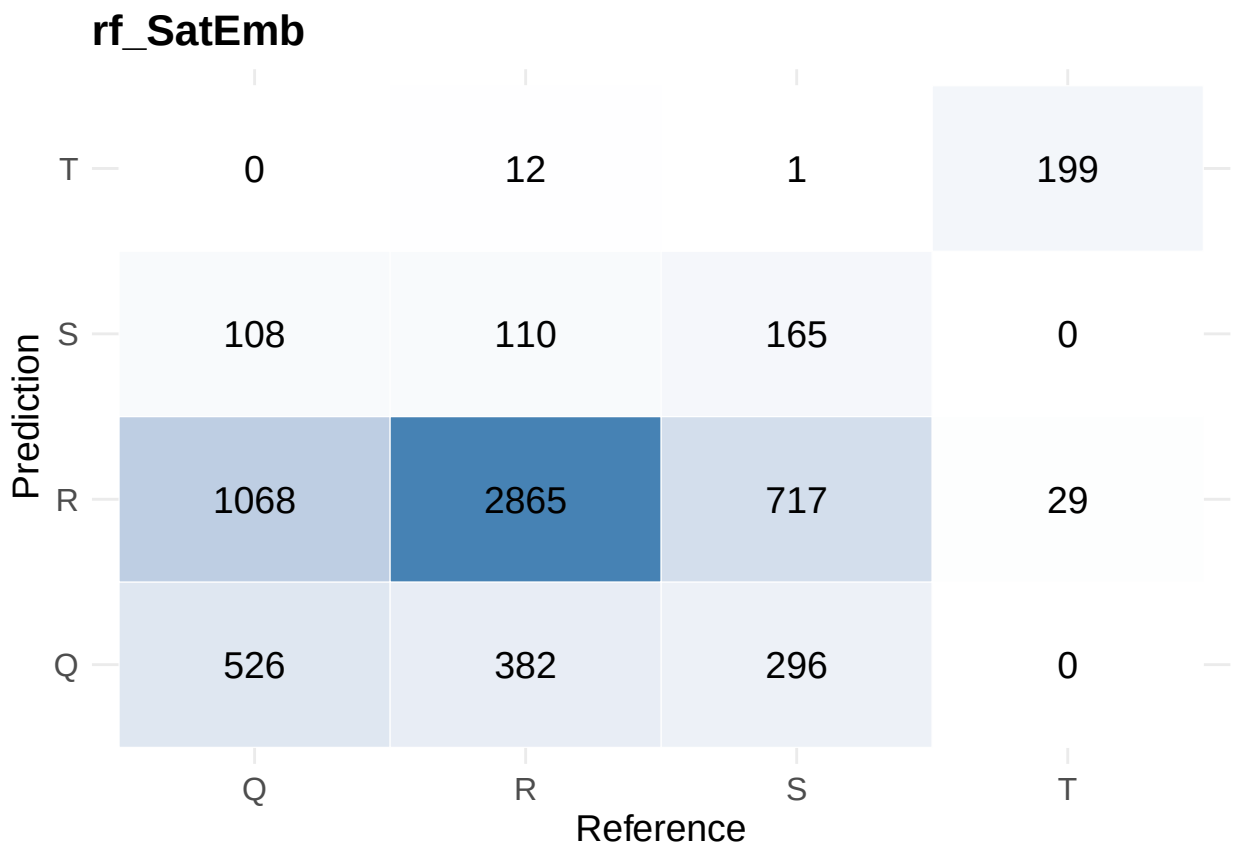

rf_months_S2
T
S
R
Q
Prediction
0
0
0
197
58
48
94
0
1050
2356
709
2
277
245
186
0
Q
R
S
T
Reference

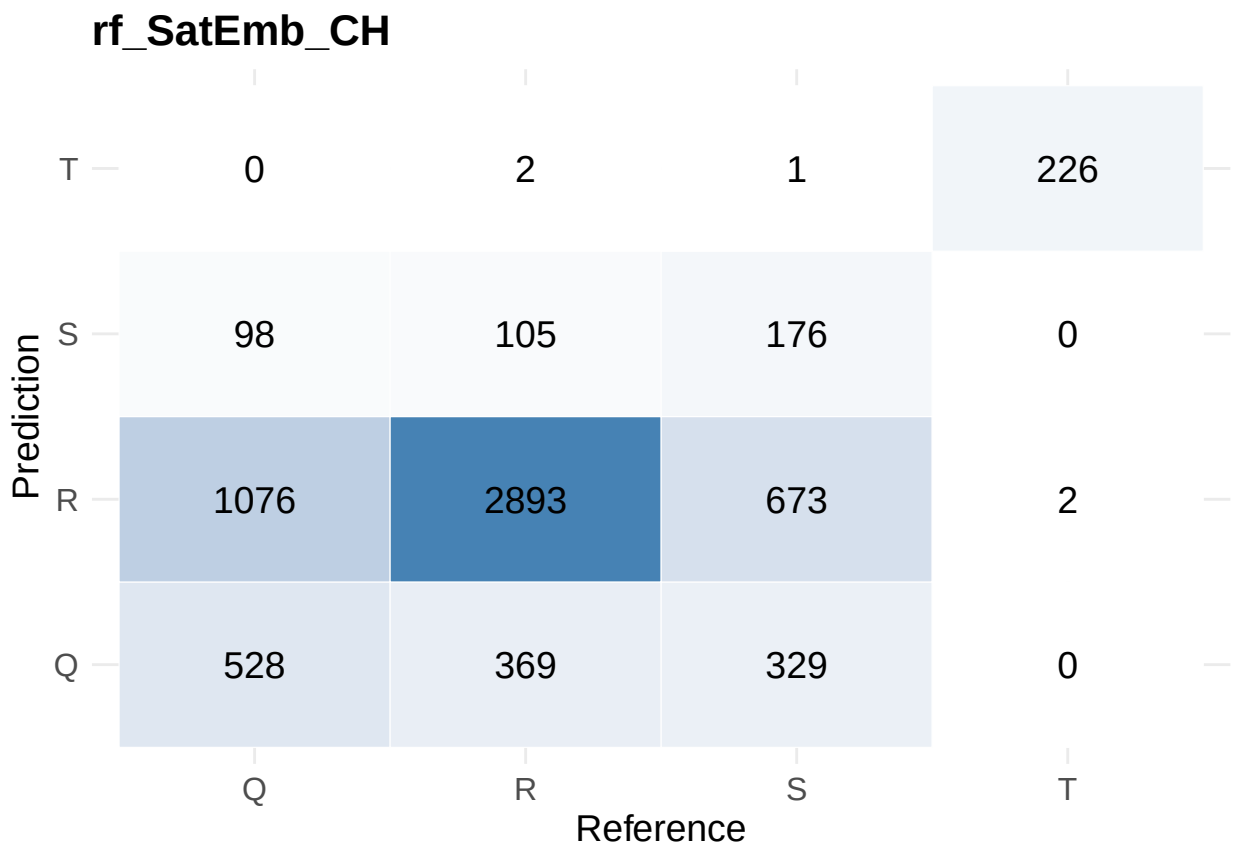

rf_both_S2
T
S
R
Q
Prediction
0
0
0
194
81
78
91
0
1014
2225
665
2
296
262
236
0
Q
R
S
T
Reference


rf_slope_Landsat
Prediction
T
S
R
Q
48
101
30
323
292
239
356
1
4052
8171
2512
212
711
599
420
13
Q
R
S
T
Reference


rf_months_Landsat
Prediction
T
S
R
Q
22
54
21
229
131
136
167
0
2280
4860
1411
121
667
513
377
19
Q
R
S
T
Reference


rf_both_Landsat
Prediction
T
S
R
Q
29
74
21
230
123
120
166
0
2211
4781
1373
99
667
476
367
13
Q
R
S
T
Reference


rf_SatEmb
T
S
R
Q
Prediction
0
12
1
199
108
110
165
0
1068
2865
717
29
526
382
296
0
Q
R
S
T
Reference



Refinement step 1

Based on PAs "Morralla protocol".

Compute predictions for all data and combine with original data

```
filtered_data_with_probs <- c("filtered_data_slope_S2_with_probs",
                              "filtered_data_months_S2_with_probs",
                              "filtered_data_both_S2_with_probs",
                              "filtered_data_slope_Landsat_with_probs",
                              "filtered_data_months_Landsat_with_probs",
                              "filtered_data_both_Landsat_with_probs",
                              "filtered_data_SatEmb_with_probs",
                              "filtered_data_SatEmb_CH_with_probs")
```

Save:

Detect probability columns

```
prob_cols_slope_S2 <- names(filtered_data_slope_S2_with_probs)[grepl("^prob_", names(filtered_data_slope
prob_cols_months_S2 <- names(filtered_data_months_S2_with_probs)[grepl("^prob_", names(filtered_data_mon
```

```
prob_cols_both_S2 <- names(filtered_data_both_S2_with_probs)[grepl("^prob_", names(filtered_data_both_S2
prob_cols_slope_Landsat <- names(filtered_data_slope_Landsat_with_probs)[grepl("^prob_", names(filtered_
prob_cols_months_Landsat <- names(filtered_data_months_Landsat_with_probs)[grepl("^prob_", names(filtere
prob_cols_both_Landsat <- names(filtered_data_both_Landsat_with_probs)[grepl("^prob_", names(filtered_da
prob_cols_SatEmb <- names(filtered_data_SatEmb_with_probs)[grepl("^prob_", names(filtered_data_SatEmb_wi
prob_cols_SatEmb_CH <- names(filtered_data_SatEmb_CH_with_probs)[grepl("^prob_", names(filtered_data_Sat
```

Calculate auxiliary metrics

```
df_aux_slope_S2 <- calculate_df_aux(filtered_data_slope_S2_with_probs,
                                    prob_cols_slope_S2)
df_aux_months_S2 <- calculate_df_aux(filtered_data_months_S2_with_probs,
                                    prob_cols_months_S2)
df_aux_both_S2 <- calculate_df_aux(filtered_data_both_S2_with_probs,
                                    prob_cols_both_S2)
df_aux_slope_Landsat <- calculate_df_aux(filtered_data_slope_Landsat_with_probs,
                                    prob_cols_slope_Landsat)
df_aux_months_Landsat <- calculate_df_aux(filtered_data_months_Landsat_with_probs,
                                    prob_cols_months_Landsat)
df_aux_both_Landsat <- calculate_df_aux(filtered_data_both_Landsat_with_probs,
                                    prob_cols_both_Landsat)
df_aux_SatEmb <- calculate_df_aux(filtered_data_SatEmb_with_probs,
                                  prob_cols_SatEmb)
df_aux_SatEmb_CH <- calculate_df_aux(filtered_data_SatEmb_CH_with_probs,
                                    prob_cols_SatEmb_CH)
```

Apply filtering function

Applying "Morralla" protocol once, removing negatives until 20% of the class and, if there is room, uncertain positives until 10% of the rest.

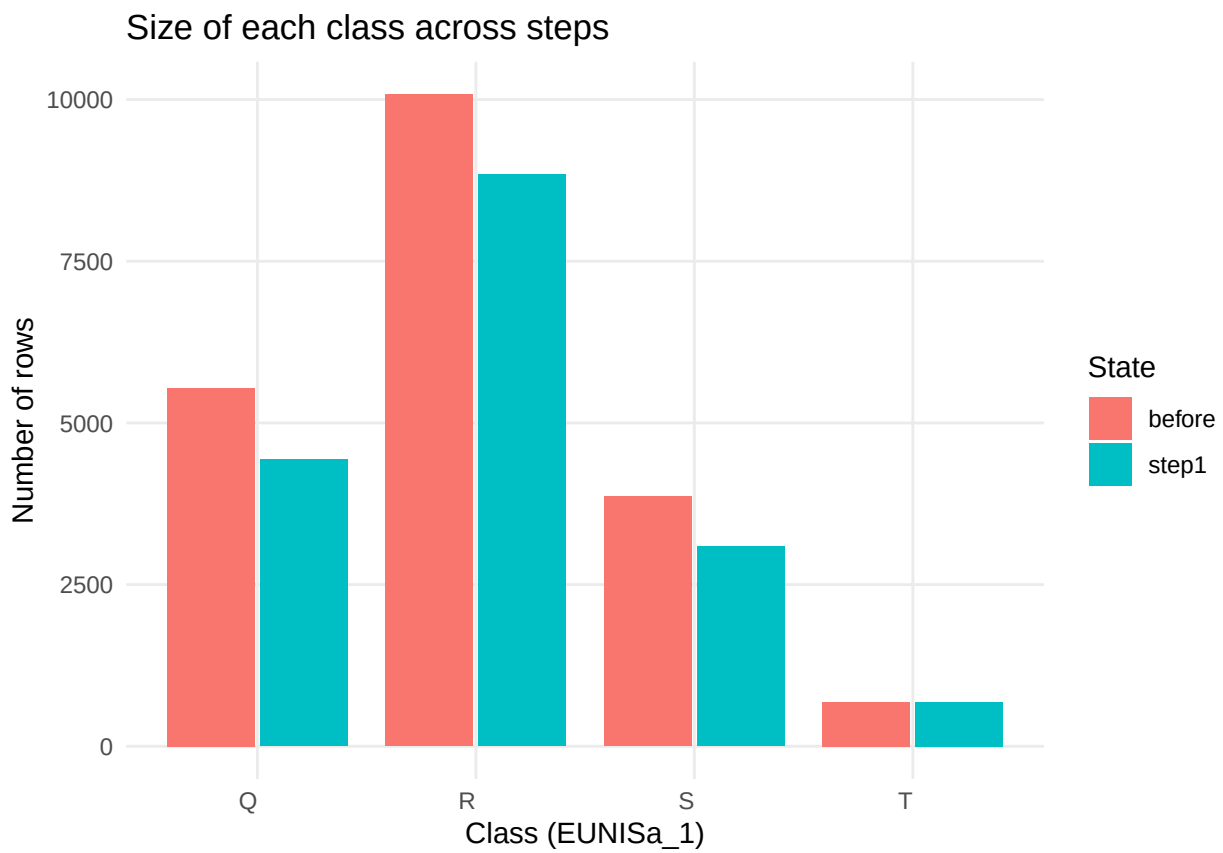
```
df_final_20_10_slope_S2 <- df_aux_slope_S2 %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
df_final_20_10_months_S2 <- df_aux_months_S2 %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
df_final_20_10_both_S2 <- df_aux_both_S2 %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
df_final_20_10_slope_Landsat <- df_aux_slope_Landsat %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
df_final_20_10_months_Landsat <- df_aux_months_Landsat %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
df_final_20_10_both_Landsat <- df_aux_both_Landsat %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
df_final_20_10_SatEmb <- df_aux_SatEmb %>%
```

```
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
df_final_20_10_SatEmb_CH <- df_aux_SatEmb_CH %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20_pos10)
```

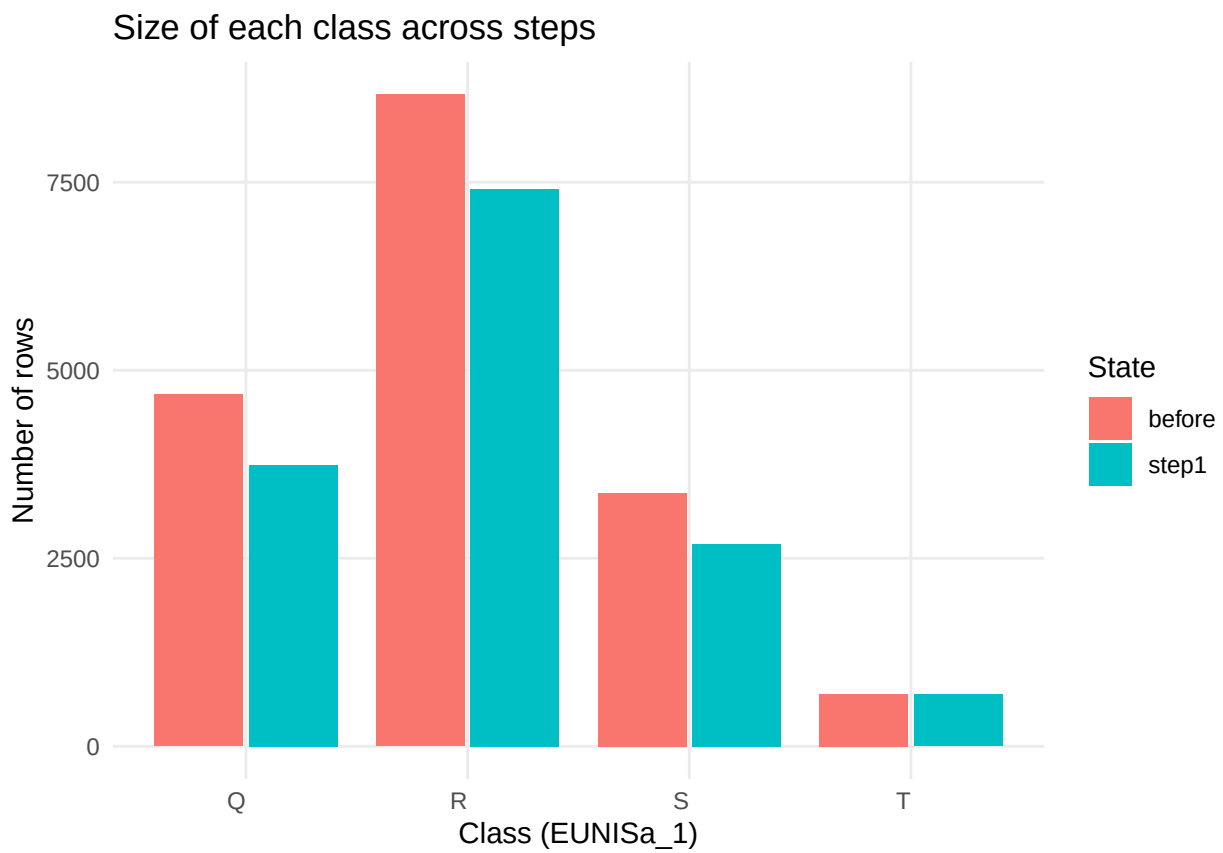
```
filtered_data_20_10_slope_S2_probs <- filtered_data_slope_S2 %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_slope_S2$PlotObservationID)
filtered_data_20_10_months_S2_probs <- filtered_data_months_S2 %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_months_S2$PlotObservationID)
filtered_data_20_10_both_S2_probs <- filtered_data_both_S2 %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_both_S2$PlotObservationID)
filtered_data_20_10_slope_Landsat_probs <- filtered_data_slope_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_slope_Landsat$PlotObservationID)
filtered_data_20_10_months_Landsat_probs <- filtered_data_months_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_months_Landsat$PlotObservationID)
filtered_data_20_10_both_Landsat_probs <- filtered_data_both_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_both_Landsat$PlotObservationID)
filtered_data_20_10_SatEmb_probs <- filtered_data_SatEmb %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_SatEmb$PlotObservationID)
filtered_data_20_10_SatEmb_CH_probs <- filtered_data_SatEmb_CH %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_SatEmb_CH$PlotObservationID)
```

Compare classes before and after

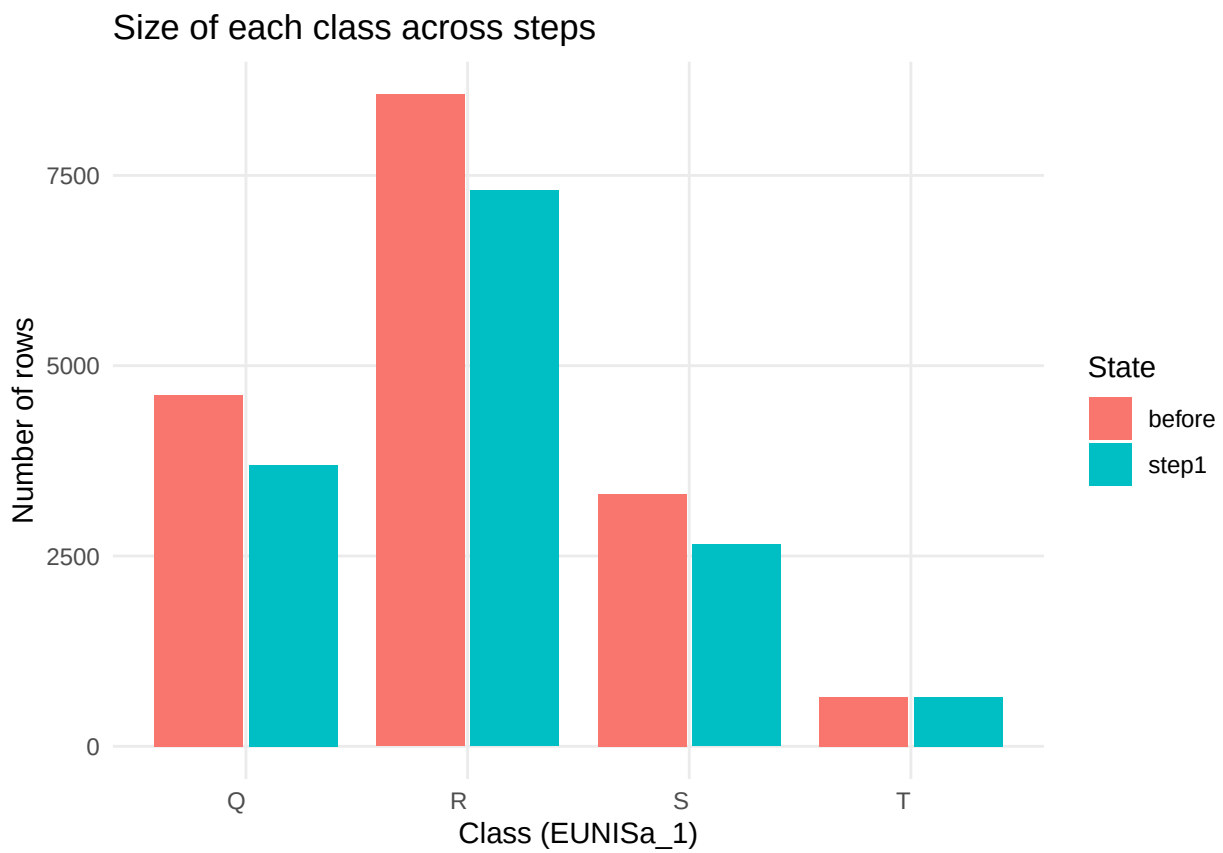
```
compare_classes(filtered_data_slope_S2, filtered_data_20_10_slope_S2_probs)
```



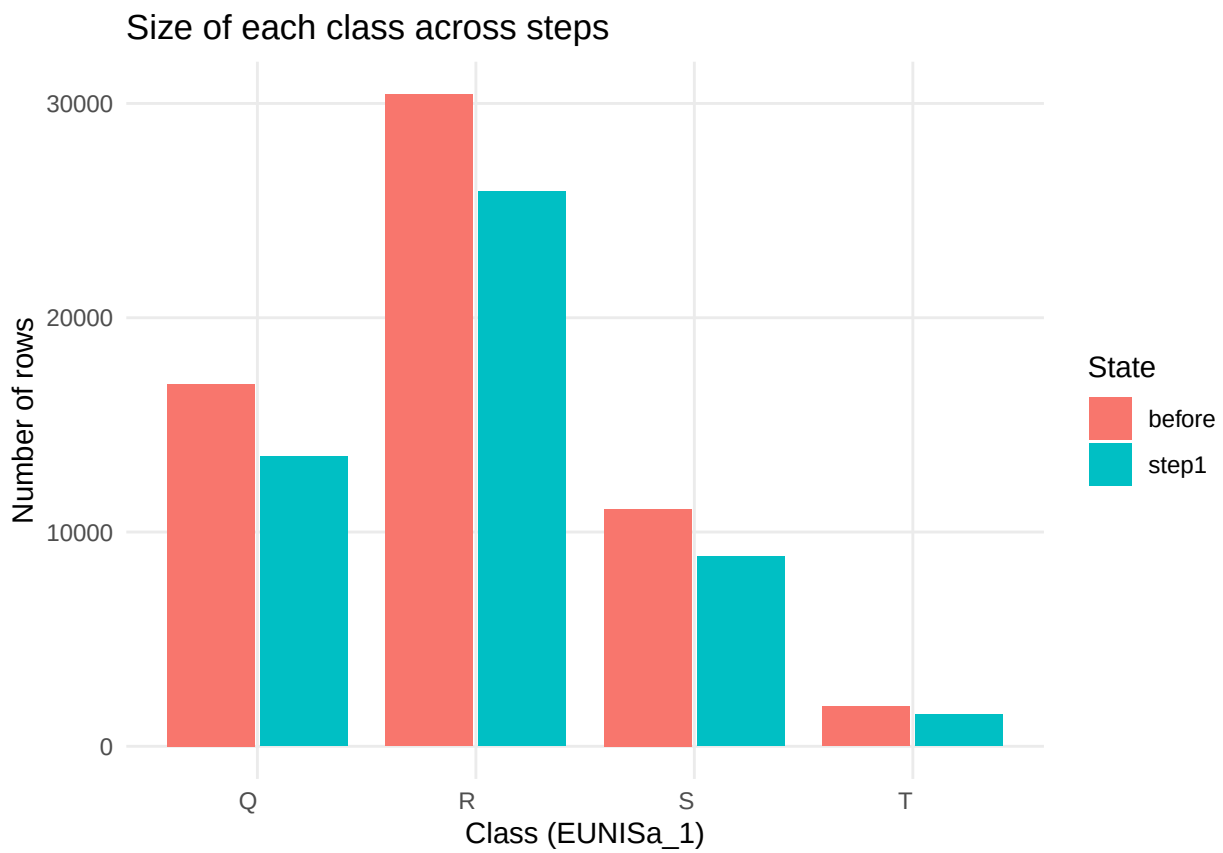

```
compare_classes(filtered_data_months_S2, filtered_data_20_10_months_S2_probs)
```



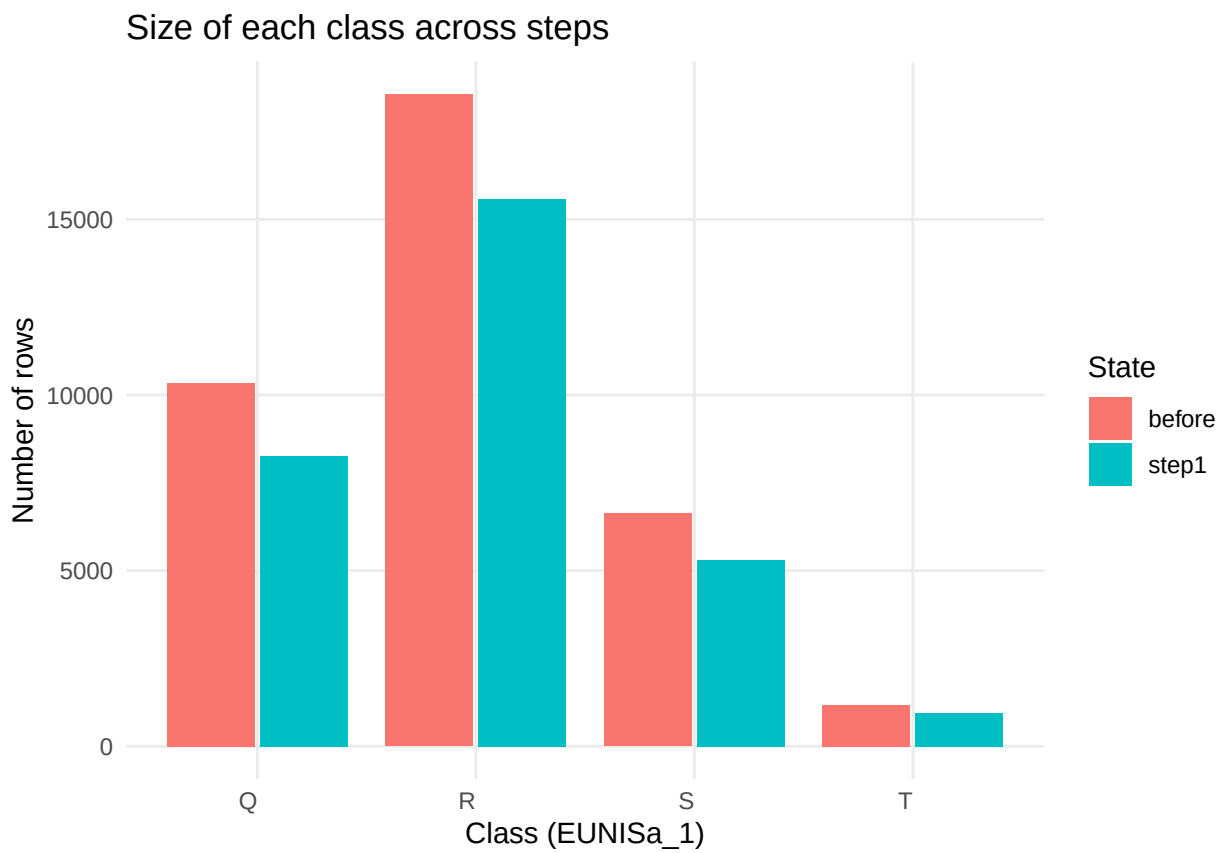

```
compare_classes(filtered_data_both_S2, filtered_data_20_10_both_S2_probs)
```



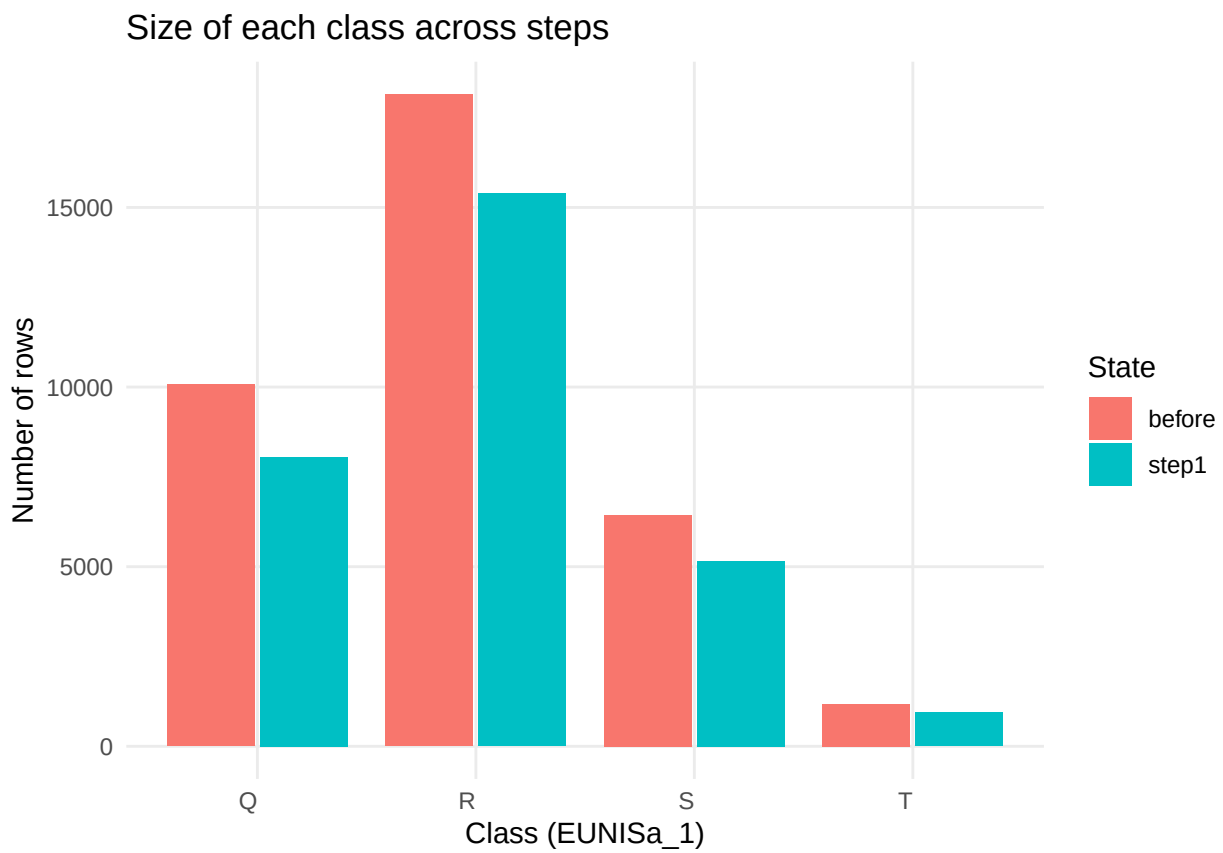

```
compare_classes(filtered_data_slope_Landsat, filtered_data_20_10_slope_Landsat_probs)
```



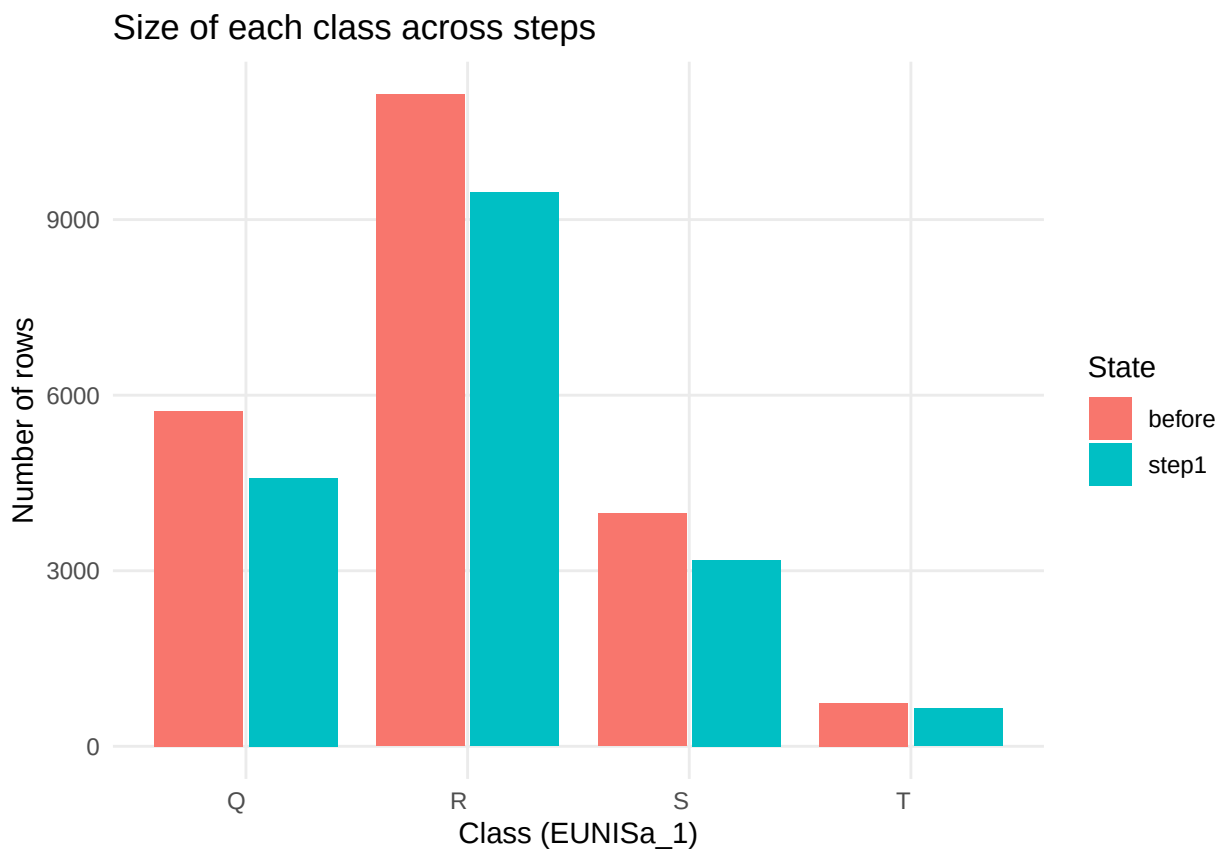

```
compare_classes(filtered_data_months_Landsat, filtered_data_20_10_months_Landsat_probs)
```



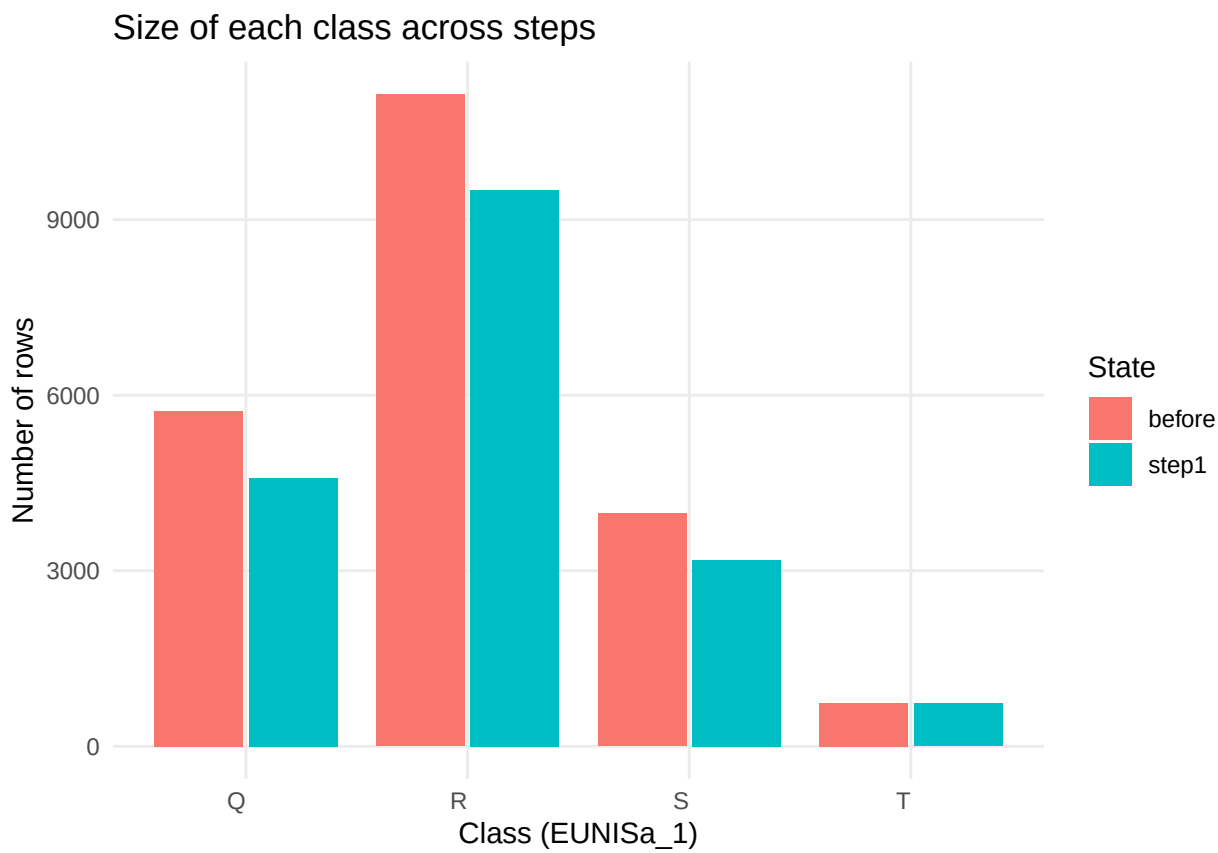

```
compare_classes(filtered_data_both_Landsat, filtered_data_20_10_both_Landsat_probs)
```




```
compare_classes(filtered_data_SatEmb, filtered_data_20_10_SatEmb_probs)
```




```
compare_classes(filtered_data_SatEmb_CH, filtered_data_20_10_SatEmb_CH_probs)
```



Split into training and testing

```
# S2
# Slope
set.seed(123)
split_data_20_10_slope_S2 <- split_train_test(filtered_data_20_10_slope_S2_probs)

train_data_20_10_slope_S2 <- split_data_20_10_slope_S2$train
test_data_20_10_slope_S2 <- split_data_20_10_slope_S2$test

# Check overlap between training and testing datasets
sum(train_data_20_10_slope_S2$PlotObservationID %in% test_data_20_10_slope_S2$PlotObservationID)
```

```
## [1] 0
```

```
# Months
set.seed(123)
split_data_20_10_months_S2 <- split_train_test(filtered_data_20_10_months_S2_probs)

train_data_20_10_months_S2 <- split_data_20_10_months_S2$train
test_data_20_10_months_S2 <- split_data_20_10_months_S2$test

# Check overlap between training and testing datasets
sum(train_data_20_10_months_S2$PlotObservationID %in% test_data_20_10_months_S2$PlotObservationID)
```

```
## [1] 0
```

```r
# Both
set.seed(123)
split_data_20_10_both_S2 <- split_train_test(filtered_data_20_10_both_S2_probs)

train_data_20_10_both_S2 <- split_data_20_10_both_S2$train
test_data_20_10_both_S2 <- split_data_20_10_both_S2$test

# Check overlap between training and testing datasets
sum(train_data_20_10_both_S2$PlotObservationID %in% test_data_20_10_both_S2$PlotObservationID)
```

```
## [1] 0
```

```r
# Landsat
# Slope
set.seed(123)
split_data_20_10_slope_Landsat <- split_train_test(filtered_data_20_10_slope_Landsat_probs)

train_data_20_10_slope_Landsat <- split_data_20_10_slope_Landsat$train
test_data_20_10_slope_Landsat <- split_data_20_10_slope_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_slope_Landsat$PlotObservationID %in% test_data_20_10_slope_Landsat$PlotObservation
```

```
## [1] 0
```

```r
# Months
set.seed(123)
split_data_20_10_months_Landsat <- split_train_test(filtered_data_20_10_months_Landsat_probs)

train_data_20_10_months_Landsat <- split_data_20_10_months_Landsat$train
test_data_20_10_months_Landsat <- split_data_20_10_months_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_months_Landsat$PlotObservationID %in% test_data_20_10_months_Landsat$PlotObservati
```

```
## [1] 0
```

```r
# Both
set.seed(123)
split_data_20_10_both_Landsat <- split_train_test(filtered_data_20_10_both_Landsat_probs)

train_data_20_10_both_Landsat <- split_data_20_10_both_Landsat$train
test_data_20_10_both_Landsat <- split_data_20_10_both_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_both_Landsat$PlotObservationID %in% test_data_20_10_both_Landsat$PlotObservationID
```

```
## [1] 0
```

```r
# SatEmb
set.seed(123)
split_data_20_10_SatEmb <- split_train_test(filtered_data_20_10_SatEmb_probs)

train_data_20_10_SatEmb <- split_data_20_10_SatEmb$train
test_data_20_10_SatEmb <- split_data_20_10_SatEmb$test

# Check overlap between training and testing datasets
sum(train_data_20_10_SatEmb$PlotObservationID %in% test_data_20_10_SatEmb$PlotObservationID)
```

```
## [1] 0
```

```r
# SatEmb_CH
set.seed(123)
split_data_20_10_SatEmb_CH <- split_train_test(filtered_data_20_10_SatEmb_CH_probs)

train_data_20_10_SatEmb_CH <- split_data_20_10_SatEmb_CH$train
test_data_20_10_SatEmb_CH <- split_data_20_10_SatEmb_CH$test

# Check overlap between training and testing datasets
sum(train_data_20_10_SatEmb_CH$PlotObservationID %in% test_data_20_10_SatEmb_CH$PlotObservationID)
```

```
## [1] 0
```

Fit models

```r
models_refin_20_10 <- c("rf_slope_S2_refin_20_10", "rf_months_S2_refin_20_10",
                        "rf_both_S2_refin_20_10",
                        "rf_slope_Landsat_refin_20_10",
                        "rf_months_Landsat_refin_20_10",
                        "rf_both_Landsat_refin_20_10",
                        "rf_SatEmb_refin_20_10", "rf_SatEmb_CH_refin_20_10")
```

Save:

Predictions

```r
predictions_refin_20_10 <- c("rf_slope_S2_refin_20_10_prediction",
                        "rf_months_S2_refin_20_10_prediction",
                        "rf_both_S2_refin_20_10_prediction",
                        "rf_slope_Landsat_refin_20_10_prediction",
                        "rf_months_Landsat_refin_20_10_prediction",
                        "rf_both_Landsat_refin_20_10_prediction",
                        "rf_SatEmb_refin_20_10_prediction",
                        "rf_SatEmb_CH_refin_20_10_prediction")
```

Save:

Confusion matrices

```r
for (prediction_name in predictions_refin_20_10) {
  # Derive test data name
  test_data_name <- paste0(gsub("_refin_20_10$", "",
                              sub("^rf_", "test_data_20_10_",
                                  sub("_prediction$", "", prediction_name))))

  # Get prediction and test data
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Compute confusion matrix
  cat("Confusion Matrix for", sub("_prediction$", "", prediction_name), ":\n")
  print(confusionMatrix(prediction, test_data$EUNISa_1))
  cat("\n")
}
```

```
## Confusion Matrix for rf_slope_S2_refin_20_10 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  547  239  391    0
##          R  710 2347  395    0
##          S  107   65  110    0
##          T    0    0    0  203
##
## Overall Statistics
##
##                Accuracy : 0.6271
##                  95% CI : (0.6137, 0.6404)
##     No Information Rate : 0.5184
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3542
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.4010   0.8853  0.12277  1.00000
## Specificity            0.8320   0.5514  0.95922  1.00000
## Pos Pred Value         0.4647   0.6799  0.39007  1.00000
## Neg Pred Value         0.7925   0.8171  0.83733  1.00000
## Prevalence             0.2667   0.5184  0.17521  0.03969
## Detection Rate         0.1070   0.4589  0.02151  0.03969
## Detection Prevalence   0.2302   0.6750  0.05514  0.03969
## Balanced Accuracy      0.6165   0.7183  0.54100  1.00000
##
## Confusion Matrix for rf_months_S2_refin_20_10 :
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction    Q    R    S    T
##          Q  500  190  380    0
##          R  487 1969  272    1
##          S  130   55  155    0
##          T    0    0    0  222
##
## Overall Statistics
##
##                Accuracy : 0.6526
##                  95% CI : (0.6383, 0.6667)
##     No Information Rate : 0.5077
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4233
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.4476   0.8893  0.19207  0.99552
## Specificity            0.8243   0.6460  0.94795  1.00000
## Pos Pred Value         0.4673   0.7215  0.45588  1.00000
## Neg Pred Value         0.8125   0.8499  0.83785  0.99976
## Prevalence             0.2561   0.5077  0.18505  0.05114
## Detection Rate         0.1147   0.4515  0.03554  0.05091
## Detection Prevalence   0.2454   0.6258  0.07796  0.05091
## Balanced Accuracy      0.6360   0.7677  0.57001  0.99776
##
## Confusion Matrix for rf_both_S2_refin_20_10 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  488  205  320    0
##          R  480 1912  275    0
##          S  171   60  177    0
##          T    0    0    0  198
##
## Overall Statistics
##
##                Accuracy : 0.6475
##                  95% CI : (0.6329, 0.6618)
##     No Information Rate : 0.5079
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4142
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
```

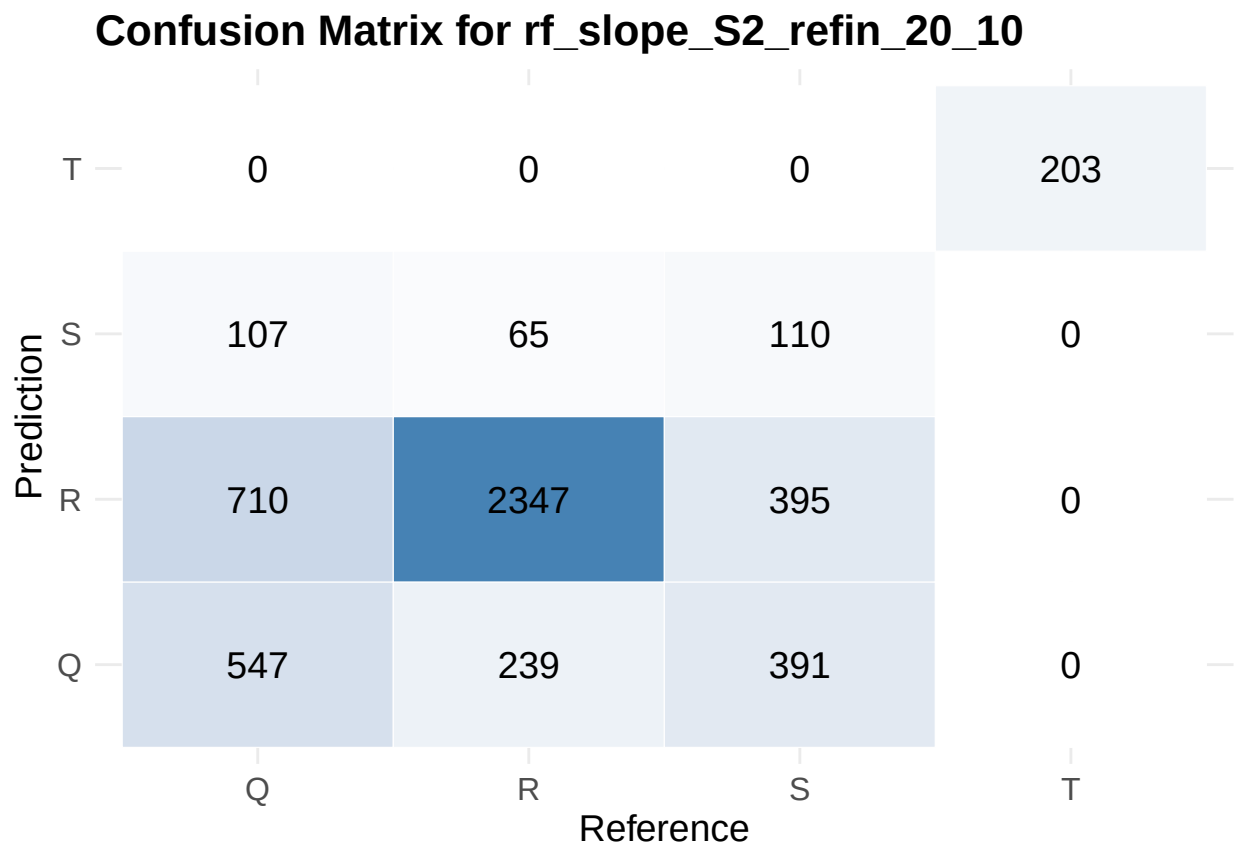
```
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.4284   0.8783  0.22927   1.0000
## Specificity            0.8332   0.6420  0.93426   1.0000
## Pos Pred Value         0.4817   0.7169  0.43382   1.0000
## Neg Pred Value         0.8011   0.8363  0.84657   1.0000
## Prevalence             0.2657   0.5079  0.18012   0.0462
## Detection Rate         0.1139   0.4461  0.04130   0.0462
## Detection Prevalence   0.2364   0.6223  0.09519   0.0462
## Balanced Accuracy      0.6308   0.7601  0.58177   1.0000
##
## Confusion Matrix for rf_slope_Landsat_refin_20_10 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q 1643  800 1016   24
##          R 1820 6820 1099   88
##          S  476  185  541   12
##          T   42   32   10  323
##
## Overall Statistics
##
##                Accuracy : 0.6247
##                  95% CI : (0.6168, 0.6324)
##     No Information Rate : 0.5249
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3495
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.4127   0.8702  0.20293  0.72260
## Specificity            0.8320   0.5761  0.94513  0.99420
## Pos Pred Value         0.4717   0.6940  0.44563  0.79361
## Neg Pred Value         0.7958   0.8007  0.84508  0.99146
## Prevalence             0.2666   0.5249  0.17855  0.02994
## Detection Rate         0.1100   0.4568  0.03623  0.02163
## Detection Prevalence   0.2333   0.6582  0.08131  0.02726
## Balanced Accuracy      0.6223   0.7232  0.57403  0.85840
##
## Confusion Matrix for rf_months_Landsat_refin_20_10 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q 1282  583  716   15
##          R  954 3988  527   35
##          S  254   87  306    0
##          T   27   14    8  231
##
## Overall Statistics
```

```
##
##                Accuracy : 0.6433
##                  95% CI : (0.6333, 0.6532)
##     No Information Rate : 0.5176
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3963
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.5093   0.8536  0.19653  0.82206
## Specificity            0.7982   0.6519  0.95435  0.99440
## Pos Pred Value         0.4938   0.7246  0.47295  0.82500
## Neg Pred Value         0.8080   0.8058  0.85072  0.99428
## Prevalence             0.2788   0.5176  0.17248  0.03113
## Detection Rate         0.1420   0.4418  0.03390  0.02559
## Detection Prevalence   0.2876   0.6097  0.07167  0.03102
## Balanced Accuracy      0.6537   0.7527  0.57544  0.90823
##
## Confusion Matrix for rf_both_Landsat_refin_20_10 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q 1205  525  634   10
##          R  949 3948  592   32
##          S  267  101  338    1
##          T   12    8    8  231
##
## Overall Statistics
##
##                Accuracy : 0.6458
##                  95% CI : (0.6357, 0.6557)
##     No Information Rate : 0.5171
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3988
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.4953   0.8616  0.21501  0.84307
## Specificity            0.8181   0.6324  0.94938  0.99674
## Pos Pred Value         0.5076   0.7151  0.47808  0.89189
## Neg Pred Value         0.8107   0.8102  0.84866  0.99500
## Prevalence             0.2746   0.5171  0.17741  0.03092
## Detection Rate         0.1360   0.4455  0.03814  0.02607
## Detection Prevalence   0.2679   0.6231  0.07979  0.02923
## Balanced Accuracy      0.6567   0.7470  0.58219  0.91990
```

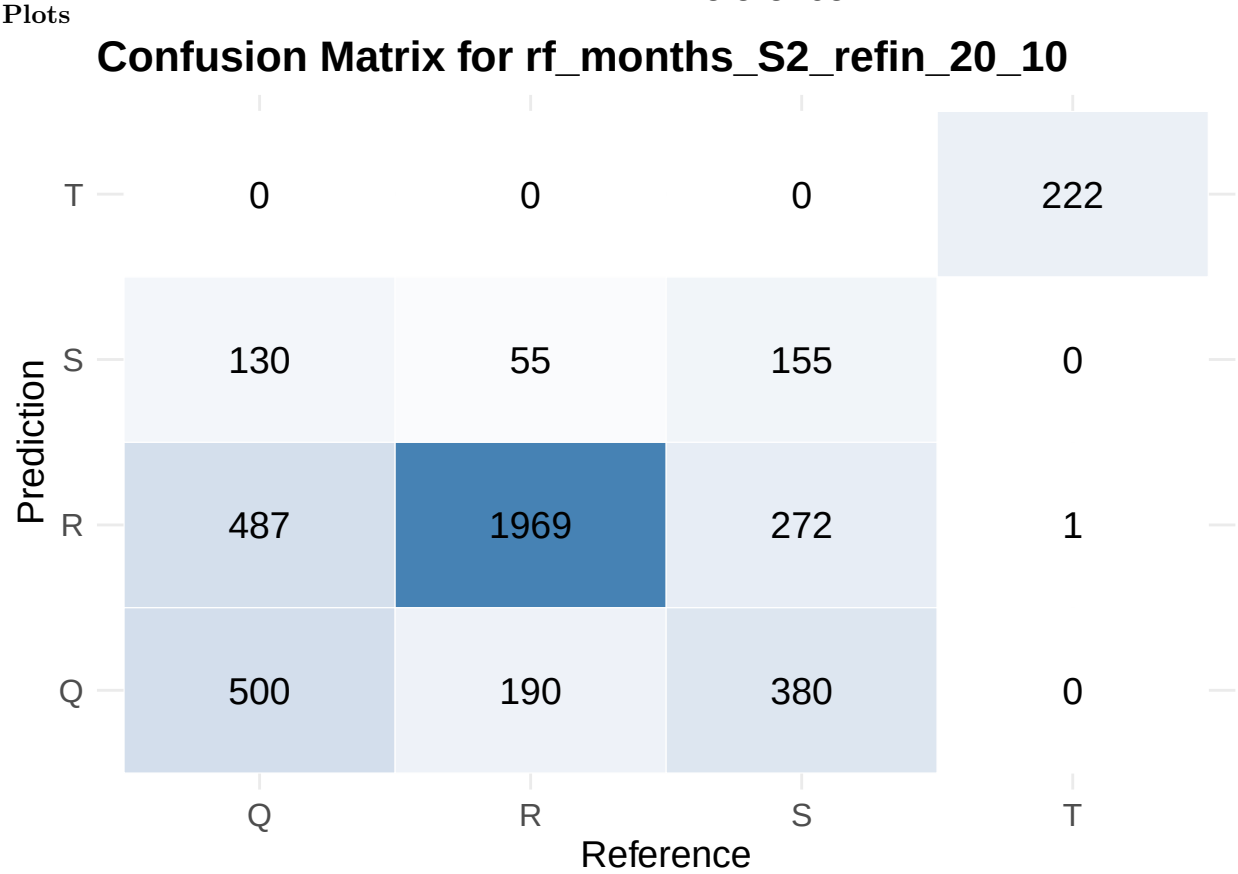
```
##
## Confusion Matrix for rf_SatEmb_refin_20_10 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  705  321  447    2
##          R  500 2474  316   13
##          S  115   78  207    0
##          T    1    1    0  184
##
## Overall Statistics
##
##                Accuracy : 0.6655
##                  95% CI : (0.6527, 0.6782)
##     No Information Rate : 0.5358
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4308
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.5337   0.8608  0.21340  0.92462
## Specificity            0.8095   0.6671  0.95608  0.99961
## Pos Pred Value         0.4780   0.7490  0.51750  0.98925
## Neg Pred Value         0.8416   0.8059  0.84629  0.99710
## Prevalence             0.2463   0.5358  0.18084  0.03710
## Detection Rate         0.1314   0.4612  0.03859  0.03430
## Detection Prevalence   0.2750   0.6158  0.07457  0.03468
## Balanced Accuracy      0.6716   0.7639  0.58474  0.96212
##
## Confusion Matrix for rf_SatEmb_CH_refin_20_10 :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  687  291  368    0
##          R  547 2490  315    1
##          S  148   76  259    0
##          T    0    0    0  222
##
## Overall Statistics
##
##                Accuracy : 0.6769
##                  95% CI : (0.6643, 0.6894)
##     No Information Rate : 0.5287
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4533
##
##  Mcnemar's Test P-Value : NA
```

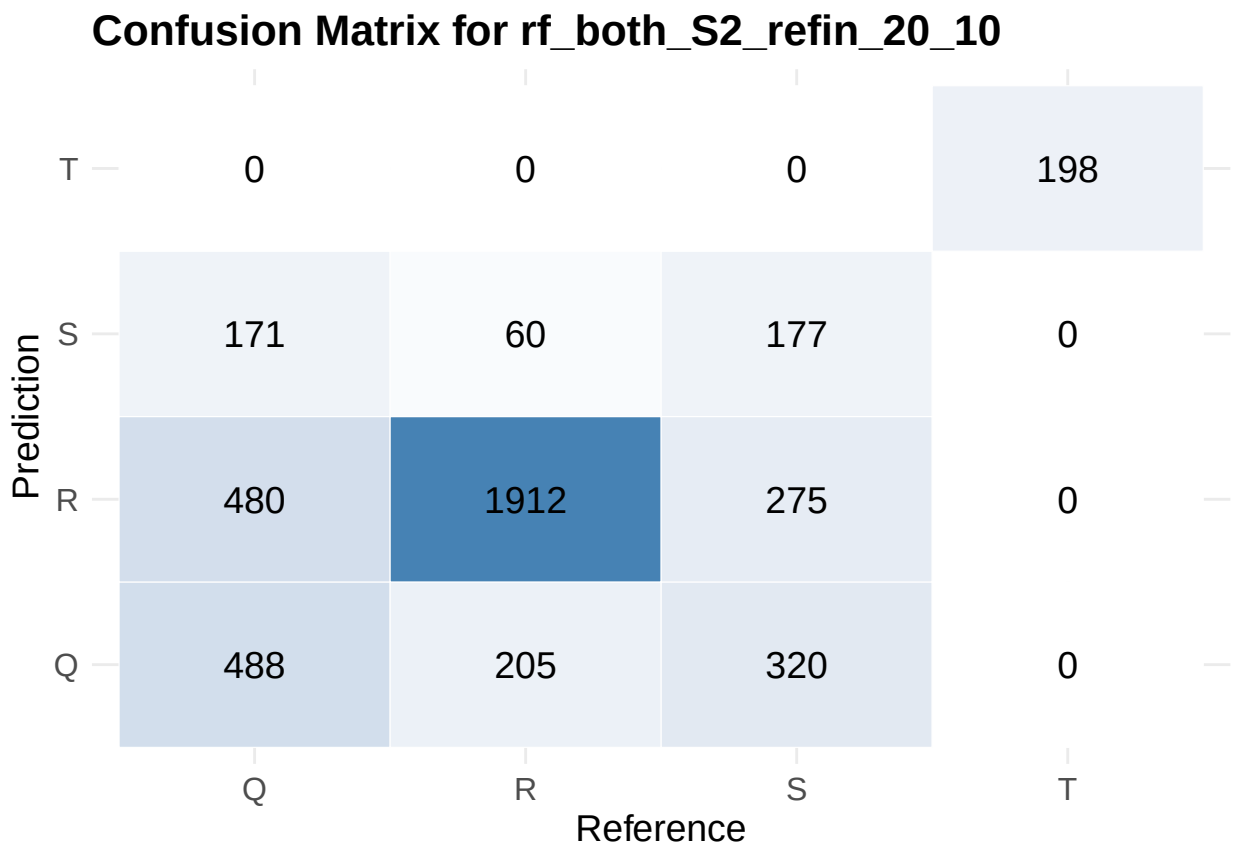
```
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.4971   0.8715  0.27495  0.99552
## Specificity            0.8362   0.6612  0.94980  1.00000
## Pos Pred Value         0.5104   0.7426  0.53623  1.00000
## Neg Pred Value         0.8287   0.8211  0.86121  0.99981
## Prevalence             0.2557   0.5287  0.17432  0.04127
## Detection Rate         0.1271   0.4608  0.04793  0.04108
## Detection Prevalence   0.2491   0.6205  0.08938  0.04108
## Balanced Accuracy      0.6666   0.7664  0.61237  0.99776
```

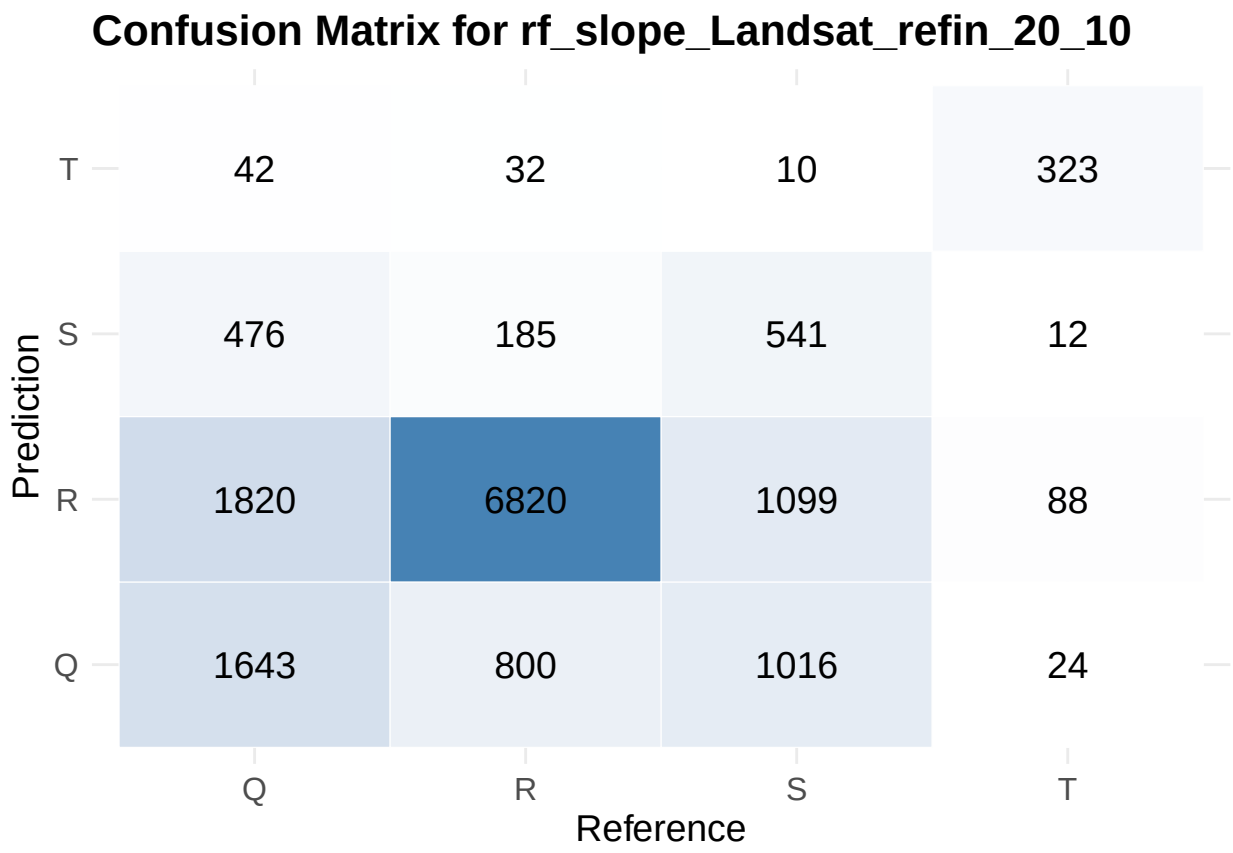
```r
for (prediction_name in predictions_refin_20_10) {
  # Derive test data name
  test_data_name <- paste0(gsub("_refin_20_10$", "",
                               sub("^rf_", "test_data_20_10_",
                                   sub("_prediction$", "", prediction_name))))

  # Obtain objects
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Personalized title
  title <- paste("Confusion Matrix for", sub("_prediction$", "",
                                             prediction_name))

  # Plot
  print(plot_confusion_matrix(prediction, test_data$EUNISa_1, title))
}
```

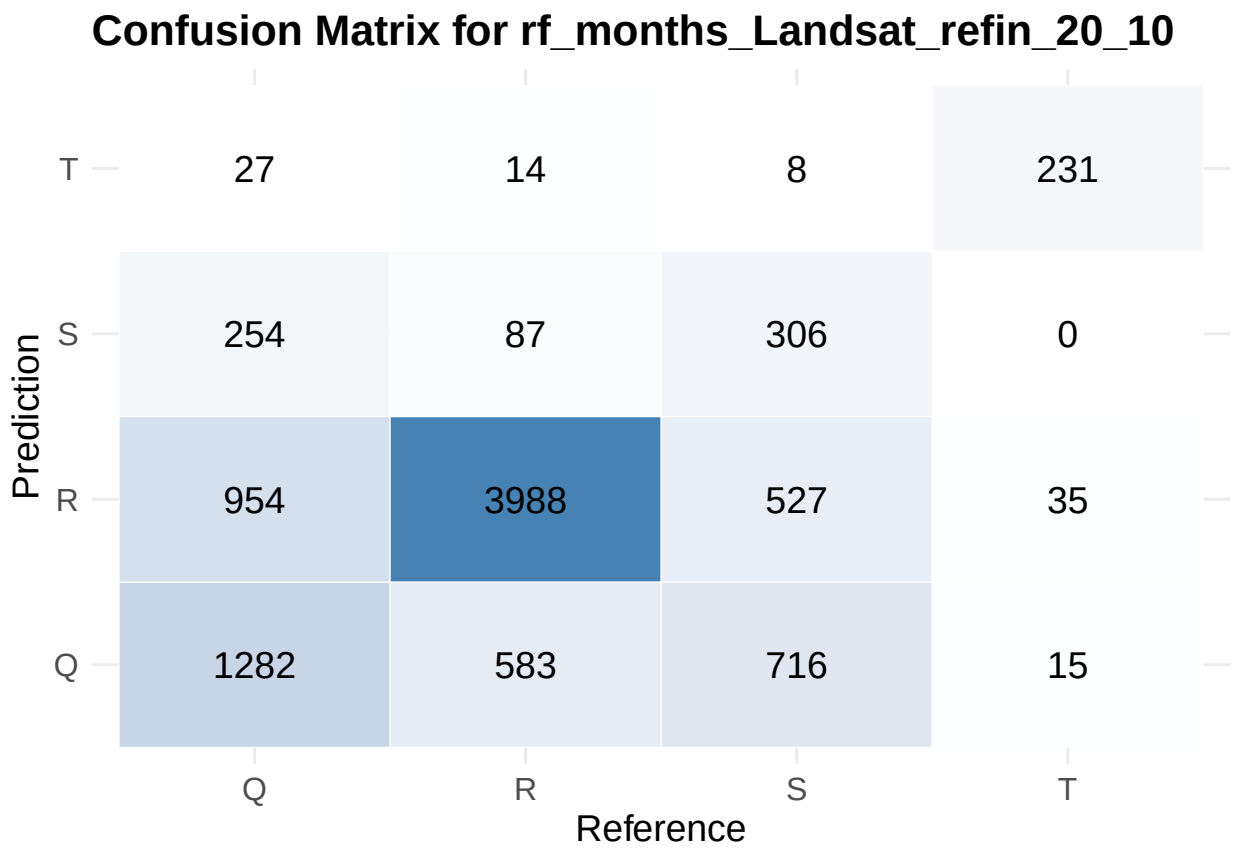

Confusion Matrix for rf_slope_S2_refin_20_10
T
S
R
Q
Prediction
0
0
0
203
107
65
110
0
710
2347
395
0
547
239
391
0
Q
R
S
T
Reference

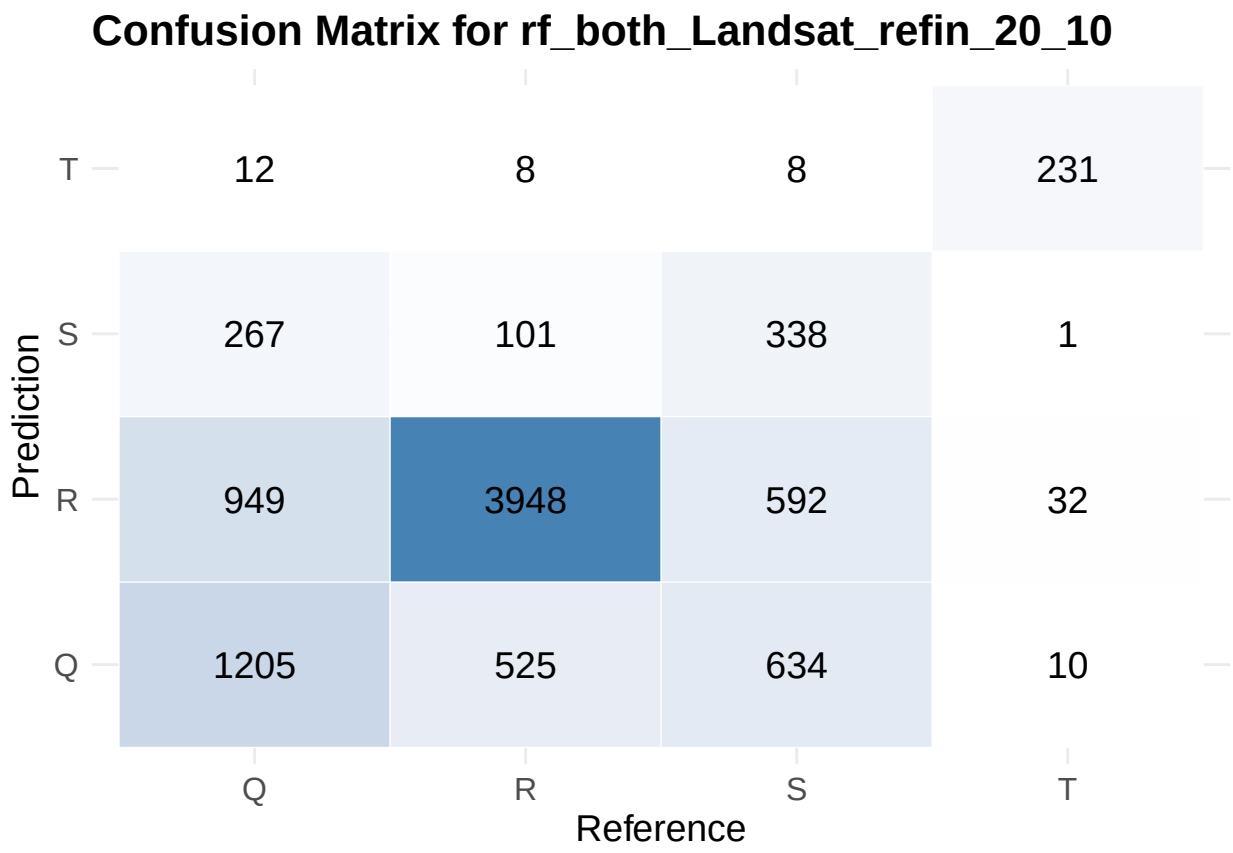
Plots

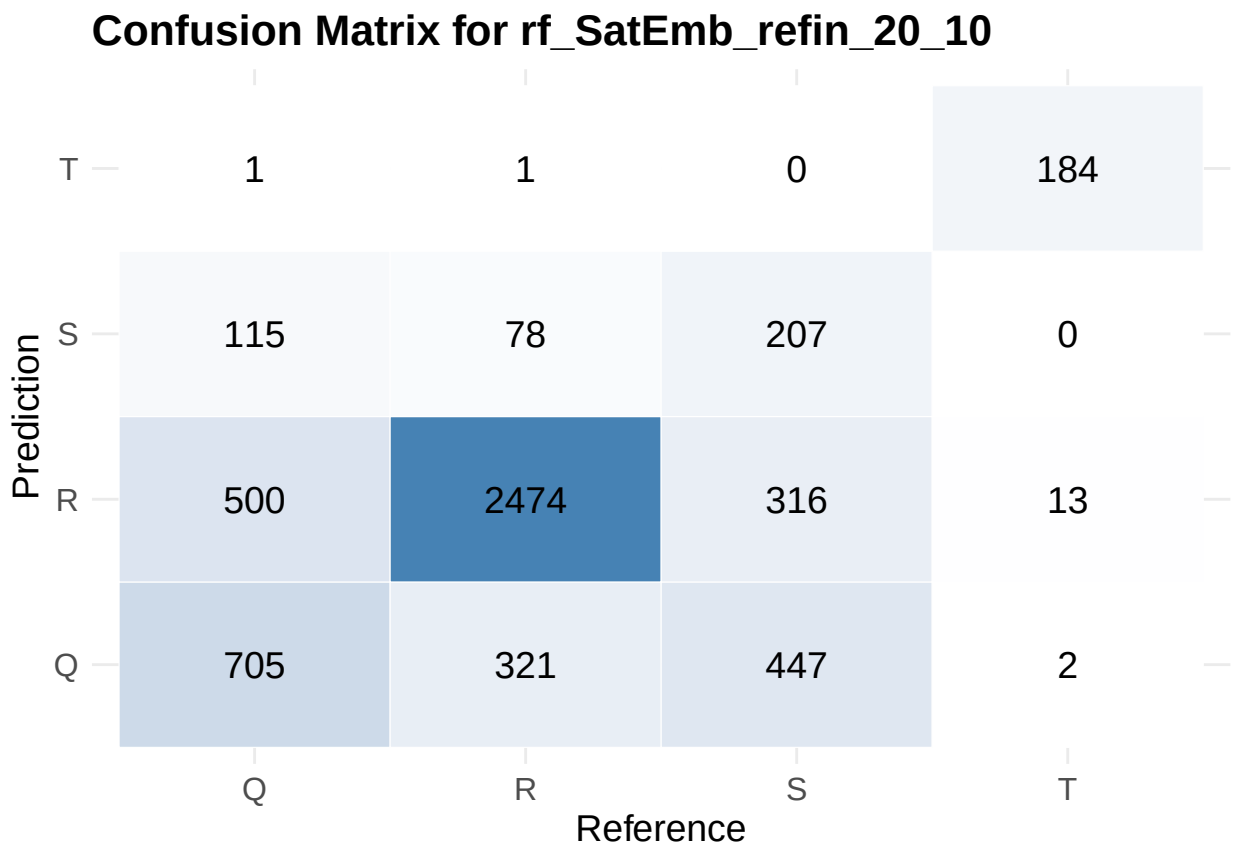

Confusion Matrix for rf_months_S2_refin_20_10
T
S
R
Q
Prediction
0
0
0
222
130
55
155
0
487
1969
272
1
500
190
380
0
Q
R
S
T
Reference

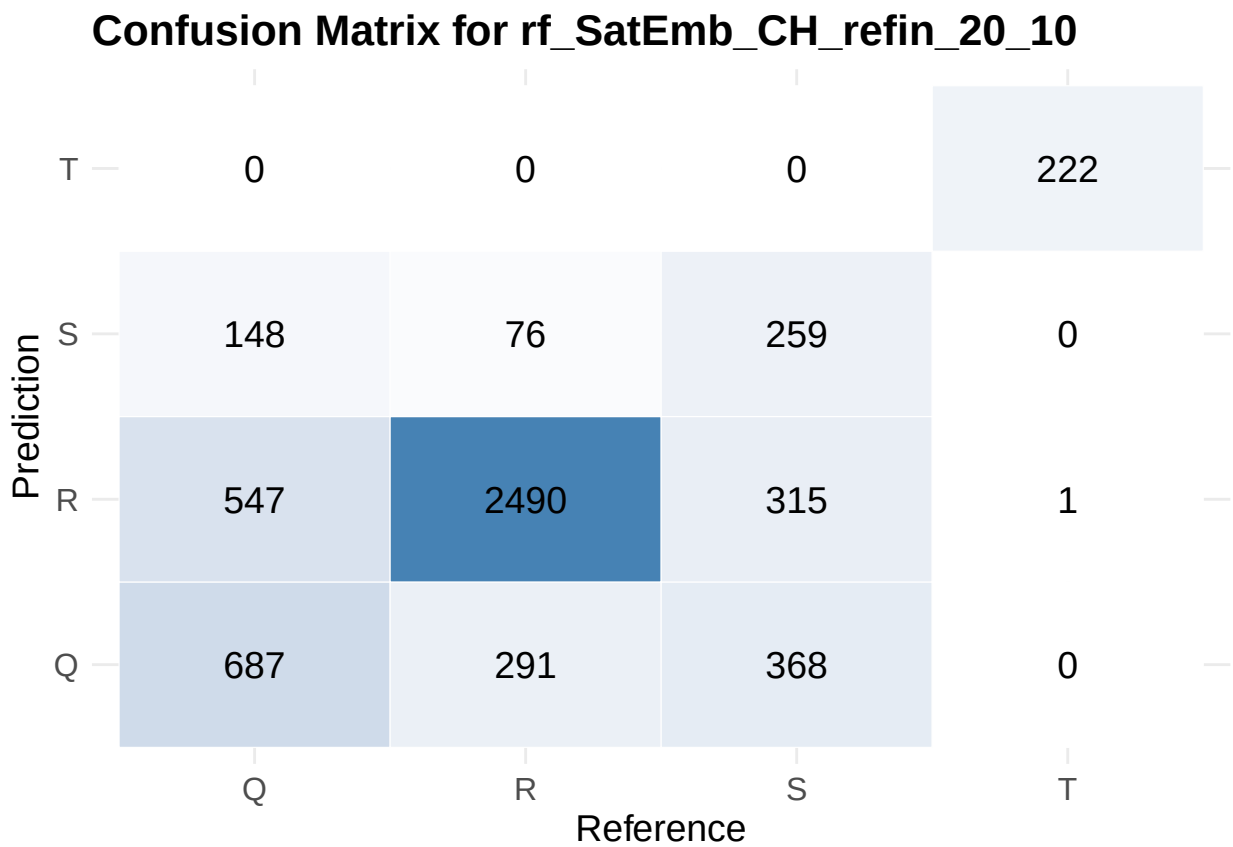

Confusion Matrix for rf_both_S2_refin_20_10
Prediction
T
S
R
Q
0
0
0
198
171
60
177
0
480
1912
275
0
488
205
320
0
Q
R
S
T
Reference


Confusion Matrix for rf_slope_Landsat_refin_20_10
Prediction
T
S
R
Q
42
32
10
323
476
185
541
12
1820
6820
1099
88
1643
800
1016
24
Q
R
S
T
Reference


Confusion Matrix for rf_months_Landsat_refin_20_10
Prediction
T
S
R
Q
27
14
8
231
254
87
306
0
954
3988
527
35
1282
583
716
15
Q
R
S
T
Reference


Confusion Matrix for rf_both_Landsat_refin_20_10
Prediction
T
S
R
Q
12
8
8
231
267
101
338
1
949
3948
592
32
1205
525
634
10
Q
R
S
T
Reference


Confusion Matrix for rf_SatEmb_refin_20_10
Prediction
T
S
R
Q
1
1
0
184
115
78
207
0
500
2474
316
13
705
321
447
2
Q
R
S
T
Reference



Refinement step 2

Then refitting the model and applying "Morralla" protocol again, removing only negatives until 20% of the class.

```
filtered_data_with_probs_2x <- c("filtered_data_slope_S2_with_probs_2x",
                                 "filtered_data_months_S2_with_probs_2x",
                                 "filtered_data_both_S2_with_probs_2x",
                                 "filtered_data_slope_Landsat_with_probs_2x",
                                 "filtered_data_months_Landsat_with_probs_2x",
                                 "filtered_data_both_Landsat_with_probs_2x",
                                 "filtered_data_SatEmb_with_probs_2x",
                                 "filtered_data_SatEmb_CH_with_probs_2x")
```

Compute predictions for all data and combine with original data Save:

```
df_aux_slope_S2_2x <- calculate_df_aux(filtered_data_slope_S2_with_probs_2x,
                                       prob_cols_slope_S2)
df_aux_months_S2_2x <- calculate_df_aux(filtered_data_months_S2_with_probs_2x,
```

```
                                    prob_cols_months_S2)
df_aux_both_S2_2x <- calculate_df_aux(filtered_data_both_S2_with_probs_2x,
                                    prob_cols_both_S2)
df_aux_slope_Landsat_2x <- calculate_df_aux(filtered_data_slope_Landsat_with_probs_2x,
                                    prob_cols_slope_Landsat)
df_aux_months_Landsat_2x <- calculate_df_aux(filtered_data_months_Landsat_with_probs_2x,
                                    prob_cols_months_Landsat)
df_aux_both_Landsat_2x <- calculate_df_aux(filtered_data_both_Landsat_with_probs_2x,
                                    prob_cols_both_Landsat)
df_aux_SatEmb_2x <- calculate_df_aux(filtered_data_SatEmb_with_probs_2x,
                                 prob_cols_SatEmb)
df_aux_SatEmb_CH_2x <- calculate_df_aux(filtered_data_SatEmb_CH_with_probs_2x,
                                    prob_cols_SatEmb_CH)
```

Calculate auxiliary metrics

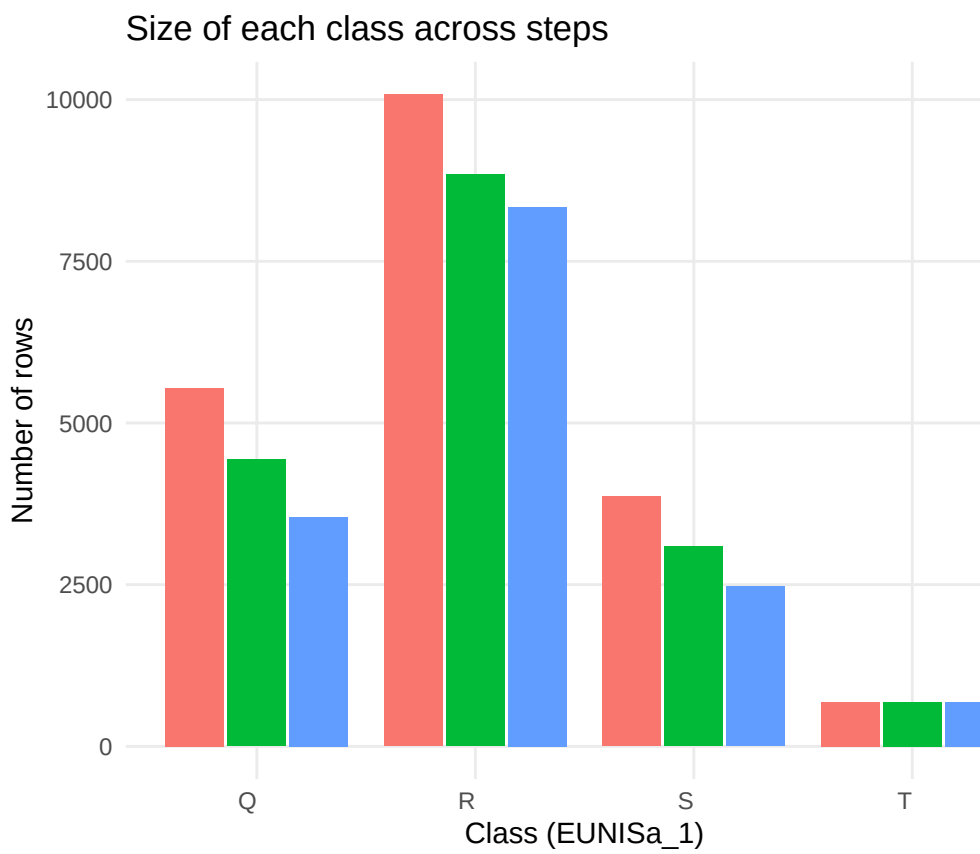
```
df_final_20_10_slope_S2_2x <- df_aux_slope_S2_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
df_final_20_10_months_S2_2x <- df_aux_months_S2_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
df_final_20_10_both_S2_2x <- df_aux_both_S2_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
df_final_20_10_slope_Landsat_2x <- df_aux_slope_Landsat_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
df_final_20_10_months_Landsat_2x <- df_aux_months_Landsat_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
df_final_20_10_both_Landsat_2x <- df_aux_both_Landsat_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
df_final_20_10_SatEmb_2x <- df_aux_SatEmb_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
df_final_20_10_SatEmb_CH_2x <- df_aux_SatEmb_CH_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
```

```
filtered_data_20_10_slope_S2_probs_2x <- filtered_data_slope_S2 %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_slope_S2_2x$PlotObservationID)
filtered_data_20_10_months_S2_probs_2x <- filtered_data_months_S2 %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_months_S2_2x$PlotObservationID)
filtered_data_20_10_both_S2_probs_2x <- filtered_data_both_S2 %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_both_S2_2x$PlotObservationID)
filtered_data_20_10_slope_Landsat_probs_2x <- filtered_data_slope_Landsat %>%
```

```
  dplyr::filter(PlotObservationID %in% df_final_20_10_slope_Landsat_2x$PlotObservationID)
filtered_data_20_10_months_Landsat_probs_2x <- filtered_data_months_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_months_Landsat_2x$PlotObservationID)
filtered_data_20_10_both_Landsat_probs_2x <- filtered_data_both_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_both_Landsat_2x$PlotObservationID)
filtered_data_20_10_SatEmb_probs_2x <- filtered_data_SatEmb %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_SatEmb_2x$PlotObservationID)
filtered_data_20_10_SatEmb_CH_probs_2x <- filtered_data_SatEmb_CH %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_SatEmb_CH_2x$PlotObservationID)
```

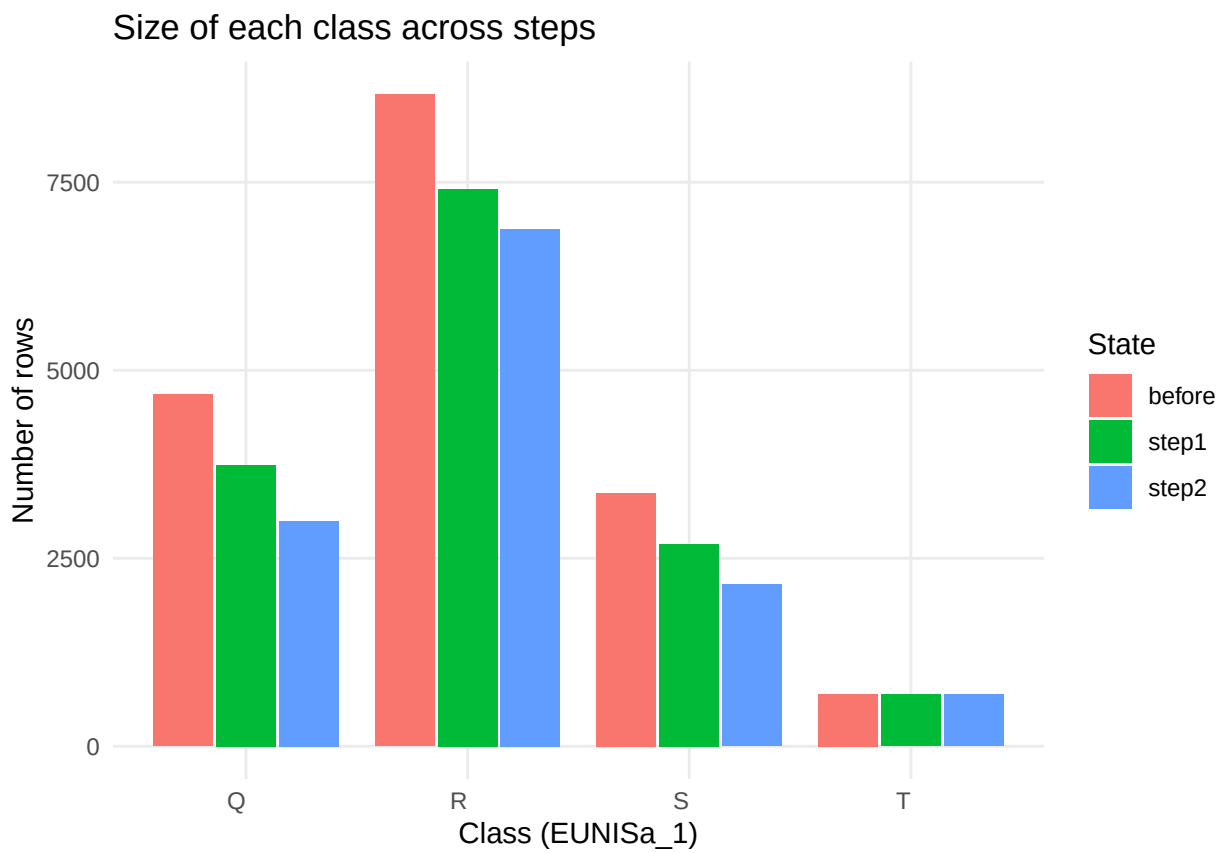
Apply filtering function

```
compare_classes(filtered_data_slope_S2, filtered_data_20_10_slope_S2_probs,
                filtered_data_20_10_slope_S2_probs_2x)
```

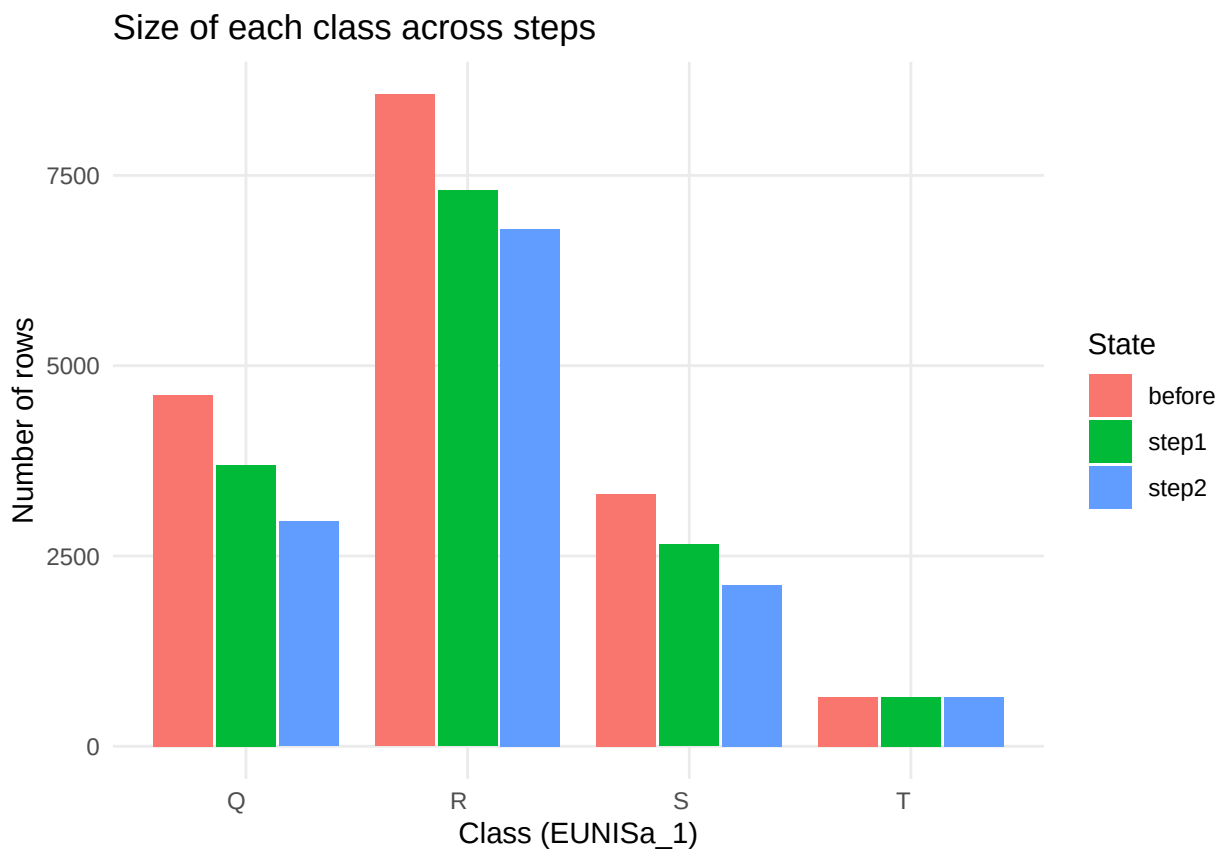


Compare classes before and after

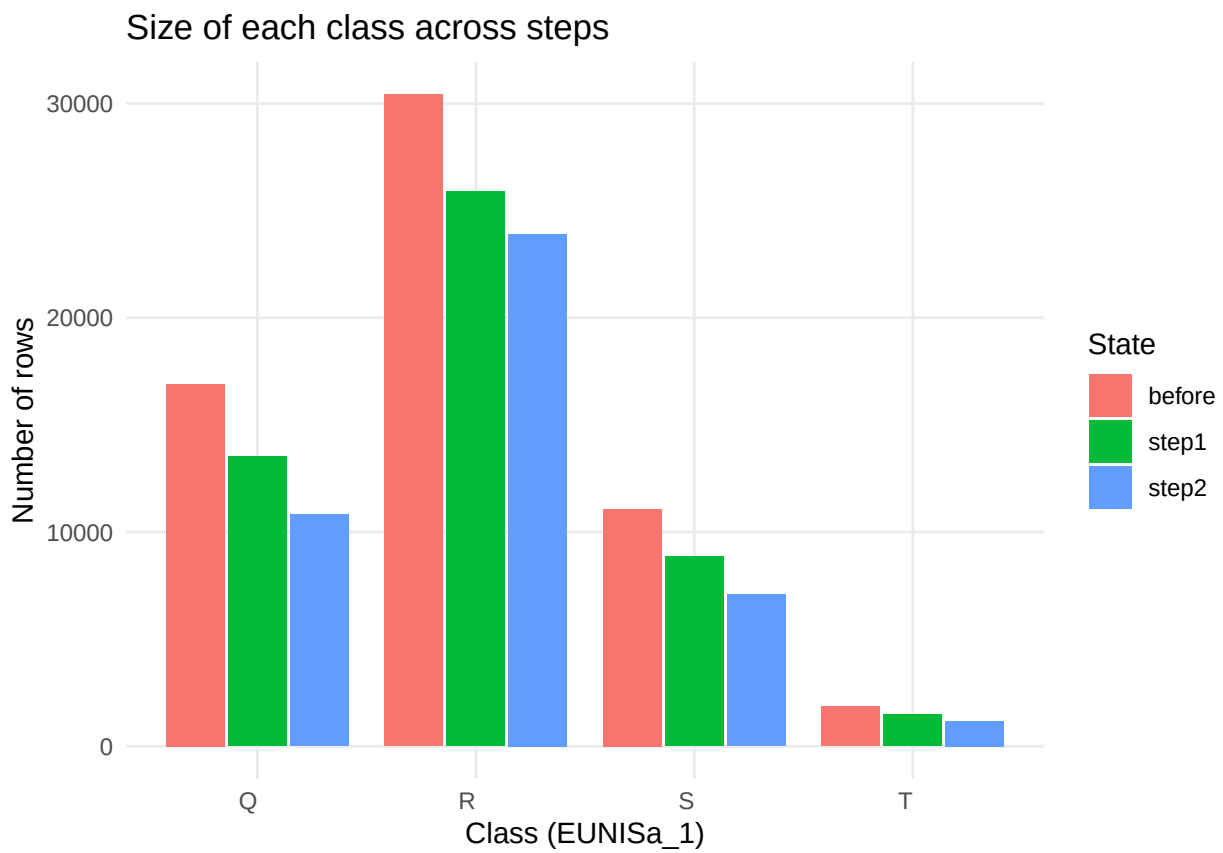
```
compare_classes(filtered_data_months_S2, filtered_data_20_10_months_S2_probs,
                filtered_data_20_10_months_S2_probs_2x)
```



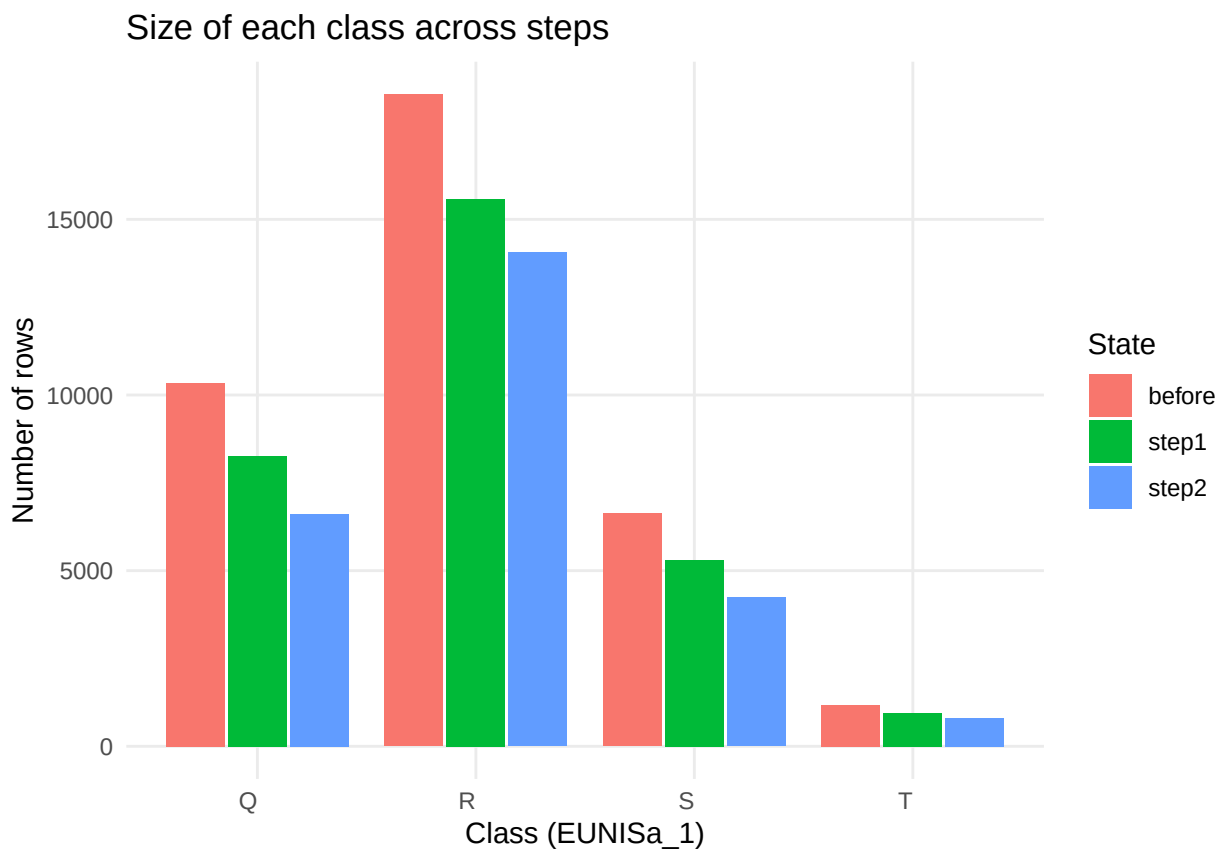

```
compare_classes(filtered_data_both_S2, filtered_data_20_10_both_S2_probs,
                filtered_data_20_10_both_S2_probs_2x)
```



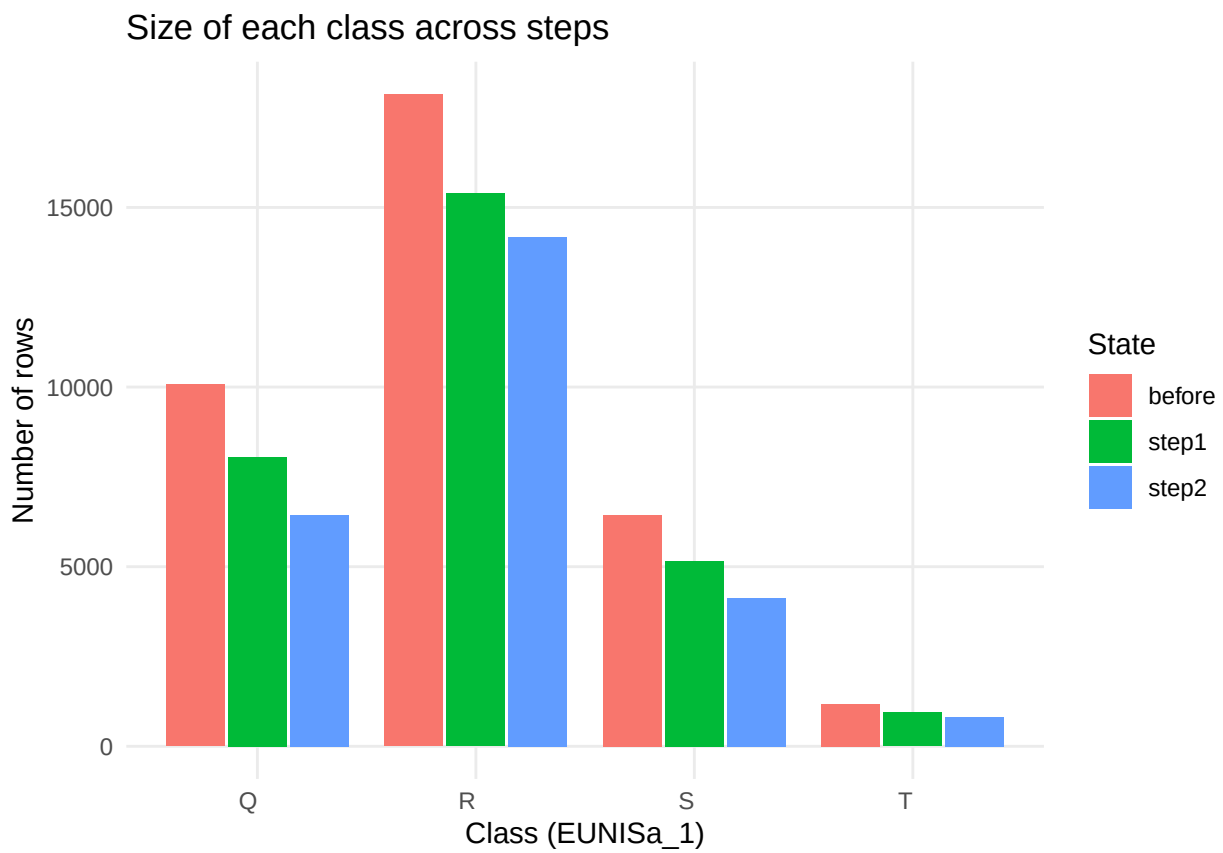

```
compare_classes(filtered_data_slope_Landsat,
                filtered_data_20_10_slope_Landsat_probs,
                filtered_data_20_10_slope_Landsat_probs_2x)
```



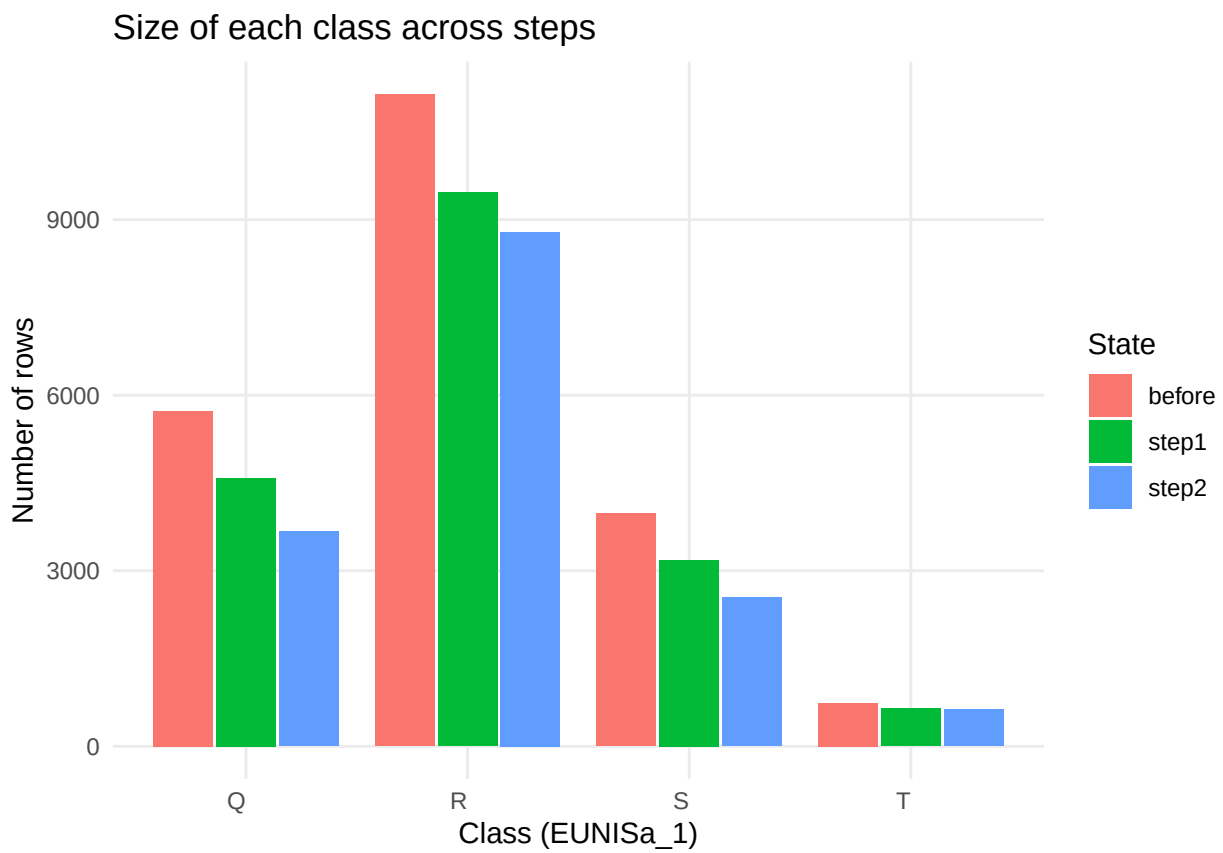

```
compare_classes(filtered_data_months_Landsat,
                filtered_data_20_10_months_Landsat_probs,
                filtered_data_20_10_months_Landsat_probs_2x)
```



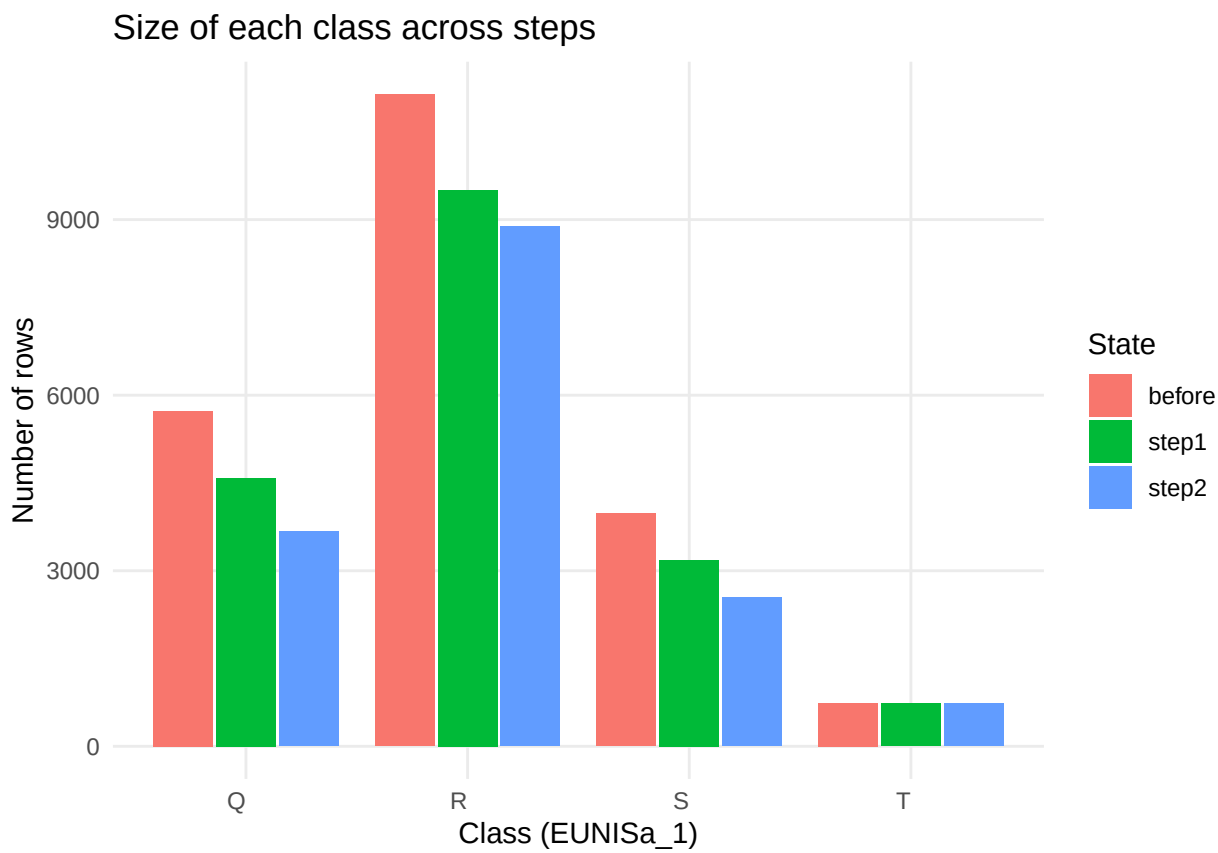

```
compare_classes(filtered_data_both_Landsat,
                filtered_data_20_10_both_Landsat_probs,
                filtered_data_20_10_both_Landsat_probs_2x)
```




```
compare_classes(filtered_data_SatEmb, filtered_data_20_10_SatEmb_probs,
                filtered_data_20_10_SatEmb_probs_2x)
```




```
compare_classes(filtered_data_SatEmb_CH, filtered_data_20_10_SatEmb_CH_probs,
                filtered_data_20_10_SatEmb_CH_probs_2x)
```




```
# S2
# Slope
set.seed(123)
split_data_20_10_2x_slope_S2 <- split_train_test(filtered_data_20_10_slope_S2_probs_2x)

train_data_20_10_2x_slope_S2 <- split_data_20_10_2x_slope_S2$train
test_data_20_10_2x_slope_S2 <- split_data_20_10_2x_slope_S2$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_slope_S2$PlotObservationID %in% test_data_20_10_2x_slope_S2$PlotObservationID)
```

Split into training and testing

```
## [1] 0
```

```
# Months
set.seed(123)
split_data_20_10_2x_months_S2 <- split_train_test(filtered_data_20_10_months_S2_probs_2x)

train_data_20_10_2x_months_S2 <- split_data_20_10_2x_months_S2$train
test_data_20_10_2x_months_S2 <- split_data_20_10_2x_months_S2$test
```

```r
# Check overlap between training and testing datasets
sum(train_data_20_10_2x_months_S2$PlotObservationID %in% test_data_20_10_2x_months_S2$PlotObservationID)
```

```
## [1] 0
```

```r
# Both
set.seed(123)
split_data_20_10_2x_both_S2 <- split_train_test(filtered_data_20_10_both_S2_probs_2x)

train_data_20_10_2x_both_S2 <- split_data_20_10_2x_both_S2$train
test_data_20_10_2x_both_S2 <- split_data_20_10_2x_both_S2$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_both_S2$PlotObservationID %in% test_data_20_10_2x_both_S2$PlotObservationID)
```

```
## [1] 0
```

```r
# Landsat
# Slope
set.seed(123)
split_data_20_10_2x_slope_Landsat <- split_train_test(filtered_data_20_10_slope_Landsat_probs_2x)

train_data_20_10_2x_slope_Landsat <- split_data_20_10_2x_slope_Landsat$train
test_data_20_10_2x_slope_Landsat <- split_data_20_10_2x_slope_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_slope_Landsat$PlotObservationID %in% test_data_20_10_2x_slope_Landsat$PlotObser
```

```
## [1] 0
```

```r
# Months
set.seed(123)
split_data_20_10_2x_months_Landsat <- split_train_test(filtered_data_20_10_months_Landsat_probs_2x)

train_data_20_10_2x_months_Landsat <- split_data_20_10_2x_months_Landsat$train
test_data_20_10_2x_months_Landsat <- split_data_20_10_2x_months_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_months_Landsat$PlotObservationID %in% test_data_20_10_2x_months_Landsat$PlotObs
```

```
## [1] 0
```

```r
# Both
set.seed(123)
split_data_20_10_2x_both_Landsat <- split_train_test(filtered_data_20_10_both_Landsat_probs_2x)

train_data_20_10_2x_both_Landsat <- split_data_20_10_2x_both_Landsat$train
test_data_20_10_2x_both_Landsat <- split_data_20_10_2x_both_Landsat$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_both_Landsat$PlotObservationID %in% test_data_20_10_2x_both_Landsat$PlotObserva
```

```
## [1] 0
```

```r
# SatEmb
set.seed(123)
split_data_20_10_2x_SatEmb <- split_train_test(filtered_data_20_10_SatEmb_probs_2x)

train_data_20_10_2x_SatEmb <- split_data_20_10_2x_SatEmb$train
test_data_20_10_2x_SatEmb <- split_data_20_10_2x_SatEmb$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_SatEmb$PlotObservationID %in% test_data_20_10_2x_SatEmb$PlotObservationID)
```

```
## [1] 0
```

```r
# SatEmb_CH
set.seed(123)
split_data_20_10_2x_SatEmb_CH <- split_train_test(filtered_data_20_10_SatEmb_CH_probs)

train_data_20_10_2x_SatEmb_CH <- split_data_20_10_2x_SatEmb_CH$train
test_data_20_10_2x_SatEmb_CH <- split_data_20_10_2x_SatEmb_CH$test

# Check overlap between training and testing datasets
sum(train_data_20_10_2x_SatEmb_CH$PlotObservationID %in% test_data_20_10_2x_SatEmb_CH$PlotObservationID
```

```
## [1] 0
```

```r
models_refin_20_10_2x <- c("rf_slope_S2_refin_20_10_2x",
                      "rf_months_S2_refin_20_10_2x",
                      "rf_both_S2_refin_20_10_2x",
                      "rf_slope_Landsat_refin_20_10_2x",
                      "rf_months_Landsat_refin_20_10_2x",
                      "rf_both_Landsat_refin_20_10_2x",
                      "rf_SatEmb_refin_20_10_2x",
                      "rf_SatEmb_CH_refin_20_10_2x")
```

Fit models Save:

```r
predictions_refin_20_10_2x <- c("rf_slope_S2_refin_20_10_2x_prediction",
                         "rf_months_S2_refin_20_10_2x_prediction",
                         "rf_both_S2_refin_20_10_2x_prediction",
                         "rf_slope_Landsat_refin_20_10_2x_prediction",
                         "rf_months_Landsat_refin_20_10_2x_prediction",
                         "rf_both_Landsat_refin_20_10_2x_prediction",
                         "rf_SatEmb_refin_20_10_2x_prediction",
                         "rf_SatEmb_CH_refin_20_10_2x_prediction")
```

Predictions Save:

```r
for (prediction_name in predictions_refin_20_10_2x) {
  # Derive test data name
  test_data_name <- paste0(gsub("_refin_20_10_2x$", "",
                              sub("^rf_", "test_data_20_10_2x_",
                                  sub("_prediction$", "", prediction_name))))

  # Get prediction and test data
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Compute confusion matrix
  cat("Confusion Matrix for", sub("_prediction$", "", prediction_name), ":\n")
  print(confusionMatrix(prediction, test_data$EUNISa_1))
  cat("\n")
}
```

Confusion matrices

```
## Confusion Matrix for rf_slope_S2_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  528  169  386    0
##          R  419 2290  240    0
##          S  118   45  115    0
##          T    0    0    0  202
##
## Overall Statistics
##
##                Accuracy : 0.6948
##                  95% CI : (0.6811, 0.7082)
##     No Information Rate : 0.555
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4632
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.4958   0.9145  0.15520  1.00000
## Specificity            0.8390   0.6718  0.95678  1.00000
## Pos Pred Value         0.4875   0.7765  0.41367  1.00000
## Neg Pred Value         0.8434   0.8631  0.85215  1.00000
## Prevalence             0.2360   0.5550  0.16423  0.04477
## Detection Rate         0.1170   0.5075  0.02549  0.04477
## Detection Prevalence   0.2400   0.6536  0.06161  0.04477
## Balanced Accuracy      0.6674   0.7932  0.55599  1.00000
##
## Confusion Matrix for rf_months_S2_refin_20_10_2x :
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  516   95  386    0
##          R  232 1966   98    0
##          S  122   34  145    0
##          T    0    0    0  219
##
## Overall Statistics
##
##                Accuracy : 0.7464
##                  95% CI : (0.7323, 0.7601)
##     No Information Rate : 0.5494
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5725
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.5931   0.9384  0.23052  1.00000
## Specificity            0.8366   0.8079  0.95101  1.00000
## Pos Pred Value         0.5176   0.8563  0.48173  1.00000
## Neg Pred Value         0.8743   0.9150  0.86219  1.00000
## Prevalence             0.2282   0.5494  0.16496  0.05744
## Detection Rate         0.1353   0.5156  0.03803  0.05744
## Detection Prevalence   0.2615   0.6022  0.07894  0.05744
## Balanced Accuracy      0.7148   0.8732  0.59076  1.00000
##
## Confusion Matrix for rf_both_S2_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  533  129  372    0
##          R  241 1845  154    1
##          S  107   22  142    0
##          T    0    0    0  208
##
## Overall Statistics
##
##                Accuracy : 0.7267
##                  95% CI : (0.7121, 0.7409)
##     No Information Rate : 0.5317
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.546
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
```

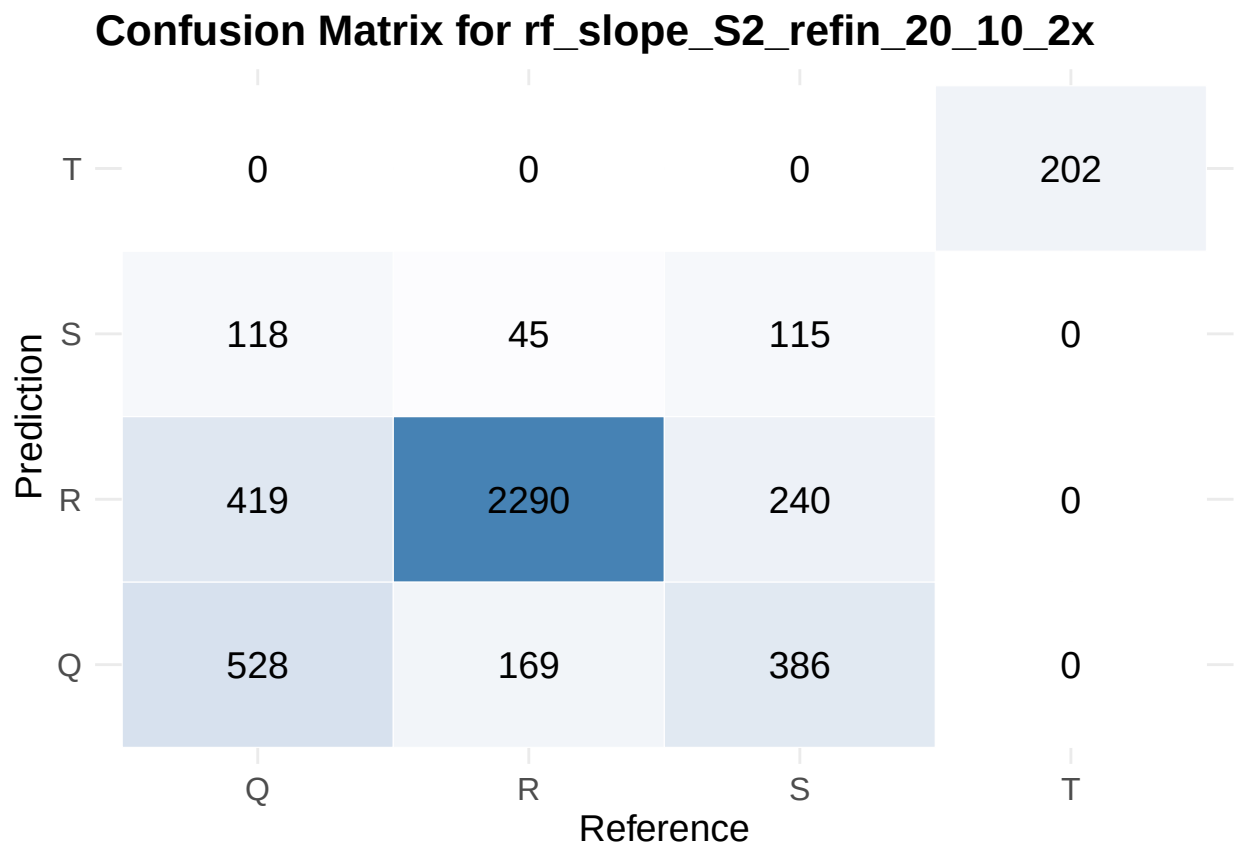
```
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.6050   0.9243  0.21257  0.99522
## Specificity            0.8256   0.7747  0.95820  1.00000
## Pos Pred Value         0.5155   0.8233  0.52399  1.00000
## Neg Pred Value         0.8721   0.9002  0.84898  0.99972
## Prevalence             0.2347   0.5317  0.17794  0.05567
## Detection Rate         0.1420   0.4915  0.03783  0.05541
## Detection Prevalence   0.2754   0.5970  0.07219  0.05541
## Balanced Accuracy      0.7153   0.8495  0.58539  0.99761
##
## Confusion Matrix for rf_slope_Landsat_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q 1700  455  939    7
##          R 1046 6579  479   14
##          S  489  152  644    1
##          T   16   25    4  351
##
## Overall Statistics
##
##                Accuracy : 0.7189
##                  95% CI : (0.711, 0.7266)
##     No Information Rate : 0.5589
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5075
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.5229   0.9124  0.31171  0.94102
## Specificity            0.8548   0.7295  0.94075  0.99641
## Pos Pred Value         0.5482   0.8104  0.50078  0.88636
## Neg Pred Value         0.8417   0.8679  0.87757  0.99824
## Prevalence             0.2520   0.5589  0.16014  0.02891
## Detection Rate         0.1318   0.5100  0.04992  0.02721
## Detection Prevalence   0.2404   0.6293  0.09968  0.03070
## Balanced Accuracy      0.6889   0.8209  0.62623  0.96871
##
## Confusion Matrix for rf_months_Landsat_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q 1223  295  645    6
##          R  468 3830  278   16
##          S  293   64  353    0
##          T    8    8    2  226
##
```

```
## Overall Statistics
##
##                Accuracy : 0.73
##                  95% CI : (0.7199, 0.7399)
##     No Information Rate : 0.544
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5403
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.6140   0.9126  0.27621  0.91129
## Specificity            0.8347   0.7834  0.94454  0.99759
## Pos Pred Value         0.5639   0.8341  0.49718  0.92623
## Neg Pred Value         0.8613   0.8825  0.86795  0.99706
## Prevalence             0.2582   0.5440  0.16565  0.03215
## Detection Rate         0.1585   0.4964  0.04576  0.02929
## Detection Prevalence   0.2811   0.5952  0.09203  0.03163
## Balanced Accuracy      0.7243   0.8480  0.61038  0.95444
##
## Confusion Matrix for rf_both_Landsat_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q 1153  326  556    2
##          R  514 3836  335   11
##          S  234   84  370    0
##          T    9    3    3  233
##
## Overall Statistics
##
##                Accuracy : 0.7292
##                  95% CI : (0.7191, 0.7391)
##     No Information Rate : 0.554
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5321
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.6037   0.9028  0.29272  0.94715
## Specificity            0.8465   0.7485  0.95035  0.99798
## Pos Pred Value         0.5660   0.8169  0.53779  0.93952
## Neg Pred Value         0.8656   0.8611  0.87194  0.99825
## Prevalence             0.2491   0.5540  0.16482  0.03208
## Detection Rate         0.1503   0.5002  0.04825  0.03038
## Detection Prevalence   0.2656   0.6123  0.08971  0.03234
```

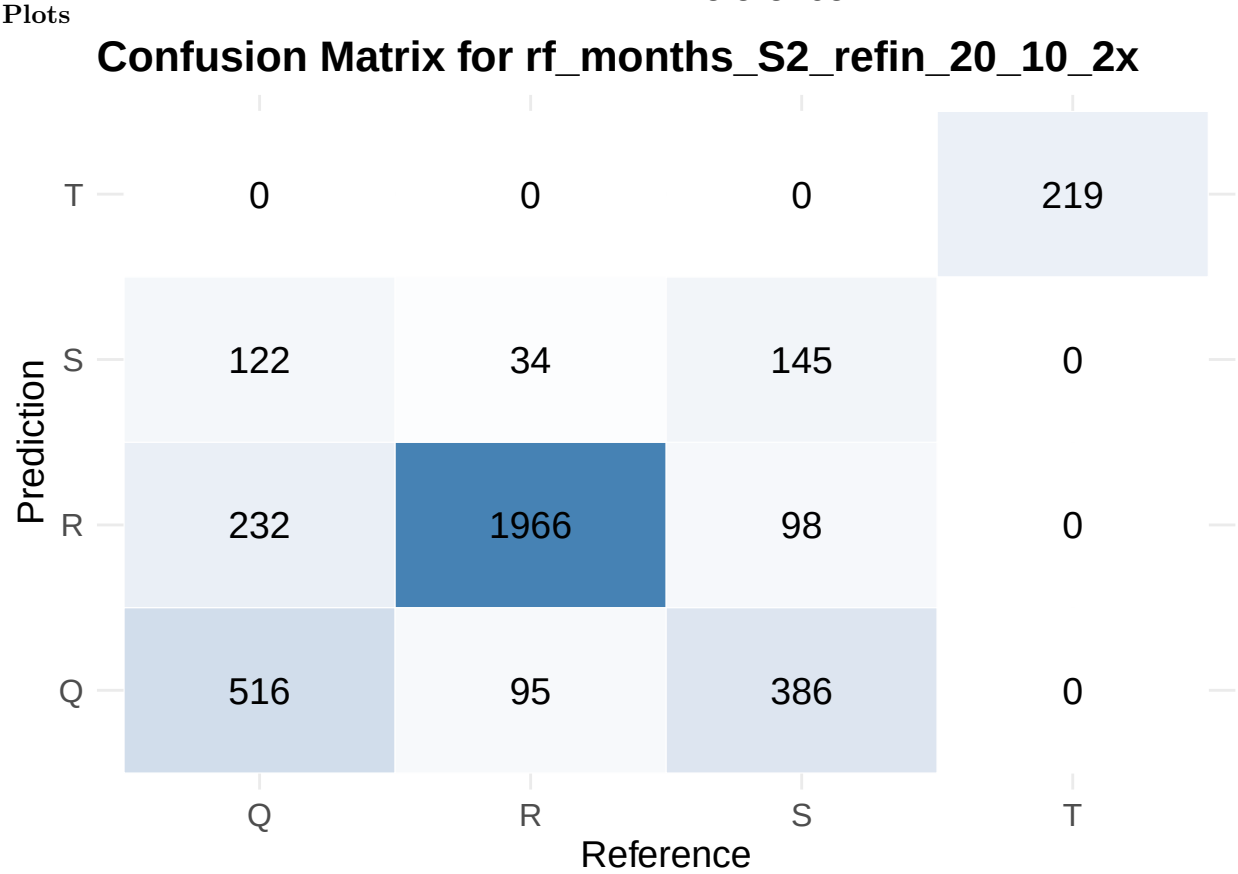
```
## Balanced Accuracy      0.7251   0.8257  0.62154  0.97257
##
## Confusion Matrix for rf_SatEmb_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  584  156  317    0
##          R  330 2466  173    5
##          S  182   47  243    0
##          T    0    3    0  183
##
## Overall Statistics
##
##                Accuracy : 0.7413
##                  95% CI : (0.7285, 0.7538)
##     No Information Rate : 0.5698
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.545
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.5328   0.9229  0.33151  0.97340
## Specificity            0.8684   0.7481  0.94211  0.99933
## Pos Pred Value         0.5525   0.8292  0.51483  0.98387
## Neg Pred Value         0.8590   0.8799  0.88380  0.99889
## Prevalence             0.2337   0.5698  0.15632  0.04009
## Detection Rate         0.1245   0.5259  0.05182  0.03903
## Detection Prevalence   0.2254   0.6343  0.10066  0.03967
## Balanced Accuracy      0.7006   0.8355  0.63681  0.98637
##
## Confusion Matrix for rf_SatEmb_CH_refin_20_10_2x :
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    Q    R    S    T
##          Q  684  275  340    0
##          R  551 2507  341    1
##          S  147   73  261    0
##          T    0    2    0  222
##
## Overall Statistics
##
##                Accuracy : 0.6799
##                  95% CI : (0.6672, 0.6923)
##     No Information Rate : 0.5287
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4562
##
```

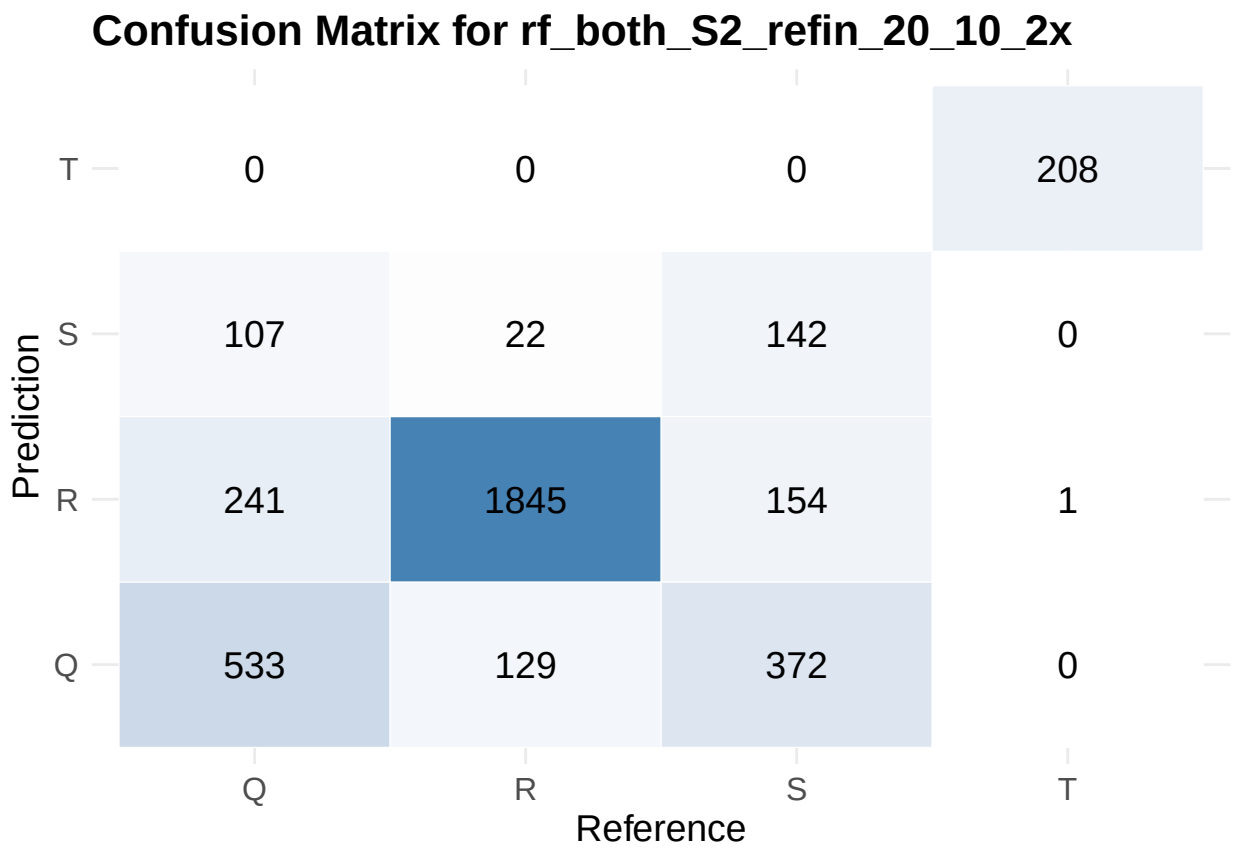
```
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Q Class: R Class: S Class: T
## Sensitivity            0.4949   0.8775  0.27707  0.99552
## Specificity            0.8471   0.6494  0.95069  0.99961
## Pos Pred Value         0.5266   0.7374  0.54262  0.99107
## Neg Pred Value         0.8300   0.8253  0.86167  0.99981
## Prevalence             0.2557   0.5287  0.17432  0.04127
## Detection Rate         0.1266   0.4639  0.04830  0.04108
## Detection Prevalence   0.2404   0.6292  0.08901  0.04145
## Balanced Accuracy      0.6710   0.7634  0.61388  0.99756
```

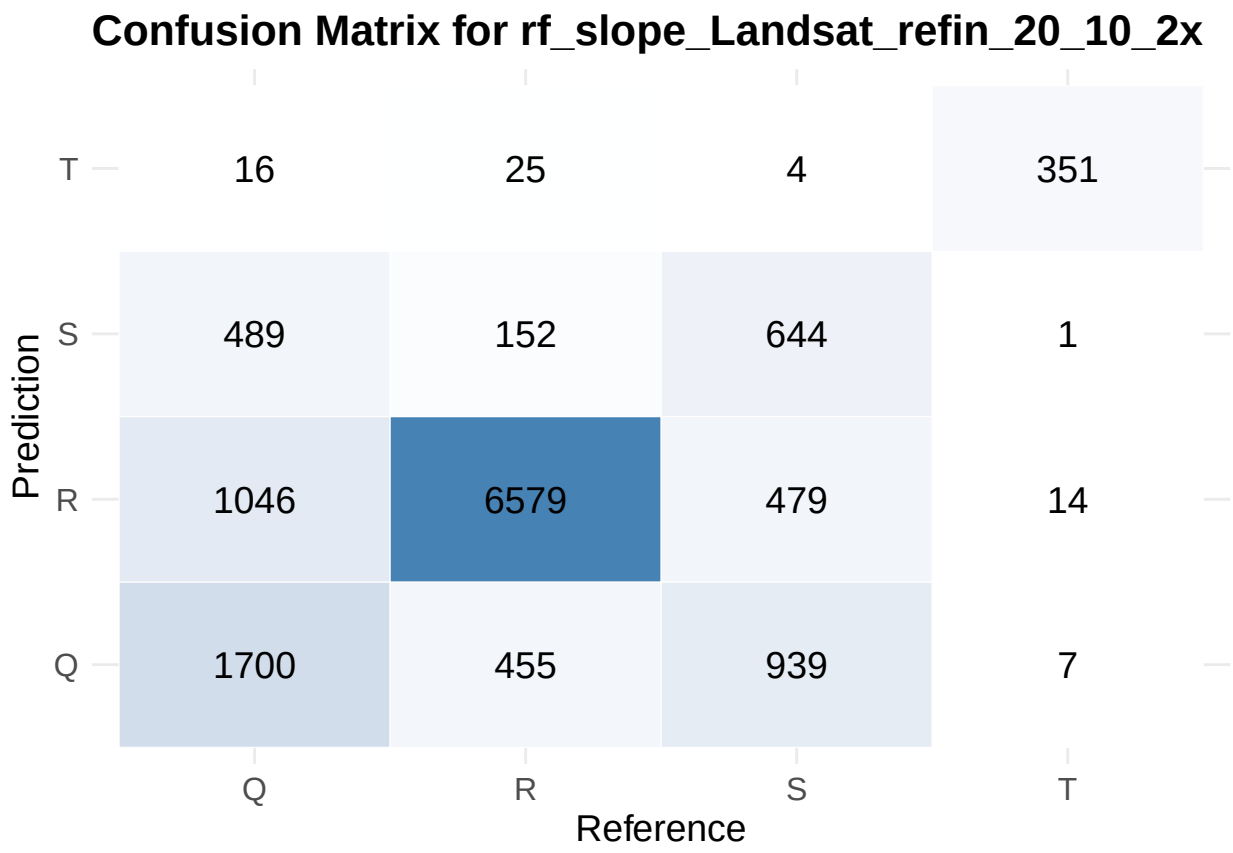
```r
for (prediction_name in predictions_refin_20_10_2x) {
  # Derive test data name
  test_data_name <- paste0(gsub("_refin_20_10_2x$", "",
                              sub("^rf_", "test_data_20_10_2x_",
                                  sub("_prediction$", "", prediction_name))))

  # Obtain objects
  prediction <- get(prediction_name)
  test_data <- get(test_data_name)

  # Personalized title
  title <- paste("Confusion Matrix for", sub("_prediction$", "",
                                             prediction_name))

  # Plot
  print(plot_confusion_matrix(prediction, test_data$EUNISa_1, title))
}
```

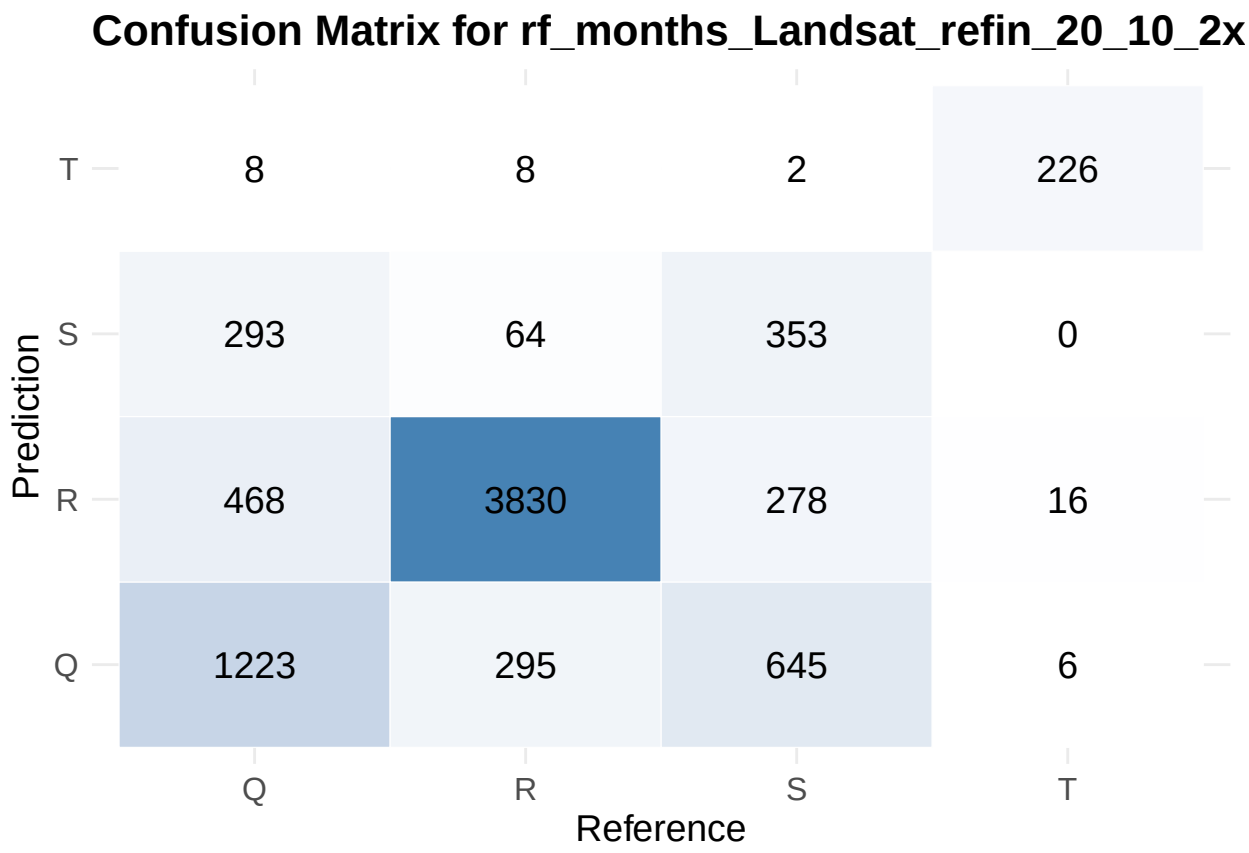

Confusion Matrix for rf_slope_S2_refin_20_10_2x
T
S
R
Q
Prediction
0
0
0
202
118
45
115
0
419
2290
240
0
528
169
386
0
Q
R
S
T
Reference

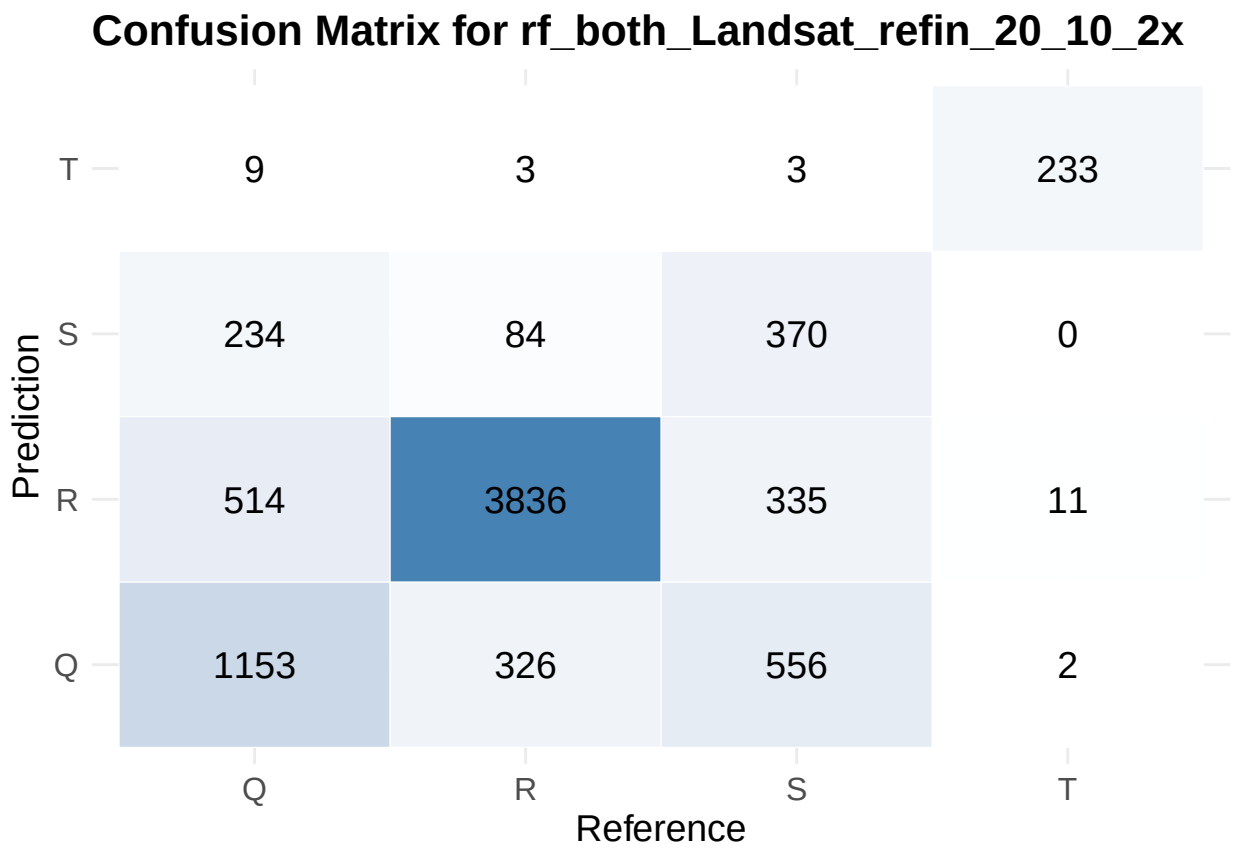
Plots

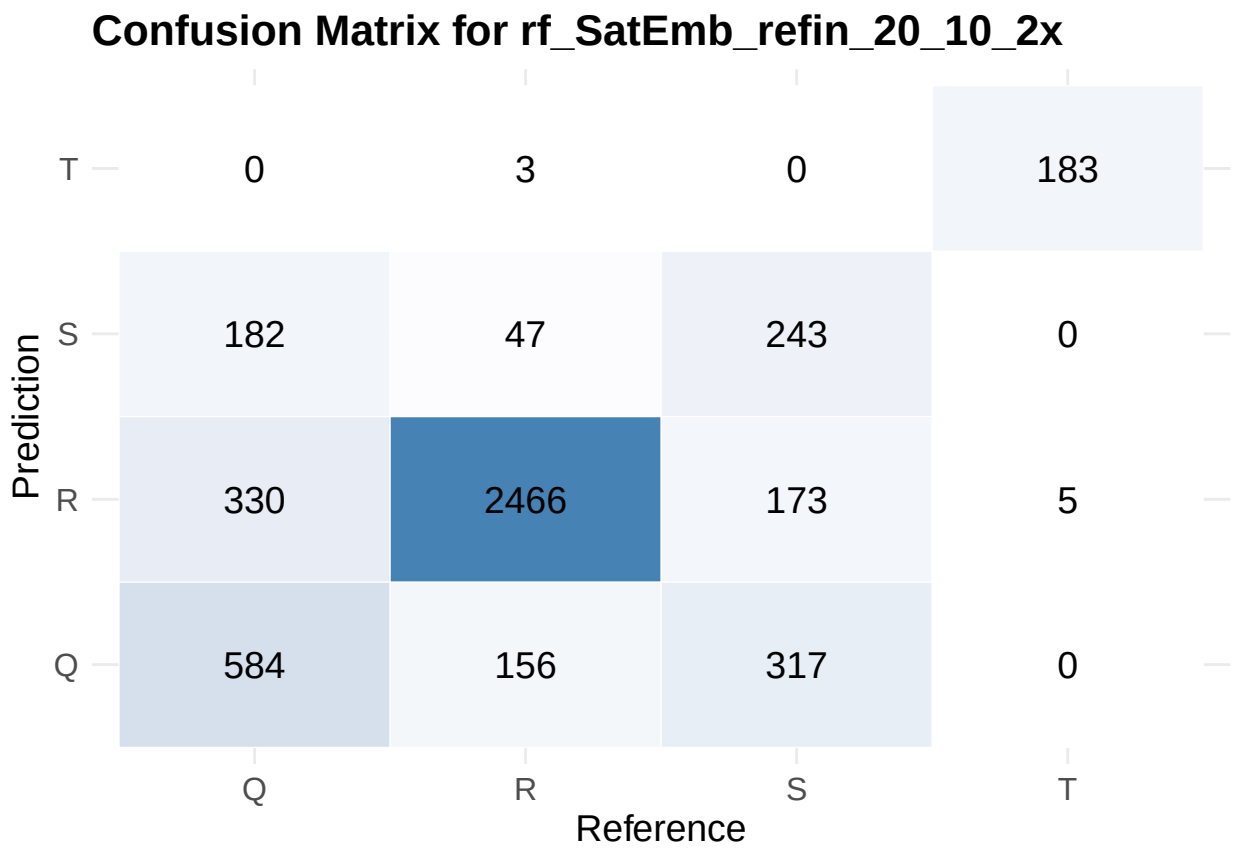

Confusion Matrix for rf_months_S2_refin_20_10_2x
T
S
R
Q
Prediction
0
0
0
219
122
34
145
0
232
1966
98
0
516
95
386
0
Q
R
S
T
Reference

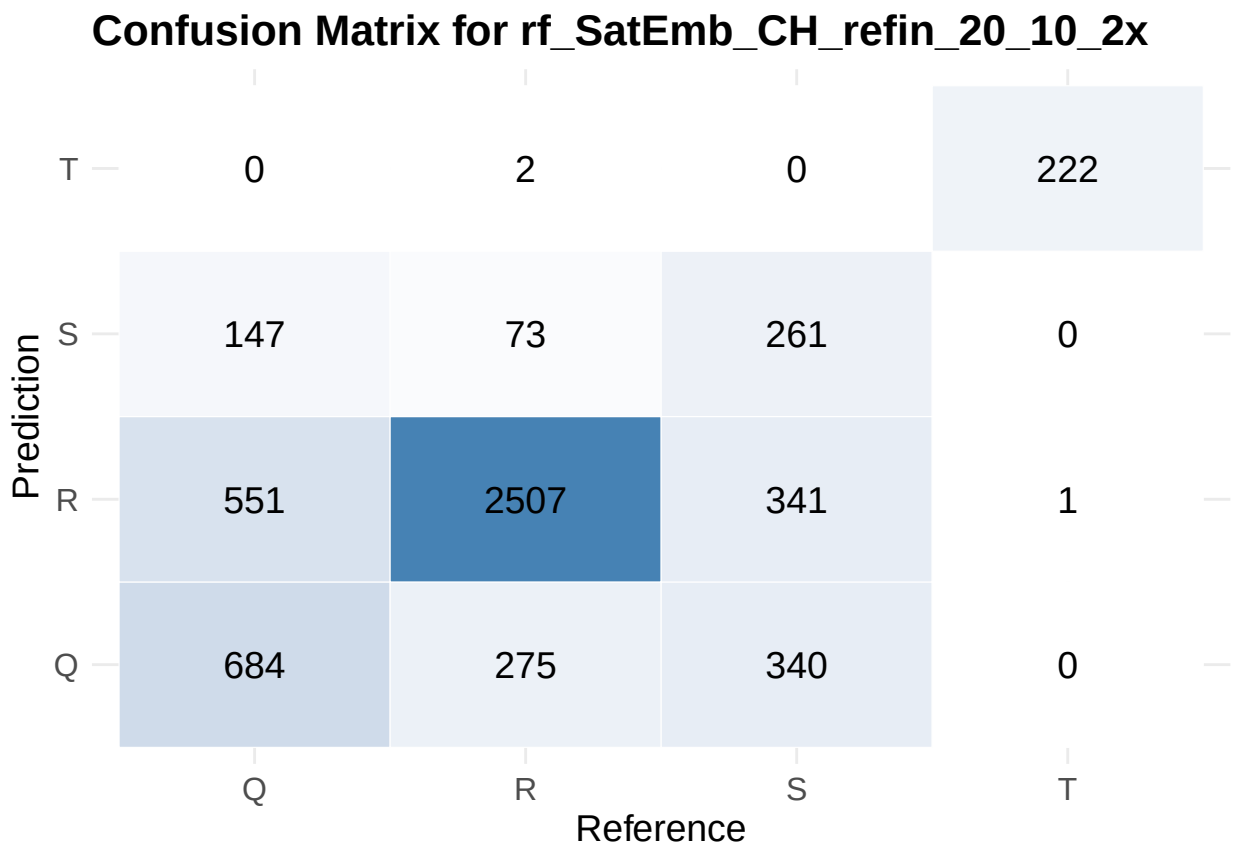

Confusion Matrix for rf_both_S2_refin_20_10_2x
Prediction
T
S
R
Q
0
0
0
208
107
22
142
0
241
1845
154
1
533
129
372
0
Q
R
S
T
Reference


Confusion Matrix for rf_slope_Landsat_refin_20_10_2x
Prediction
T
S
R
Q
16
25
4
351
489
152
644
1
1046
6579
479
14
1700
455
939
7
Q
R
S
T
Reference


Confusion Matrix for rf_months_Landsat_refin_20_10_2x
Prediction
T
S
R
Q
8
8
2
226
293
64
353
0
468
3830
278
16
1223
295
645
6
Q
R
S
T
Reference


Confusion Matrix for rf_both_Landsat_refin_20_10_2x
Prediction
T
S
R
Q
9
3
3
233
234
84
370
0
514
3836
335
11
1153
326
556
2
Q
R
S
T
Reference


Confusion Matrix for rf_SatEmb_refin_20_10_2x
Prediction
T
S
R
Q
0
3
0
183
182
47
243
0
330
2466
173
5
584
156
317
0
Q
R
S
T
Reference



Session info

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Spanish_Spain.utf8  LC_CTYPE=Spanish_Spain.utf8
## [3] LC_MONETARY=Spanish_Spain.utf8 LC_NUMERIC=C
## [5] LC_TIME=Spanish_Spain.utf8
##
## time zone: Europe/Madrid
## tzcode source: internal
##
## attached base packages:
##  [1] stats4    grid      parallel  stats     graphics  grDevices utils
##  [8] datasets  methods   base
##
```

```
## other attached packages:
##  [1] patchwork_1.3.2      gghalves_0.1.4       permimp_1.1-0
##  [4] beepr_2.0            rlang_1.1.6          pROC_1.19.0.1
##  [7] randomForest_4.7-1.2 caret_7.0-1          lattice_0.22-7
## [10] moreparty_0.4.2      party_1.3-18         strucchange_1.5-4
## [13] sandwich_3.1-1       zoo_1.8-14           modeltools_0.2-24
## [16] mvtnorm_1.3-3        doParallel_1.0.17    iterators_1.0.14
## [19] foreach_1.5.2        here_1.0.2           lubridate_1.9.4
## [22] forcats_1.0.1        stringr_1.6.0        dplyr_1.1.4
## [25] purrr_1.2.0          readr_2.1.6          tidyr_1.3.1
## [28] tibble_3.3.0         ggplot2_4.0.1        tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] magrittr_2.0.3       multcomp_1.4-29      otel_0.2.0
##  [4] e1071_1.7-16         matrixStats_1.5.0    compiler_4.5.2
##  [7] reshape2_1.4.5       vctrs_0.6.5          crayon_1.5.3
## [10] pkgconfig_2.0.3      fastmap_1.2.0        labeling_0.4.3
## [13] utf8_1.2.6           promises_1.5.0       phosphoricons_0.2.1
## [16] rmarkdown_2.30       prodlim_2025.04.28   tzdb_0.5.0
## [19] bit_4.6.0            xfun_0.54            cachem_1.1.0
## [22] jsonlite_2.0.0       recipes_1.3.1        later_1.4.4
## [25] R6_2.6.1             coin_1.4-3           bslib_0.9.0
## [28] stringi_1.8.7        RColorBrewer_1.1-3   parallelly_1.45.1
## [31] rpart_4.1.24         jquerylib_0.1.4      Rcpp_1.1.0
## [34] knitr_1.50           future.apply_1.20.1  rclipboard_0.2.1
## [37] audio_0.1-11         httpuv_1.6.16        Matrix_1.7-4
## [40] splines_4.5.2        nnet_7.3-20          varImp_0.4
## [43] timechange_0.3.0     tidyselect_1.2.1     rstudioapi_0.17.1
## [46] yaml_2.3.10          timeDate_4051.111    codetools_0.2-20
## [49] listenv_0.10.0       plyr_1.8.9           shiny_1.12.1
## [52] withr_3.0.2          S7_0.2.1             evaluate_1.0.5
## [55] future_1.68.0        survival_3.8-3       proxy_0.4-27
## [58] pillar_1.11.1        DT_0.34.0            generics_0.1.4
## [61] vroom_1.6.7          rprojroot_2.1.1      hms_1.1.4
## [64] scales_1.4.0         globals_0.18.0       xtable_1.8-4
## [67] class_7.3-23         glue_1.8.0           tools_4.5.2
## [70] data.table_1.17.8    ModelMetrics_1.2.2.2 gower_1.0.2
## [73] measures_0.3         libcoin_1.0-10       ipred_0.9-15
## [76] nlme_3.1-168         cli_3.6.5            lava_1.8.2
## [79] gtable_0.3.6         sass_0.4.10          digest_0.6.39
## [82] TH.data_1.1-5        htmlwidgets_1.6.4    farver_2.1.2
## [85] htmltools_0.5.9      lifecycle_1.0.4      hardhat_1.4.2
## [88] shinyWidgets_0.9.0   mime_0.13            bit64_4.6.0-1
## [91] MASS_7.3-65
```